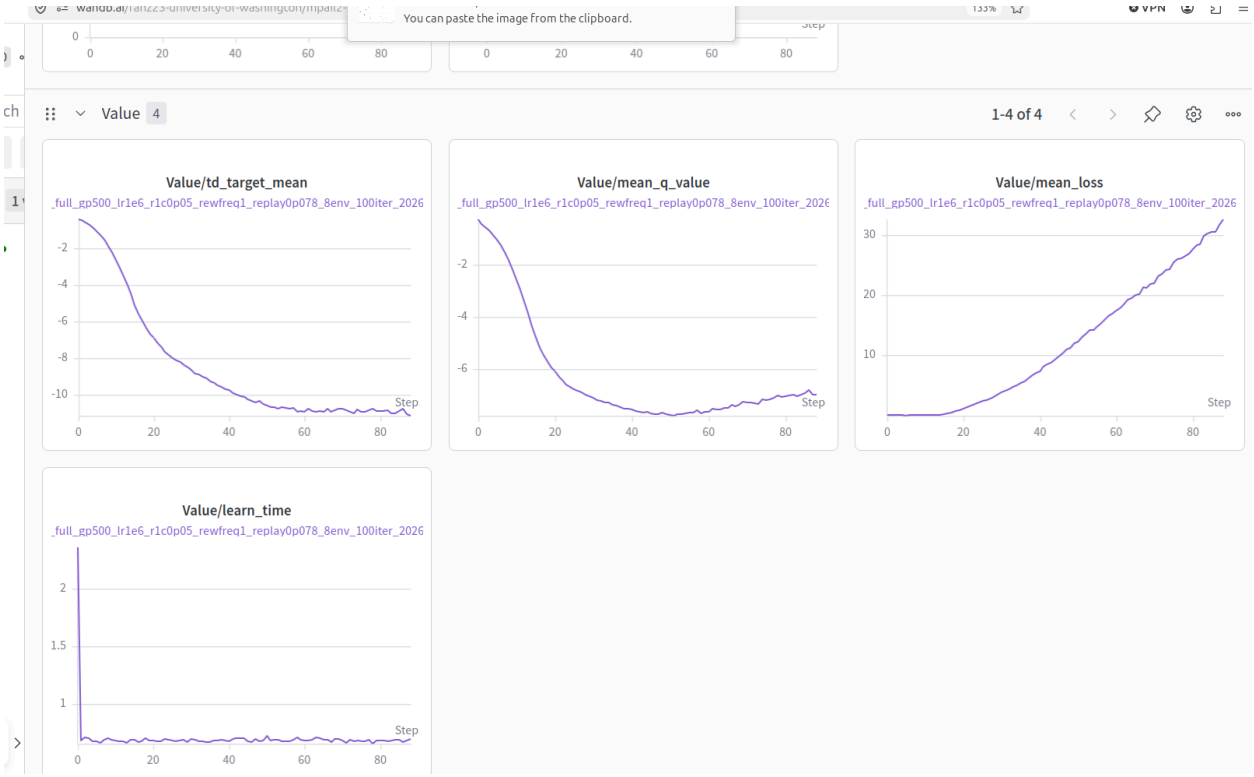
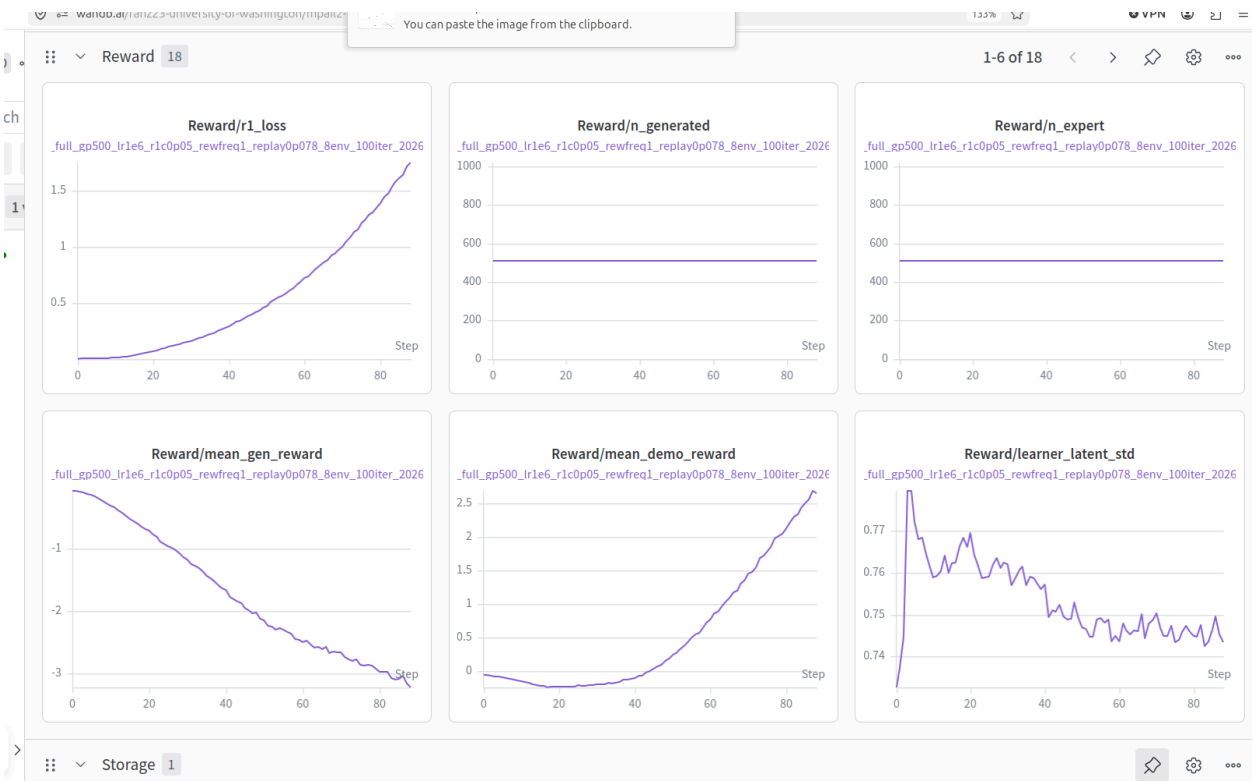
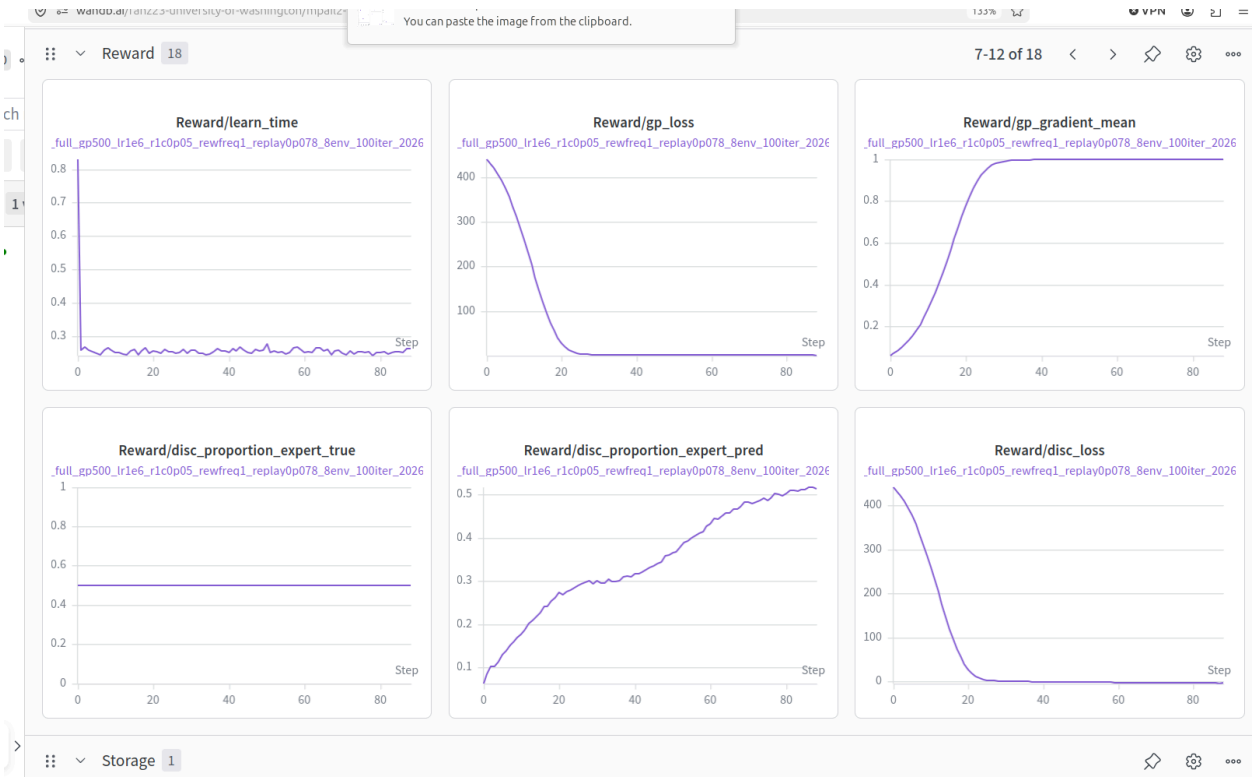
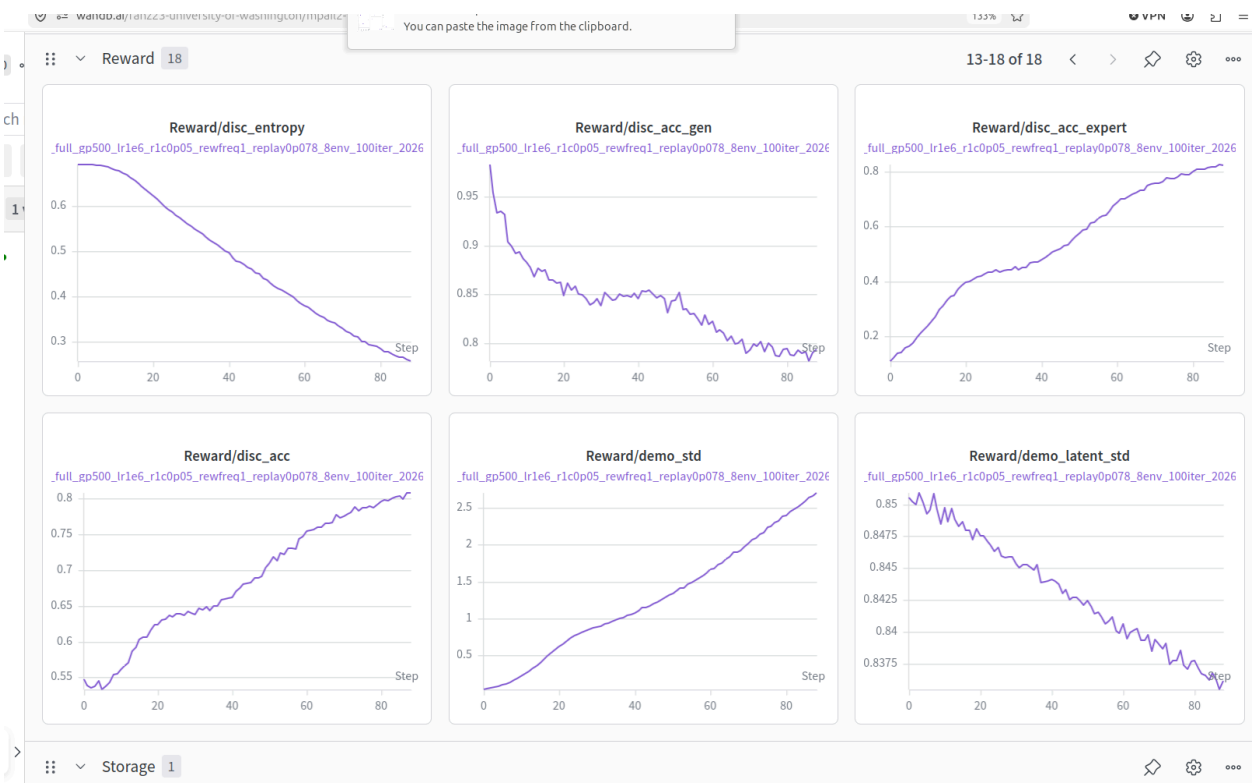
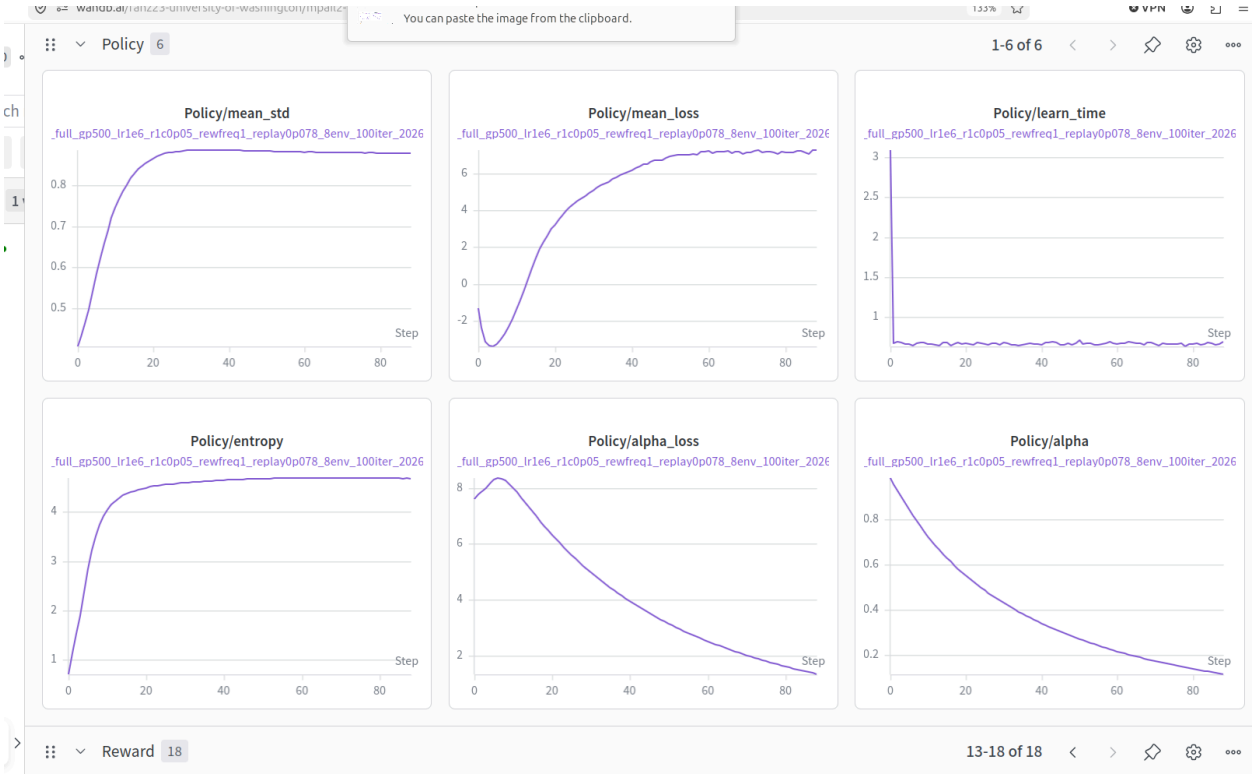
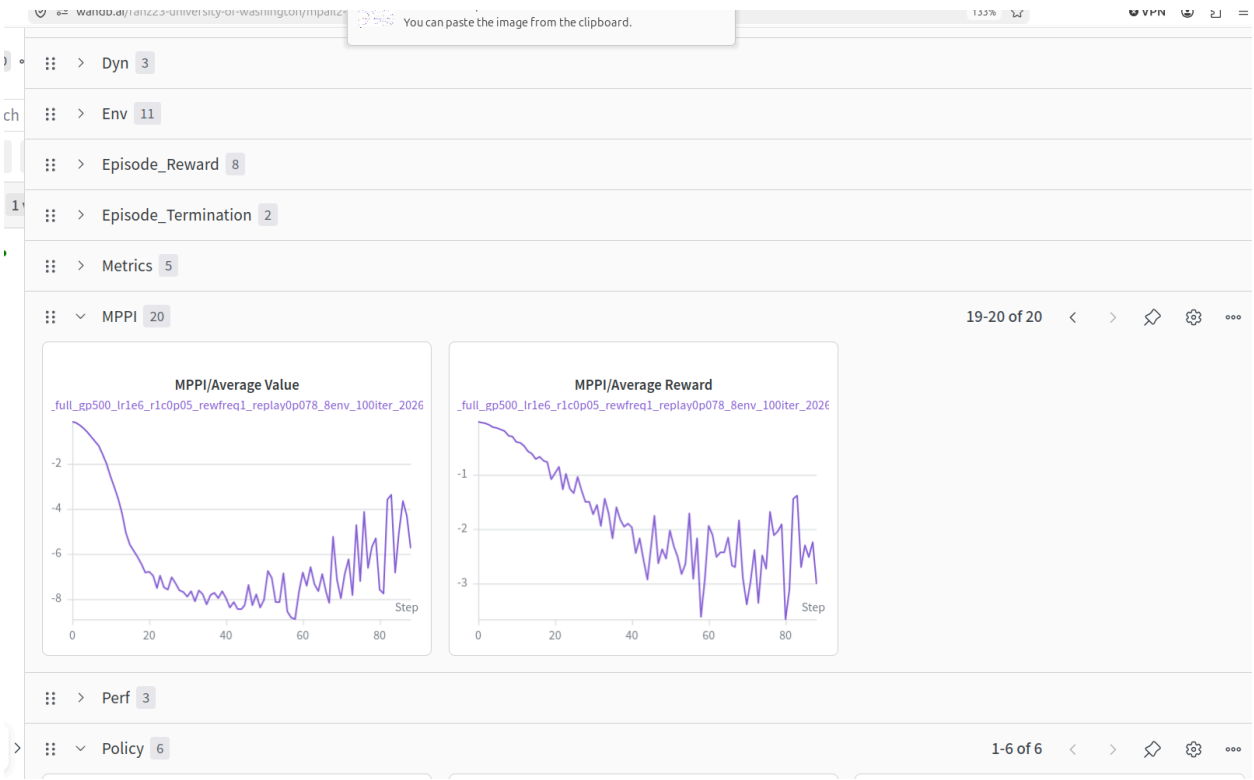

You can paste the image from the clipboard.
Value 4
1-4 of 4
Value/td_target_mean
Value/mean_q_value
Value/mean_loss
Value/learn_time
Step

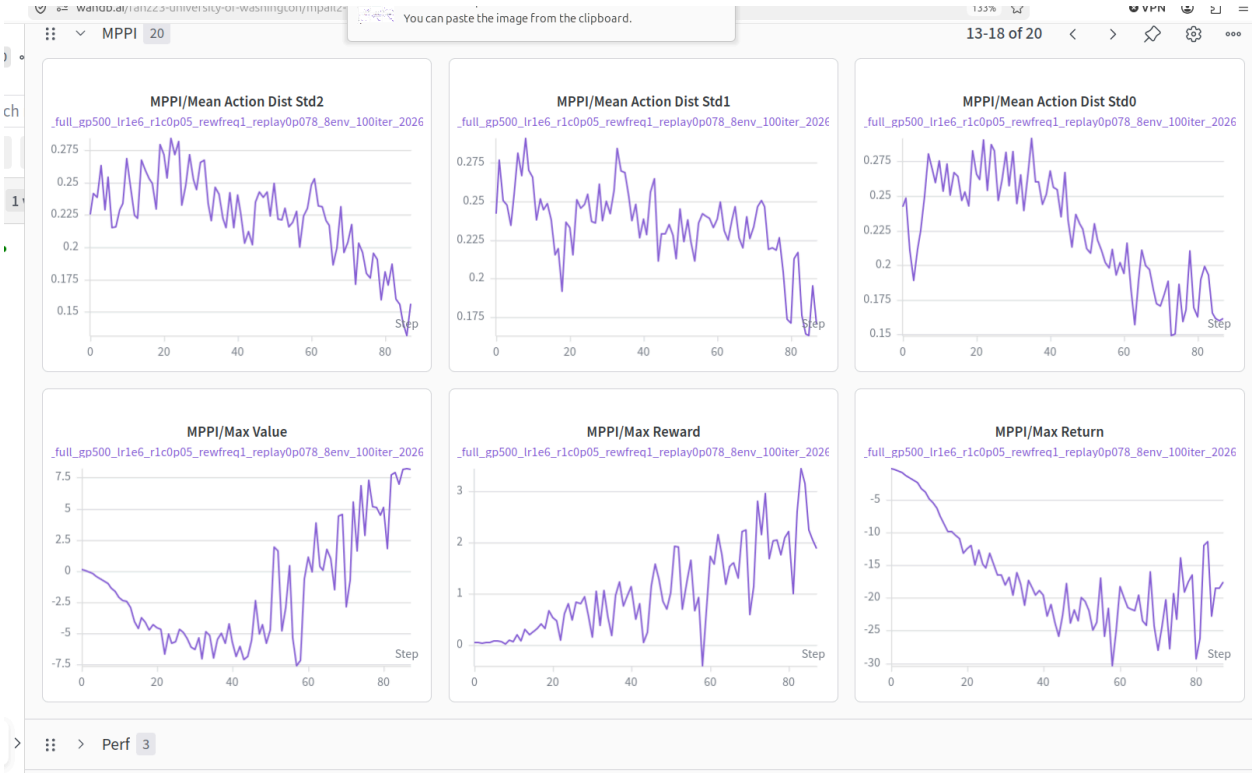

You can paste the image from the clipboard.
Reward 18
1-6 of 18
Reward/r1_loss
Reward/n_generated
Reward/n_expert
Reward/mean_gen_reward
Reward/mean_demo_reward
Reward/learner_latent_std
Step
Storage 1

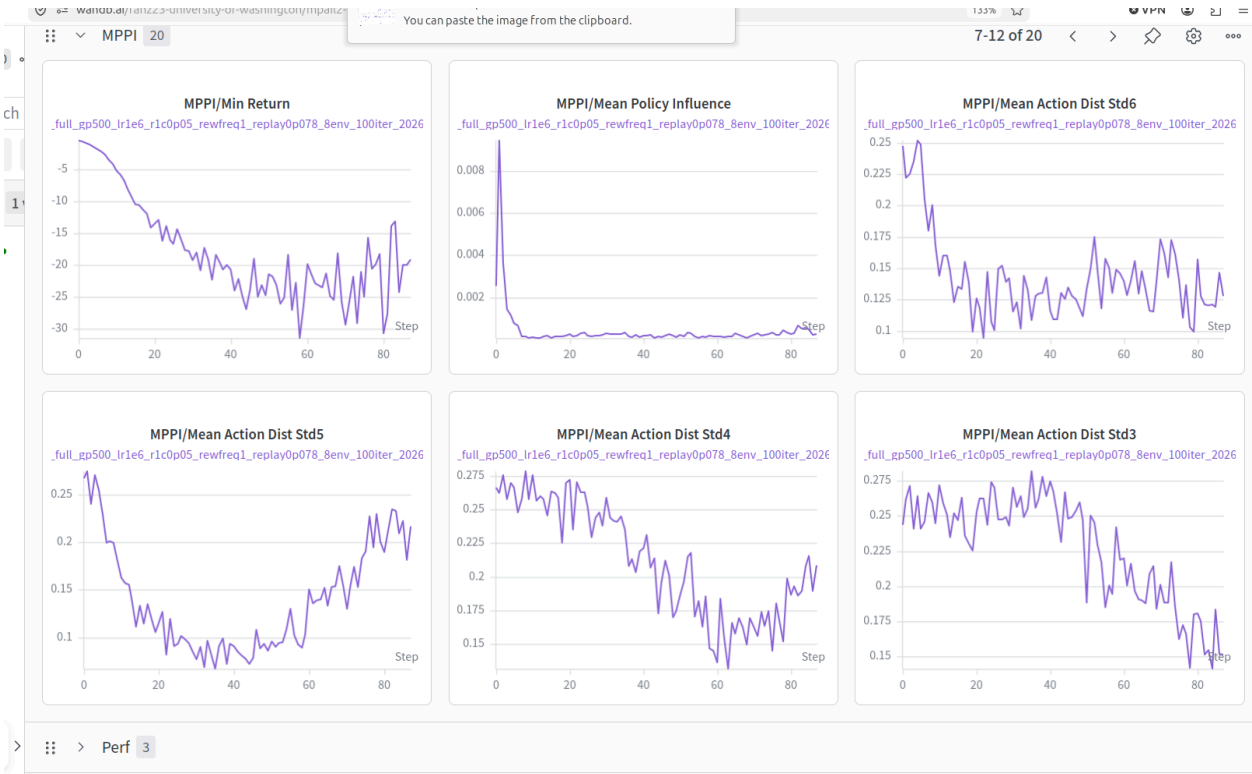

You can paste the image from the clipboard.
Reward 18
7-12 of 18
Reward/learn_time
Reward/gp_loss
Reward/gp_gradient_mean
Reward/disc_proportion_expert_true
Reward/disc_proportion_expert_pred
Reward/disc_loss
_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter_2026
Step
Storage 1

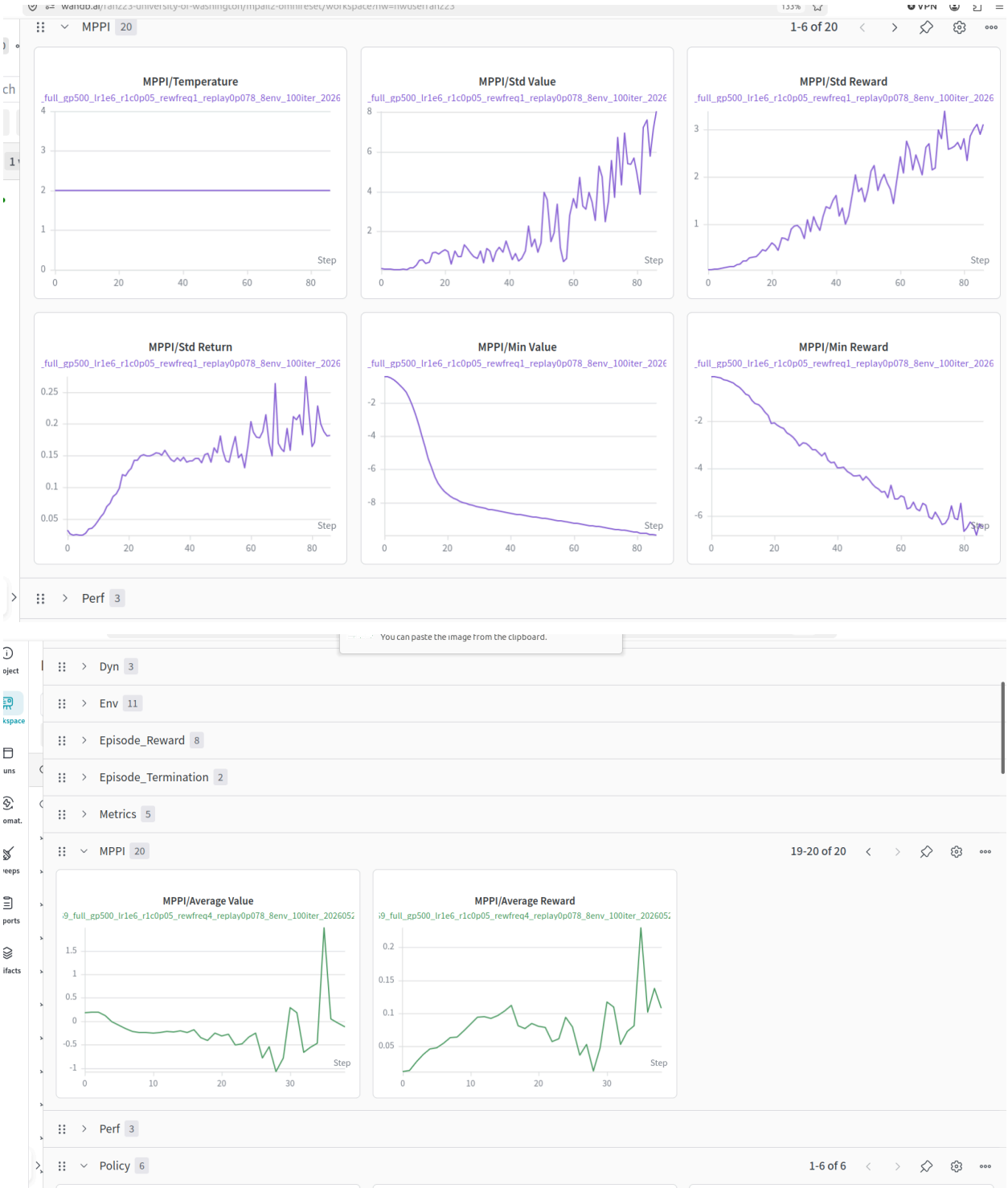

You can paste the image from the clipboard.
Reward 18
13-18 of 18
Reward/disc_entropy
Reward/disc_acc_gen
Reward/disc_acc_expert
Reward/disc_acc
Reward/demo_std
Reward/demo_latent_std
_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter_2026
Step
Storage 1


You can paste the image from the clipboard.
Policy 6
1-6 of 6
Policy/mean_std
Policy/mean_loss
Policy/learn_time
Policy/entropy
Policy/alpha_loss
Policy/alpha
Step
Reward 18
13-18 of 18


You can paste the image from the clipboard.
Dyn 3
Env 11
Episode_Reward 8
Episode_Termination 2
Metrics 5
MPPI 20
19-20 of 20
MPPI/Average Value
MPPI/Average Reward
Step
Perf 3
Policy 6
1-6 of 6

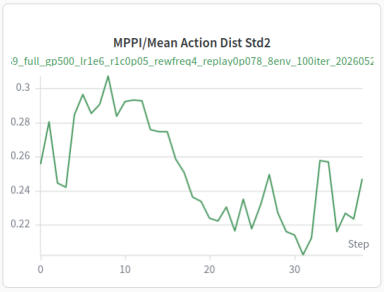

You can paste the image from the clipboard.
MPPI 20
13-18 of 20
MPPI/Mean Action Dist Std2
MPPI/Mean Action Dist Std1
MPPI/Mean Action Dist Std0
MPPI/Max Value
MPPI/Max Reward
MPPI/Max Return
_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter_2026
Step
Perf 3

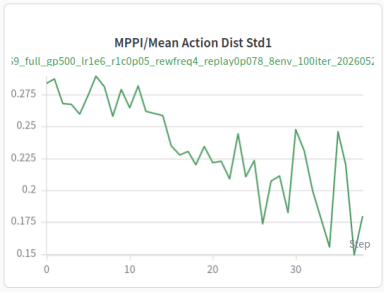

You can paste the image from the clipboard.
MPPI 20
7-12 of 20
MPPI/Min Return
MPPI/Mean Policy Influence
MPPI/Mean Action Dist Std6
MPPI/Mean Action Dist Std5
MPPI/Mean Action Dist Std4
MPPI/Mean Action Dist Std3
_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter_2026
Step
Perf 3

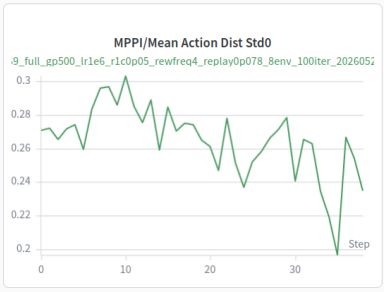

MPPI 20
1-6 of 20
MPPI/Temperature
MPPI/Std Value
MPPI/Std Reward
MPPI/Std Return
MPPI/Min Value
MPPI/Min Reward
_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter_2026
Step
Perf 3
You can paste the image from the clipboard.
Dyn 3
Env 11
Episode_Reward 8
Episode_Termination 2
Metrics 5
MPPI 20
19-20 of 20
MPPI/Average Value
MPPI/Average Reward
Perf 3
Policy 6
1-6 of 6

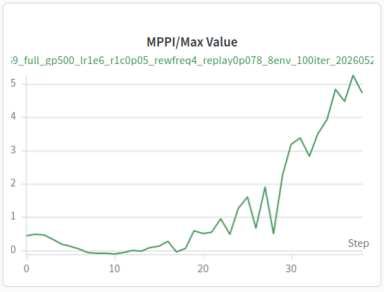
You can paste the image from the clipboard.

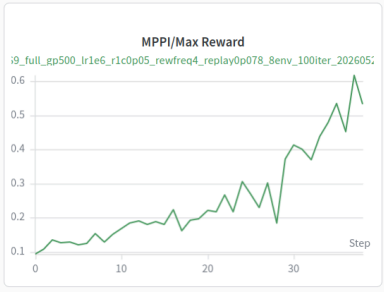
MPPI 20 13-18 of 20

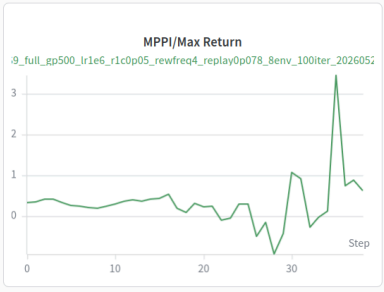








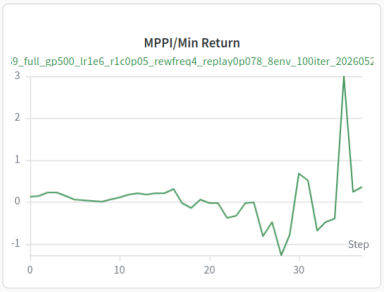


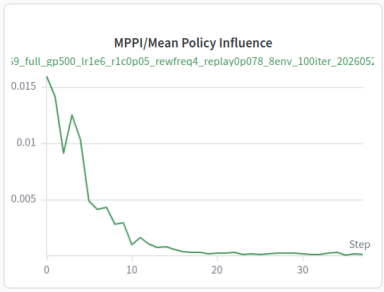


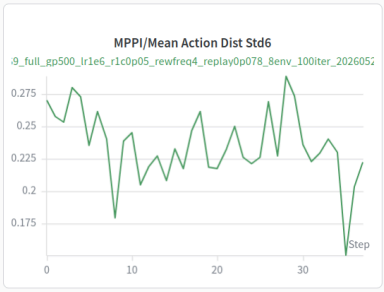
Perf 3

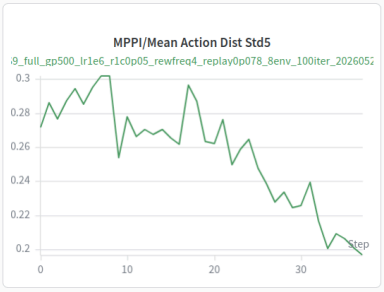
You can paste the image from the clipboard.

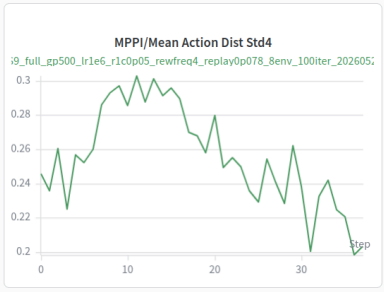
MPPI 20 7-12 of 20

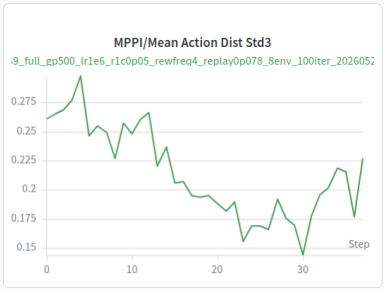








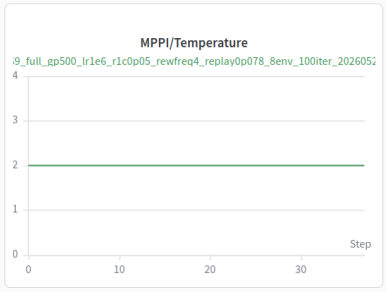


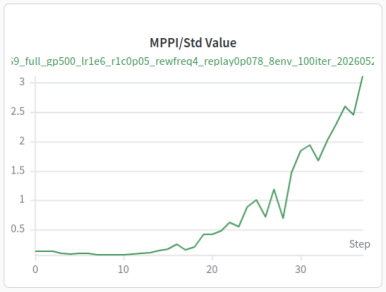


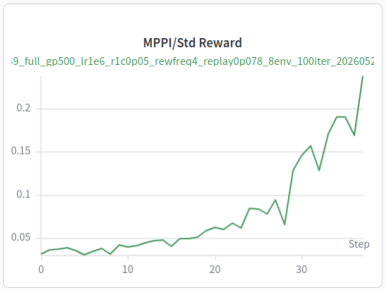
Perf 3

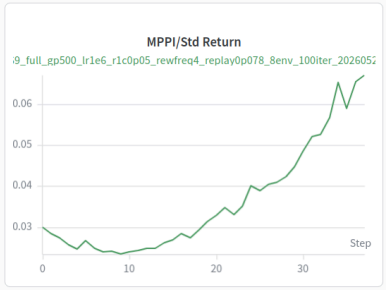
MPPI 20

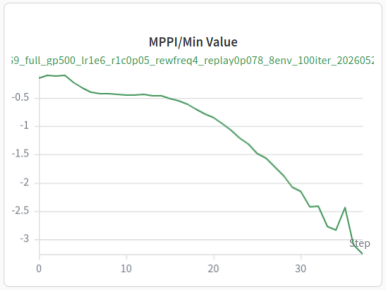
1-6 of 20

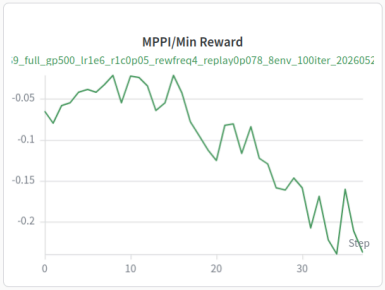










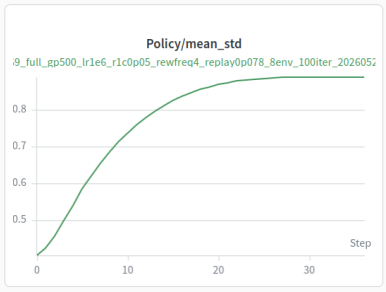


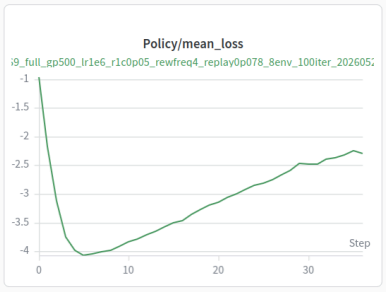
Perf 3

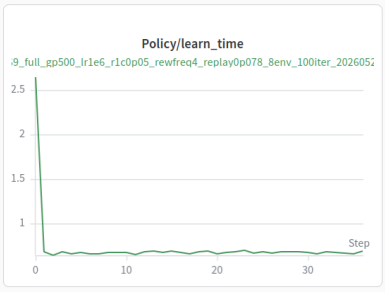
Perf 3

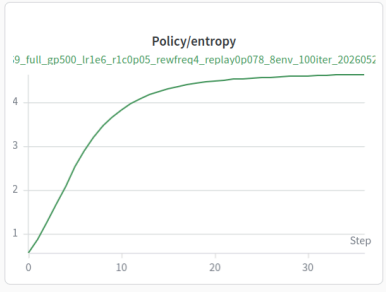
Policy 6

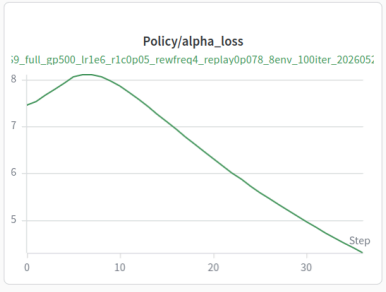
1-6 of 6

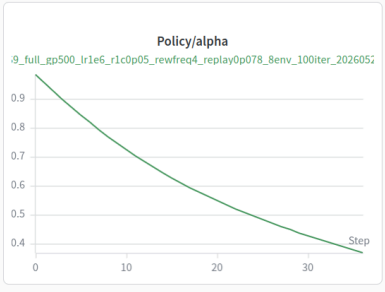


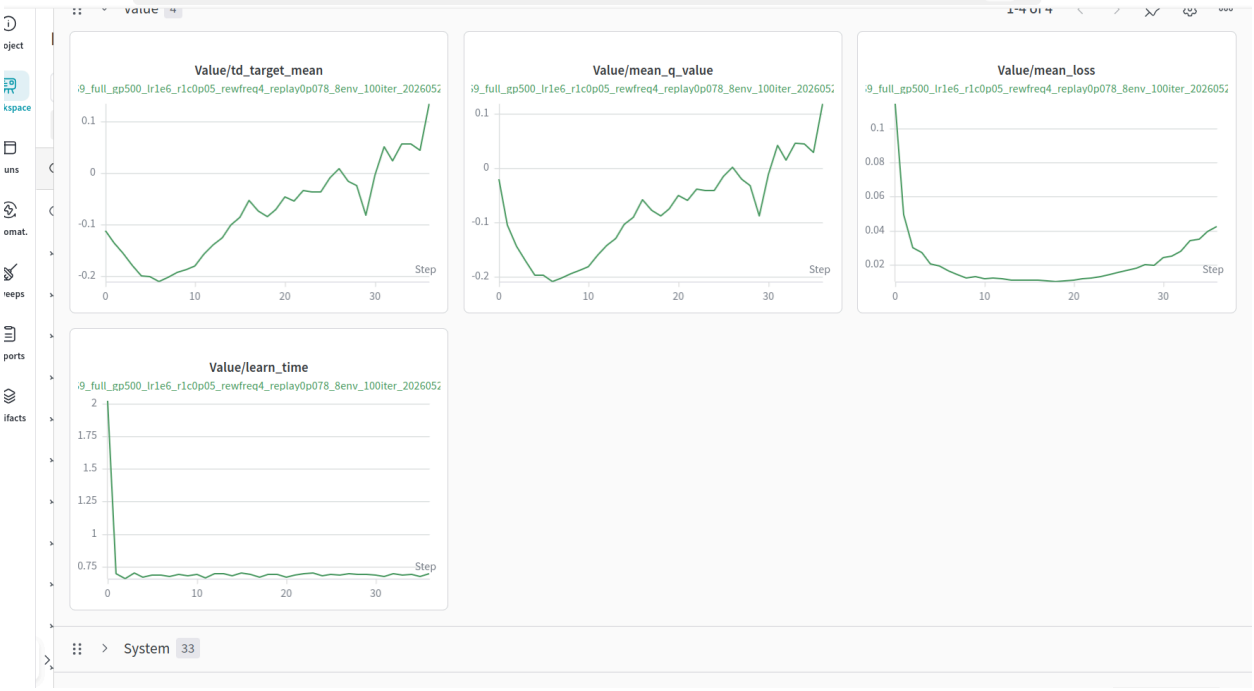


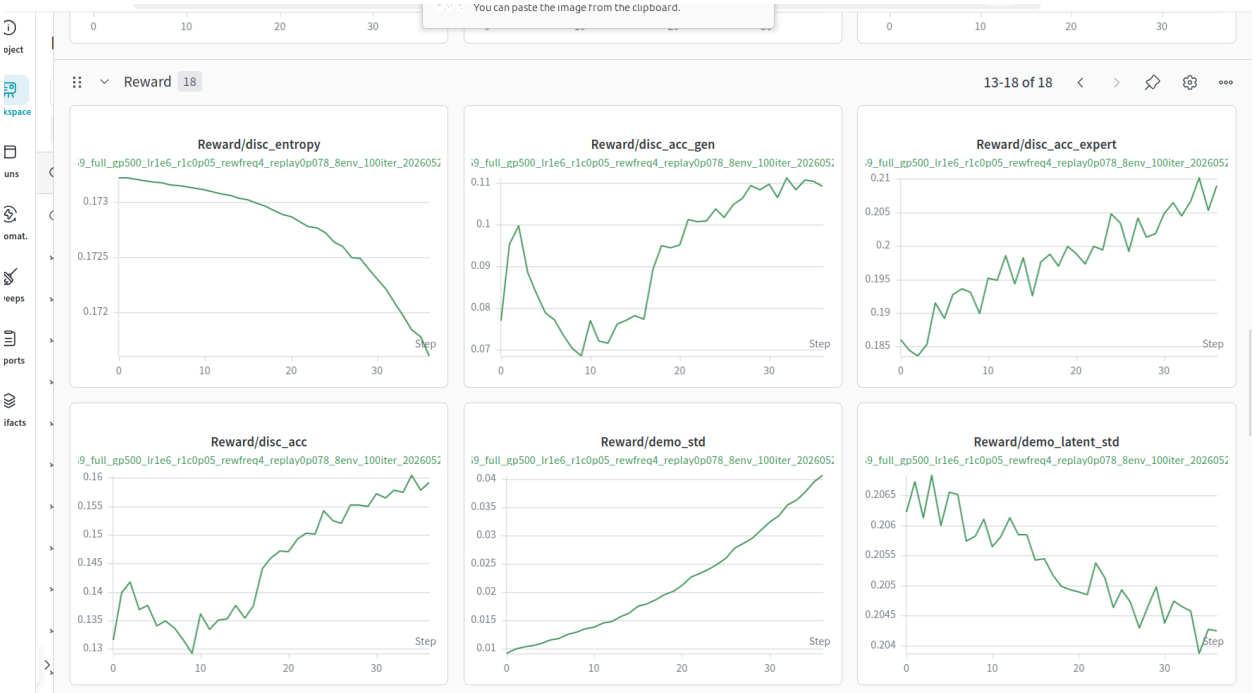






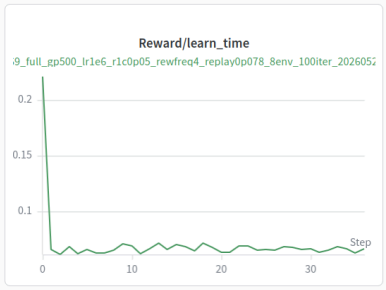


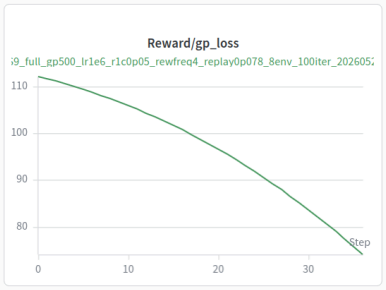

Value/td_target_mean
Value/mean_q_value
Value/mean_loss
Value/learn_time
Step
System 33

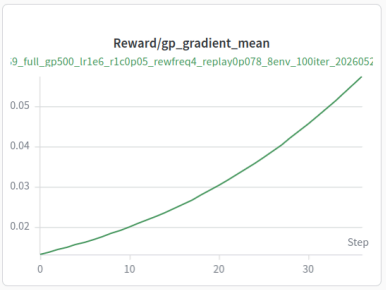

You can paste the image from the clipboard.
Reward 18
13-18 of 18
Reward/disc_entropy
Reward/disc_acc_gen
Reward/disc_acc_expert
Reward/disc_acc
Reward/demo_std
Reward/demo_latent_std
Step

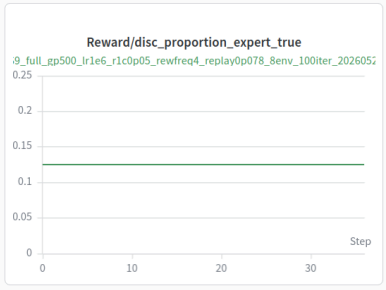
You can paste the image from the clipboard.

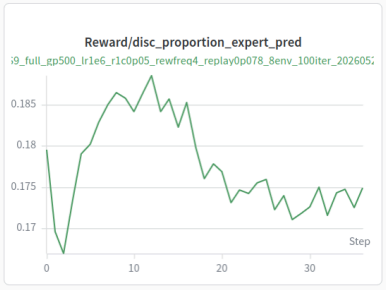
Reward 18 7-12 of 18

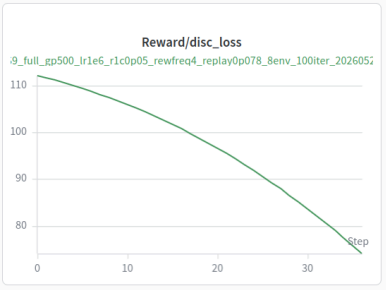




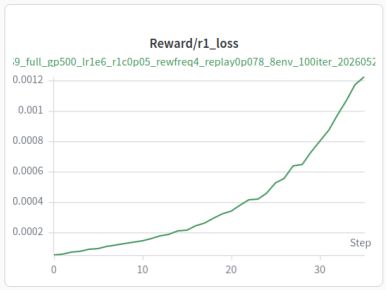






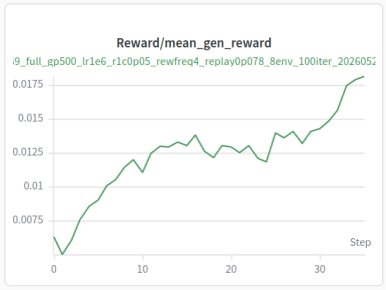


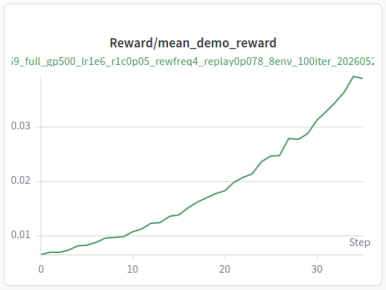
Reward 18 1-6 of 18







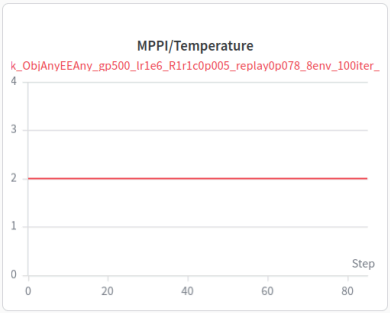


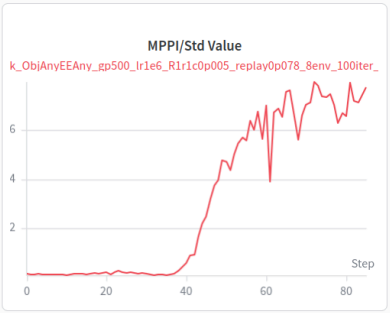


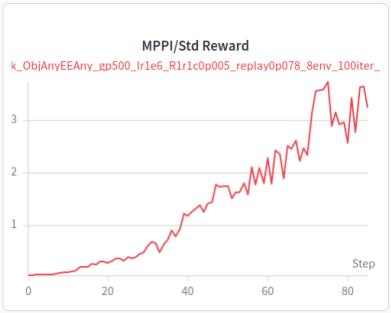


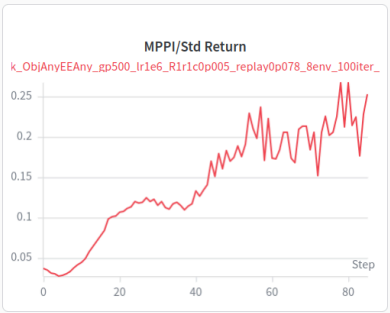
MPPI 20

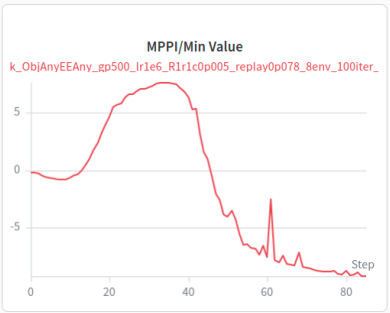
1-6 of 20

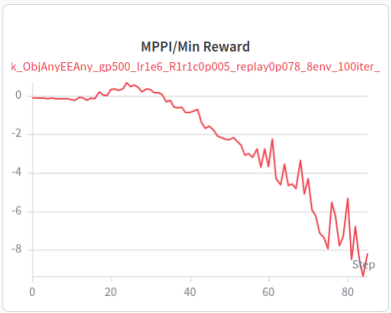








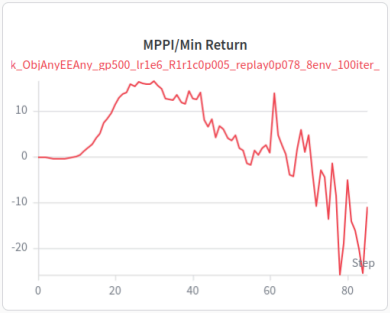


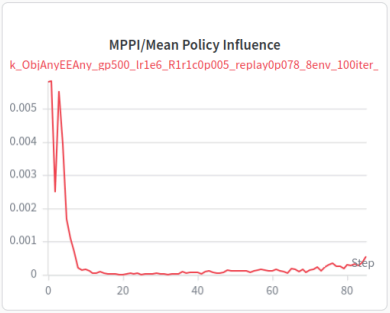


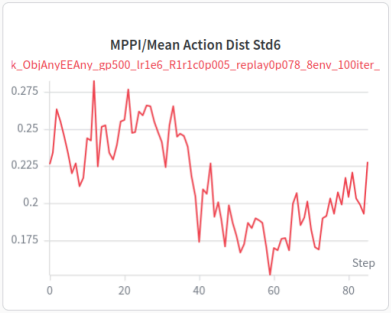
Perf 3

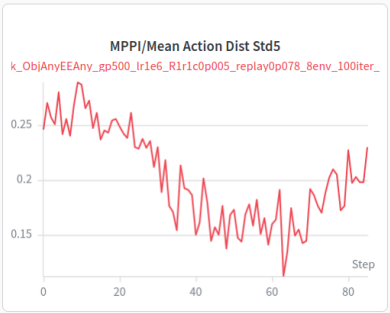
MPPI 20

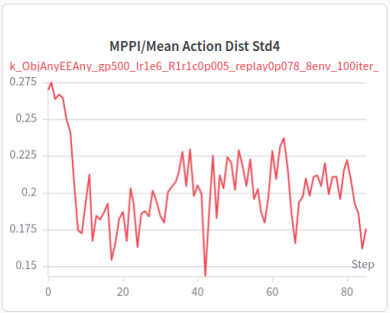
7-12 of 20

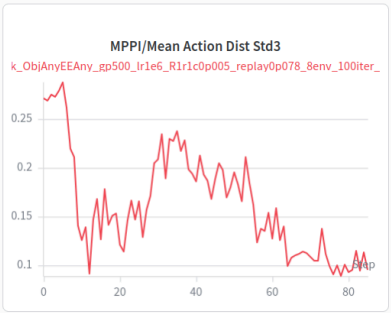






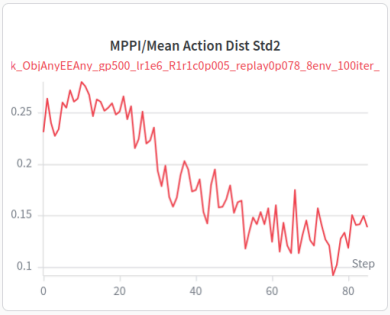


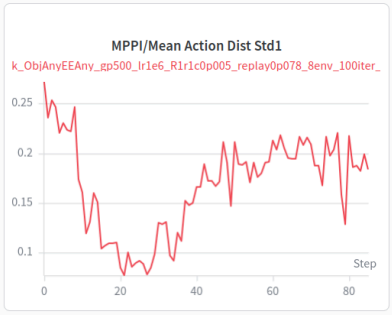


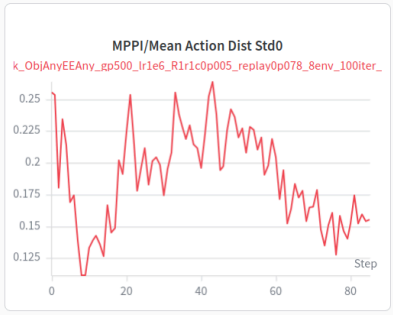


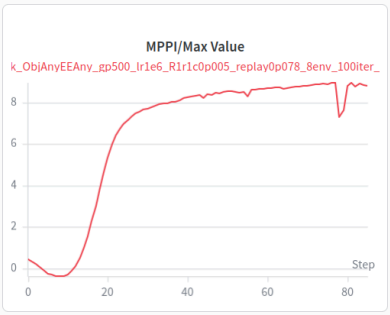
Perf 3

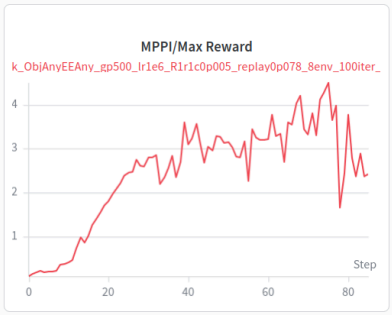
MPPI 20 13-18 of 20

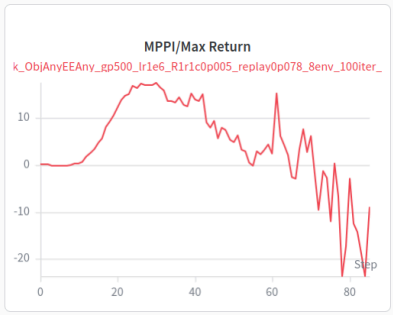












Perf 3

You can paste the image from the clipboard.

Dyn 3

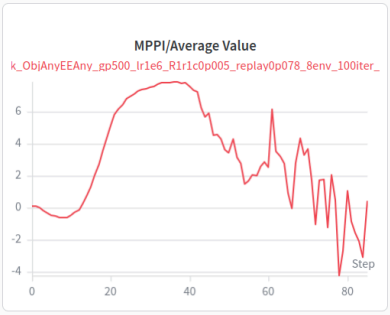
Env 11

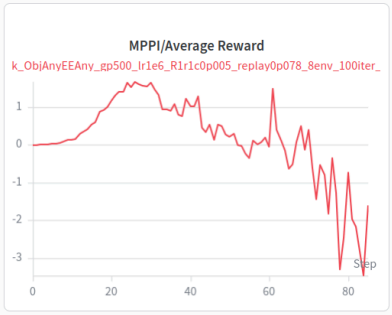
Episode_Reward 8

Episode_Termination 2

Metrics 5

MPPI 20 19-20 of 20



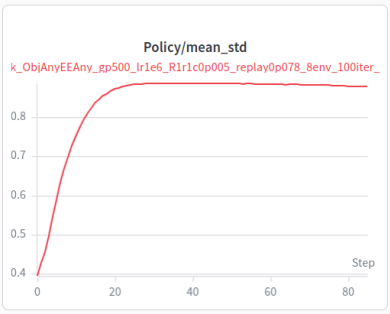


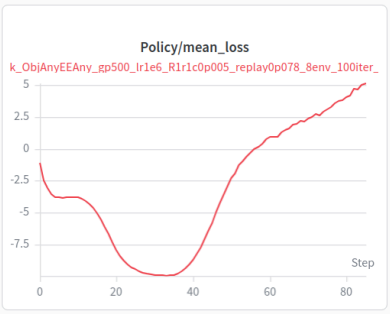
Perf 3

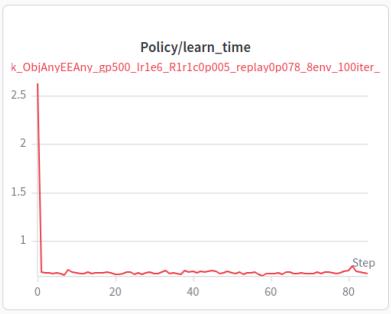
Policy 6 1-6 of 6

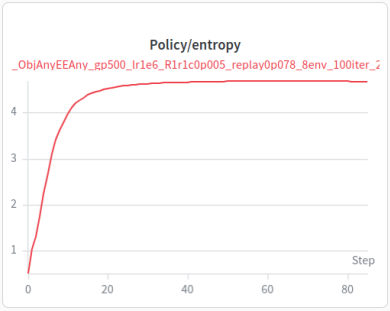
You can paste the image from the clipboard.

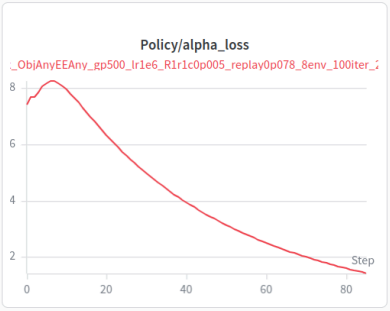
Policy 6 1-6 of 6

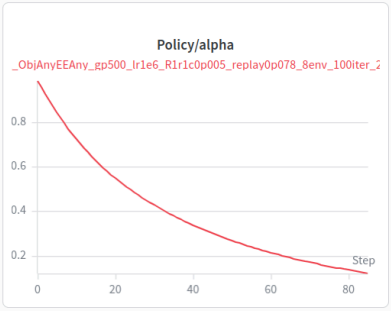








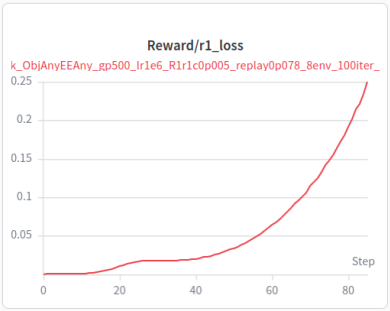


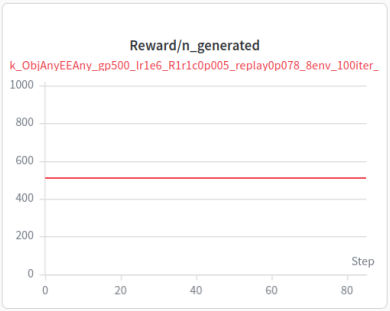


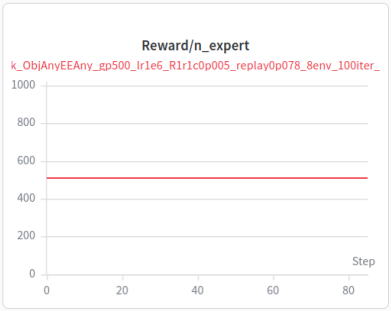
Reward 18 1-6 of 18

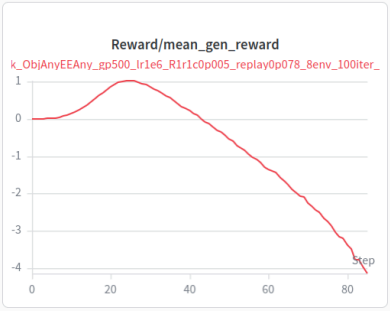
You can paste the image from the clipboard.

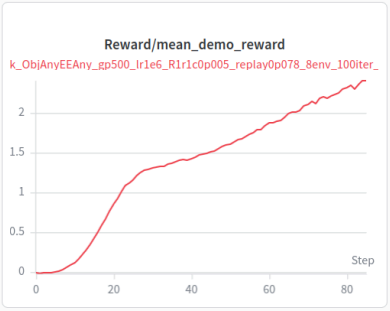
Reward 18 1-6 of 18









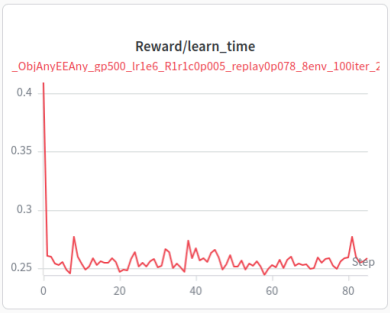


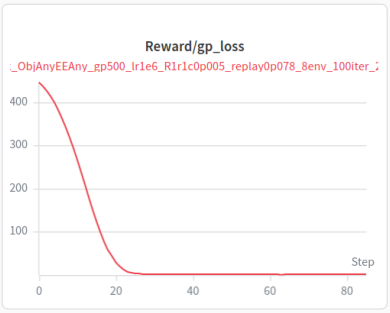


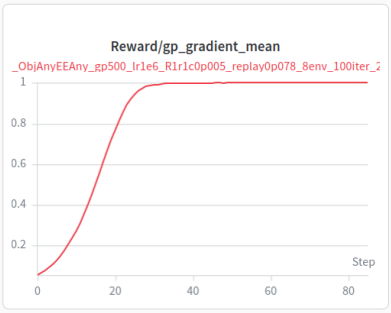
Storage 1

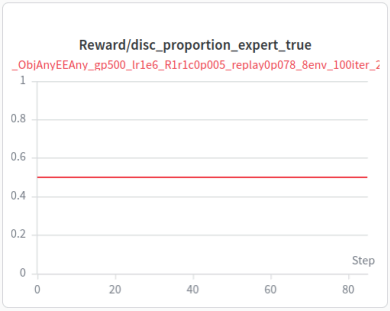
You can paste the image from the clipboard.

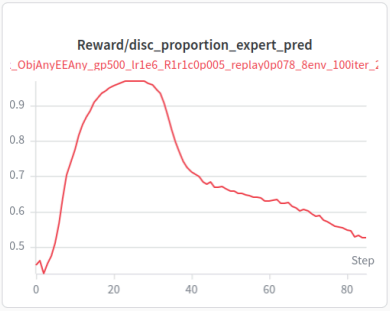
Reward 18 7-12 of 18

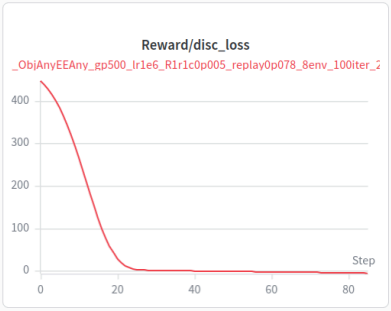






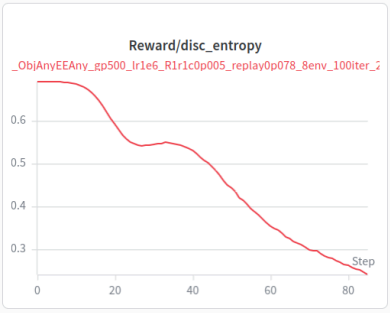


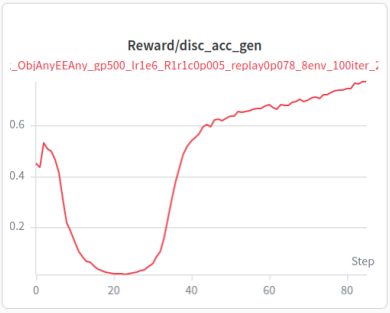


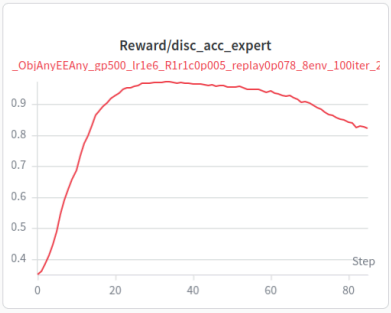


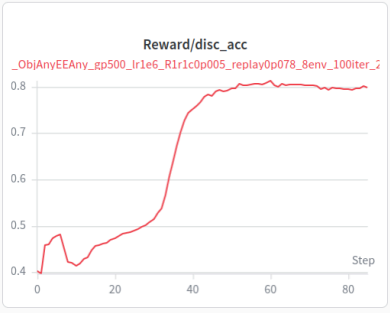
Storage 1

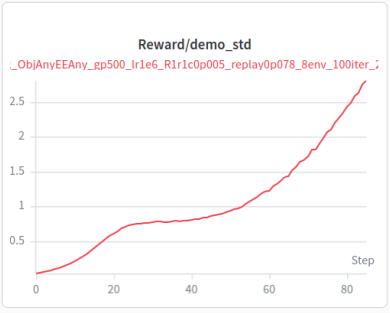
Reward 18 13-18 of 18

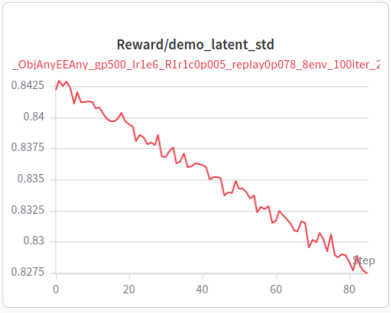




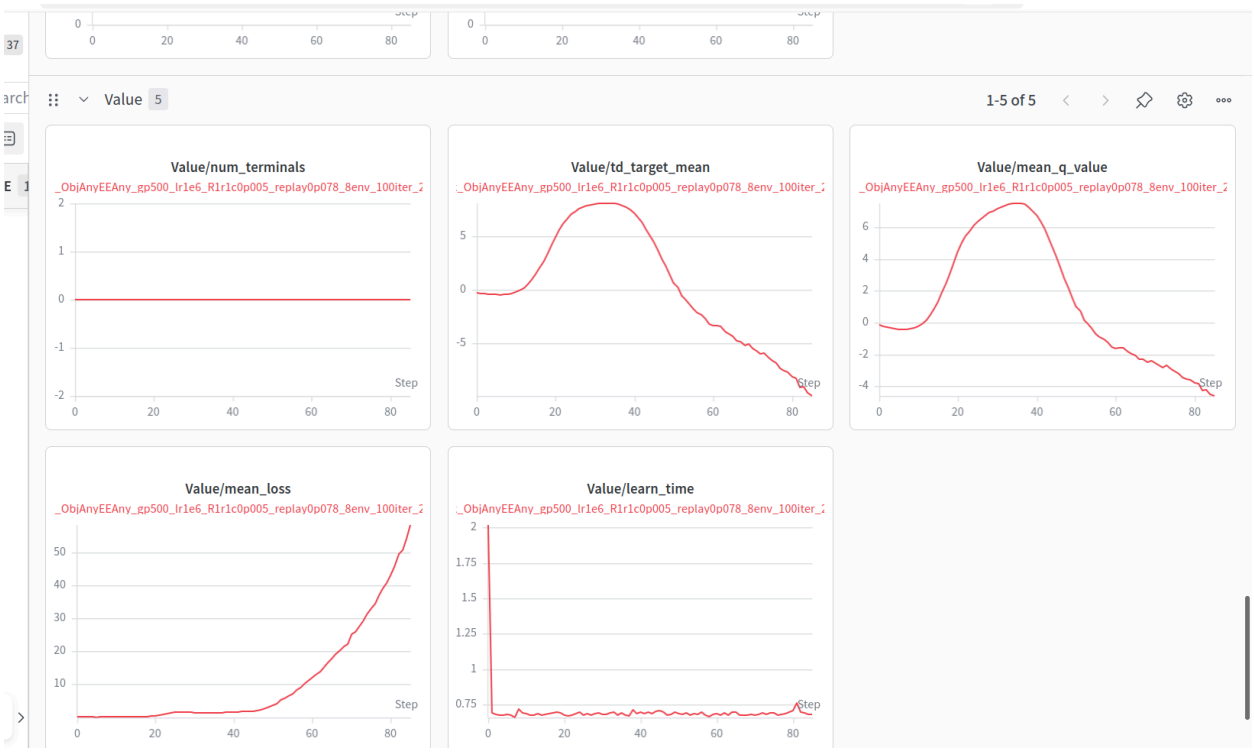








Storage 1



能不能看到我这些log的截图，现在好像效果都不好，能不能根据参数变化给我分析问题

告诉我健康的log应该怎样，逐个分析并且找原因，和接下来的方向
你能看到仓库里面上传wandb的logs都是什么吗，能不能告诉我比较重要的判断mpail训练成功正常的参数都是什么，正常的健康的mpail的logs应该是什么样子（结合每个上面的图表来讲），然后这些**4. Reward / Discriminator 是干嘛的** 这个东西不是 OmniReset env reward。它是 MPAIL 自己学出来的“像不像 expert”的评分器。它看的数据是：textexpert transition: obs, action, next_obsagent transition: obs, action, next_obs然后学：textexpert transition 分数高agent transition 分数低关键旋钮：reward learner lr reward 网络学习率。太高：reward/discriminator 很快把 expert 和 agent 分开，可能饱和，planner 只看到一堆极端分数，反而不知道“往哪边稍微好一点”。太低：reward 学得太慢，planner 没有有效信号。gradient penalty / gp_coeff 防止 discriminator 太尖、太爆、太容易过拟合。大一点：reward 更平滑，planner 更容易有连续方向感，但可能学得慢。小一点：reward 更锐利，但更容易饱和。你现在 OmniReset 已经是 gp_coeff=10.0，不算小，所以我不建议先继续加大。reward update ratio 每收一批新数据，reward 学多少次。学太多：reward 很快把 student 判死刑，agent 哪里都像错。学太少：reward 没学明白，planner 没方向。注意你代码里的 learner.replay_ratio 不是只管 reward，它会一起影响 dynamics/value/policy 更新次数。whether discriminator saturates too fast 这个是诊断，不是一个直接参数。看 W&B 里如果：textReward/mean_demo_reward 很快变很高 Reward/mean_gen_reward 很快变很低说明 discriminator 可能太快把 expert 和 agent 分开了。你之前的图里有这个倾向：demo reward 往 60+ 走，generated reward 到 -30~-40。这说明 learned reward 可能变成“只会骂学生不像”，而不是告诉 planner 怎么逐步变好。**5. Planner / Dynamics 是干嘛的** 这部分是 MPAIL 真正选动作的地方。num_rollouts planner 每一步在脑子里试多少条未来动作。大：更可能找到好动作，但慢。小：快，但可能根本没采到“往孔里插”的动作。你现在常用 256。可以试 512。num_elites 从 rollouts 里挑几个最好的来更新计划。太少：很抖，容易被偶然假高分骗。太多：平均掉好动作，动作变保守。如果 num_rollouts=512，可以用 num_elites=64。planning horizon / num_timesteps planner 往未来看几步。你现在大概是 7，env step 约 0.1s，也就是只看 0.7 秒。peg-hole 距离如果还有十几厘米，0.7 秒可能太短，它看不到“先靠近再插入”的好处。可以试 10 或 14。但太长 dynamics 会不准。temperature planner 对好动作有多“贪心”。高：动作更平均、更保守。低：更相信 top rollouts，动作更果断，但可能不稳。现在 2.0，如果 Tyler 说 movement 少，可以试 1.5 或 1.0。dynamics learning rate 小地图学习率。如果 dynamics 没学好，planner 在错误地图里做计

划，动作看起来就会莫名其妙。但你现在的 dyn loss 不一定明显爆炸，所以它不是第一个改的。value learning rate value 是估计“这个状态后面还有多少希望”。如果 value/Q 曲线大幅漂移、loss 爆，就要降。但它也不是第一优先。replay_ratio / update count 这个很重要。你 OmniReset config 里注释写得很清楚：原本想让每个 iteration 大概做 100 次更新。但你常常用 NUM_ENVS=8 跑，而配置里 replay_ratio=100/(32*160) 是按 32 env 算的。所以 8 env 时实际大概只有：text8 * 160 * 0.0195 ≈ 25 updates / iter这比预期少 4 倍。也就是说 dynamics/reward/value/policy 都可能学不够。如果继续用 8 env，应该试：textlearner.replay_ratio=0.078125这样回到大约：text8 * 160 * 0.078125 = 100 updates / iter参数详细具体是怎么作用在mpail算法训练过程中的，改变这些参数会对训练造成什么效果，这些效果在训练里面起什么作用，在动作表现上可能是什么效果，在logs表现上可能是什么效果？变动哪些参数可能通过什么逻辑对logs的哪个表的曲线造成怎么样的变化

Sunday 7:41 AM

Worked for 2m 35s

能看到。结论先说：这些图大多不是“健康收敛”，而是 reward/discriminator 在变强，但 planner/value 没把这个信号变成越来越好的动作。最危险的共同症状是：`MPPI/Average Reward`、`MPPI/Average Value`、`MPPI/Return` 后期变差或大幅抖动，同时 `Value/mean_loss` 往上炸。这说明 MPAIL 内部学到的地图和评分器在强化，但动作选择没有稳定改善。

我也看了仓库里的本地 W&B 目录，主要在 `logs/*/wandb/run-*` 和 `offline-run-*`。能看到 `config.yaml`、`wandb-summary.json`、`output.log`、`.wandb` 二进制历史文件。关键代码位置是：mpail2/learner.py (line 450)、mpail2/reward.py (line 12)、mpail2/planner.py (line 354)。

重要修正

你说 reward/discriminator 看 `obs, action, next_obs`，但当前 `mpail2` 代码里 Reward 网络实际上只吃 `state, next_state`，action 参数被忽略了：

`Reward.forward(state, next_state, action=None)` 里输入是 `torch.cat([state, next_state])`。

所以现在 discriminator 判断的是“状态转移像不像 expert”，不是“状态-动作-状态转移像不像 expert”。这会让它更容易学成“状态分布分类器”，尤其 full-task 时，agent 状态分布离 expert 太远，它很快只会判别，不会给 planner 连续方向。

健康的 MPAIL Log 应该怎样

`Reward/mean_demo_reward` 应该高于 `Reward/mean_gen_reward`，但不要很快拉开成巨大差距。健康形态是：demo 稍高、gen 逐渐上升靠近 demo，或者 gap 稳定不爆。坏形态是 demo 一路上升、gen 一路下降，说明 discriminator 只会说“student 不像”。

`Reward/disc_acc` 不应该太快接近 0.9-1.0。早期 0.55-0.75 比较健康；0.8+ 可以接受但要看 MPPI 是否同步变好。如果 `disc_entropy` 快速掉到很低，通常是饱和。

Reward/gp_gradient_mean 健康值接近 1。你的 gp=500 / 5000 / 10000 相关 run 里很多都能把它压到接近 1，这部分不算主要问题。gp_loss 从很大下降是正常的。

MPPI/Average Reward、MPPI/Max Reward、MPPI/Average Value 应该整体变好，至少 best rollout 的 Max Reward/Max Value 变好时，Average 不应该持续坍塌。健康 planner 是：max reward 上升，average reward 慢慢上升或稳定，std 不爆炸。

MPPI/Std Reward、MPPI/Std Value 太大通常不是好事，它表示 rollout 评分差异越来越极端，planner 会被少数假高分骗。你的红色/紫色 run 里都有这个味道。

Value/mean_loss 健康应该下降或保持小幅震荡。你图里紫色升到 30+，红色升到 50+，这是明显不健康：value 学的 target 和 Q 在漂移。

Policy/mean_std、Policy/entropy 上升到高位不一定坏，但如果 MPPI reward/value 同时变差，它表示 policy 在变得更随机或更宽，而不是学到有效先验。

逐组分析

紫色这组，名字像 full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_100iter。

问题：Reward/mean_demo_reward 从接近 0 到 2.6，mean_gen_reward 到 -3 左右，disc_acc 到约 0.8，disc_entropy 持续下降。说明 discriminator 开始明显分开 expert/student。与此同时 Value/mean_loss 单调涨到 30+，td_target_mean 和 mean_q_value 一路变负，MPPI/Average Reward 和 Average Value 也往负走。

判断：reward 信号变“硬”了，但 planner/value 没吃进去。主要不是 gp 不够，而是 full-task 下 agent 数据离 expert 太远，reward 给的梯度方向不连续。

绿色这组，名字像 S9_full ... r1c0p05 rewfreq4。

注意：reward_update_every=4 时，代码把 reward stats 只累加 1/4 次，但最后除以全部 actual_updates，所以 Reward/disc_acc、n_expert、reward magnitude 看起来会被压低约 4 倍。这个图不能直接和 rewfreq1 比绝对值。

从 MPPI 看，早期 Average Reward/Value 有一点改善，但后期抖，Mean Policy Influence 很快接近 0，说明 planner 基本不再受 learned policy prior 影响，更多靠 MPPI sampling 硬搜。

判断：reward 学慢一点后没那么快饱和，但动作改善仍不稳。rewfreq4 不是根治，只是把问题放慢。

红色这组，名字像 FullTask_ObjAnyEEAny_gp500_lr1e6_R1r1c0p005。

这是最像“full-task reset/demo mismatch”的失败。MPPI/Average Value 先升到很高，然后坍塌到负；Max Return 也先好后坏；Std Value/Reward 后期爆大；Value/mean_loss 冲到 50+。Reward 里 demo 持续升、gen 后期大幅掉到 -4，disc entropy 下降。

判断：早期 planner 找到一些假高分轨迹，后期 value/reward 把空间拉得越来越极端，MPPI 被误导。full-task 比 Route-A 难很多，因为随机初始/抓取阶段让 agent 访问的状态完全不像插入 expert 段。

参数怎么影响曲线

`reward_learner_cfg.opt_params.lr`:越大，`mean_demo_reward - mean_gen_reward` 拉开越快，`disc_acc` 上升越快，`disc_entropy` 掉越快。太大时 MPPI 的 `Std Reward/Value` 会涨，Value loss 容易爆。

`gp_coeff`:让 reward 对 transition 的梯度接近 1，更平滑。太小会尖锐饱和；太大则 reward 学得慢、数值被压住。你现在 gp=500 已经很强，不建议优先继续加。

`r1_coeff`:惩罚 reward logits 绝对值，防止 demo/gen 分数无限拉开。`0.005` 偏轻，`0.05` 更强，但你 purple/green 说明 r1=0.05 只能缓解饱和，不能解决 full-task 分布差。

`reward_update_every`:越大，reward 更新越少，discriminator 学慢。注意现在 W&B reward 指标会被平均稀释，所以 rewfreq4 的图要乘回约 4 倍理解。

`replay_ratio`:决定每轮对 dynamics/reward/value/policy 更新多少次。8 env、160 step 时，`0.078125` 大约 100 updates/iter；`0.0391` 约 50；`0.0195` 约 25。太低学不够，太高在坏数据上过拟合更快。

`num_rollouts/num_elites`:rollouts 越多越能搜到好动作；elites 太少会被假高分骗，太多会平均掉。512/64 是合理下一步，1024/128 可用于确认是不是采样不足。

`num_timesteps/horizon`:7 步只看约 0.7s，full-task/远距离靠近通常太短。h=10 可以试，h=14 风险是 dynamics error 累积。

`temperature`:越低越相信 top rollout，动作更果断也更容易被骗；越高更保守。现在 reward/value 不稳时，不建议先降到 1.0，容易放大假高分。

我建议下一步

先不要继续 full-task 硬扫。先把任务退回“object already grasped / Route A insertion”确认能学出健康 log：`Env/success_rate` 或 `end_of_episode_success_rate` 至少有非零趋势，`MPPI/Average Reward` 不再持续变差，`Value/mean_loss` 不爆。

最值得跑的诊断组合：

```
Route-A / already-grasped
gp_coeff=500
reward_lr=1e-6 或 2e-6
r1_coeff=0.05
reward_update_every=1
replay_ratio=0.078125
num_rollouts=512
num_elites=64
horizon=10
temperature=2.0
```

如果 Route-A 健康，再进 full-task。full-task 不要直接用插入段 expert 训练全流程，应该分阶段：先 grasp/approach，再 insertion，或者确保 demo 和 reset 分布一致。现在这些图最大的根因不是“某个曲线不好

看”，而是 expert transition 分布和 agent 初始探索分布差太远，reward 学成分类器，value/planner 被极端 learned reward 带偏。

Sunday 7:43 AM

Sunday 7:43 AM

参数	🔴 FullTask (r1=0.005, rewfreq=1)	🟢 S9 (r1=0.05, rewfreq=4)	🟣 S9b (r1=0.05, rewfreq=1)
任务	Full	Full	Full
Demo	full_object_anywhere	full_object_anywhere	full_object_anywhere
迭代数	100	100	100
num_envs	8	8	8
replay_ratio	0.078125	0.078125	0.078125
reward_lr	1e-6	1e-6	1e-6
gp_coeff	500	500	500
r1_coeff	**0.005**	**0.05**	**0.05**
reward_update_every	**1**	**4**	**1**
num_rollouts	512	512	512
num_elites	64	64	64
num_timesteps	7	7	7

变量只有两个：r1_coeff（0.005 vs 0.05）和 reward_update_every（1 vs 4）。S9b 是单独隔离 r1_coeff 效果的对照组，S9 是两个变量同时改的。

我现在只能fulltask,告诉我根据详细的log分析，这些参数怎么影响了训练，下一步怎么做

Sunday 7:53 AM

Worked for 1m

可以，只能 full-task 的话，下一步核心不是“让 discriminator 更会分 expert/student”，而是 让 full-task 的 learned reward 慢一点、平一点，同时让 planner 看得更远、更不容易被假高分骗。

你这三组里最有用的结论是：

Run	变化	结果判断
红色 FullTask `r1=0.005, rewfreq=1`	R1 太弱，reward 每轮都更新	discriminator 很快拉开 demo/gen，后期 agent reward 变差，abnormal 增多，86 iter 被 kill
绿色 S9 `r1=0.05, rewfreq=4`	R1 加强，reward 更新少 4 倍	reward 没那么快饱和，前 20-30 iter 环境表现最好，但后面学得太慢/信号太弱，63 iter 被 kill

紫色 S9b `r1=0.05, rewfreq=1`	只加强 R1，reward 每轮更新	完整跑完 100 iter，position 有改善，但 discriminator 还是分离过强，value/MPPI 后期不稳，success=0

参数怎么影响了训练

`r1_coeff`：你从 `0.005` 提到 `0.05` 是对的。红色里 `disc_demo` 到 80 多 iter 已经约 `2.4`，`disc_gen` 到 `-4` 左右，分离太快；S9b 最后 `mean_demo_reward=3.40`、`mean_gen_reward=-3.62`，还是分离，但比红色早期爆炸稍缓。R1 的作用是压住 reward logit 幅度，防止 reward 只会“骂学生不像 expert”。

`reward_update_every`：绿色 `rewfreq=4` 把 reward 学习速度砍到约 1/4，所以前期 `Env/rew` 能到 `1.3` 左右，比另外两组更像“有动作改善”。但它的问题是 reward 信号太慢，MPPI/value 没有足够稳定目标，后面开始漂。注意代码里 rewfreq=4 的 W&B reward 数值会被平均稀释，所以不能直接和 rewfreq=1 比绝对值。

`gp_coeff=500`：现在不是主要瓶颈。三组里 `gp_gradient_mean` 都能往 1 附近走，说明 GP 在工作。继续加 GP 只会让 reward 更慢，不一定救 full-task。

`replay_ratio=0.078125`：8 env 下约 100 updates/iter，这个是合理的。问题不是“学得太少”，而是 reward/value 在 full-task 分布差很大的数据上把错误方向学得很坚定。

`num_timesteps=7`：full-task 明显偏短。7 步更像“局部修正”，但 full-task 需要先靠近、抓稳、对齐、插入。短 horizon 看不到长程收益，所以 MPPI 容易追逐 learned reward 的局部假高分。

这些 log 说明了什么

S9b 虽然是三组里最完整的，但不是健康成功。最后 `disc_acc≈0.83`、`mean_demo_reward≈3.4`、`mean_gen_reward≈-3.6`，说明 discriminator 已经能明显区分 expert 和 agent。可同时 `Value/mean_loss≈39.8`、`MPPI/Average Reward≈-2.55`、`Average Value≈-4.52`，这说明 reward 强了，但 planner/value 没把它变成好动作。

环境侧最关键：`success_rate=0`，`end_of_episode_success_rate=0`。S9b 的 `average_pos_align_error≈0.136` 比较好，但 `average_rot_align_error≈1.57` 很差。动作表现上大概是：能靠近一些，但姿态/插入方向不对，最后多半 timeout，不是真正完成 full-task。

下一步建议

我建议下一个 full-task run 不要再只改 R1/rewfreq。用 S9b 做 baseline，然后改 planner horizon：

```
r1_coeff=0.05
reward_update_every=1
reward_lr=1e-6
gp_coeff=500
replay_ratio=0.078125
num_rollouts=512
```

```
num_elites=64
num_timesteps=10
loss_horizon=10
temperature=2.0
```

如果这个比 S9b 的 `average_pos_align_error` 更好、`rot_align_error` 开始下降、`MPPI/Average Reward` 不再后期坍塌，就继续。若 `Value/mean_loss` 又爆，下一步把 `reward_update_every=2`，不要直接回到 4。

第二个可并行思路是加强搜索但不改 reward：

```
num_rollouts=1024
num_elites=128
num_timesteps=7 或 10
temperature=2.0
```

这个用来判断是不是 MPPI 采样不足。如果 `Max Reward/Max Value` 提升但 `Average` 不提升，说明不是采样数问题，是 reward/value 形状问题。

我现在最不建议的是：继续加 `reward_lr`、降低 `temperature`、或者把 `gp_coeff` 继续堆高。前两个会让 planner 更快追假高分；最后一个只会让 reward 更钝。当前 full-task 的优先级应该是：horizon 7 -> 10，保持 R1=0.05，reward 更新从 1 和 2 之间找平衡，看 rot_align_error 和 Value loss。

能不能根据参数变化逐一分析logs图，然后详细讲出你是怎推理出可能需要改的问题的？
Sunday 8:05 AM

可以。我的推理方式是：不先看某一条曲线“好不好看”，而是看 参数变化 -> reward/discriminator 变化 -> value 是否能跟上 -> MPPI 是否选到更好动作 -> env/metrics 是否改善 这条链路有没有断。你这三组 full-task 的变量非常干净，所以能比较清楚地拆。

三组参数差异
红色 FullTask：

r1_coeff=0.005
reward_update_every=1
绿色 S9：

r1_coeff=0.05
reward_update_every=4
紫色 S9b：

r1_coeff=0.05
reward_update_every=1
其他主要参数一样：reward_lr=1e-6、gp_coeff=500、replay_ratio=0.078125、num_rollouts=512、num_elites=64、horizon=7。

1. Reward / Discriminator

红色：r1=0.005 太弱。
从 log 看，Reward/mean_demo_reward 一路上升，Reward/mean_gen_reward 后期一路下降到很负。这个意思是 discriminator 越来越确信 expert 和 agent 不一样。表面上这是“学会分类”，但对 MPAIL 不一定好，因为 planner 需要的是“往哪里动会更像 expert”的连续信号，不是一个极端二分类器。

我推理为：红色 reward 饱和太快。证据是 disc_entropy 下降，disc_acc 上升，同时 mean_demo_reward - mean_gen_reward gap 越来越大。后面 env reward/episode return 并没有同步变好，反而 abnormal 和负 return 增多，所以这个 discriminator 变强没有转化成好动作。

绿色：r1=0.05 + reward_update_every=4。
R1 加大以后，reward 输出被压住；rewfreq=4 又让 reward 每 4 次 update 才学一次。所以图里 reward/disc 曲线看起来更慢、更平。早期 env reward/episode return 是三组里最舒服的，说明“别让 reward 太快变尖”这个方向是对的。

但绿色的问题是：reward 学得可能又太慢。reward_update_every=4 会让 planner 很长时间拿到比较弱的 reward 信号，尤其 full-task 状态空间大，agent 离 expert 远，信号弱时 MPPI 搜不到真正能推进任务的动作。

紫色：r1=0.05 + reward_update_every=1。
这是单独测试 R1 的组。它能跑完 100 iter，说明比红色稳定；但最后 mean_demo_reward≈3.4、mean_gen_reward≈-3.6，gap 还是很大，disc_acc≈0.83。所以 R1=0.05 有帮助，但还没根治 discriminator 过强的问题。

结论：
r1_coeff=0.05 比 0.005 好；reward_update_every=4 能缓解过快饱和，但可能太慢。下一步更合理的是试 reward_update_every=2，而不是回到 4 或继续减 R1。

2. Value
红色和紫色的 Value/mean_loss 都后期上涨，紫色最后也很高。这个是我判断“不是单纯 planner 参数问题”的关键。

Value 的目标来自 learned reward + next value。如果 reward 分布越来越极端，value target 就会漂。于是你会看到：

Reward gap 变大
-> Value/td_target_mean 和 mean_q_value 漂
-> Value/mean_loss 上涨
-> MPPI/Average Value 不稳定或变差
紫色里虽然 env 平均位置误差有改善，但 Value/mean_loss≈39.8、MPPI/Average Value≈-4.5，说明内部 value 还没健康。也就是说：动作可能偶尔靠近了，但学习系统不稳定，继续训练不一定会自然变好。

结论：

后续要盯 Value/mean_loss。如果 horizon=10 后 value loss 更爆，说明长 horizon dynamics/value target 更难，需要先降低 reward 更新强度，比如 reward_update_every=2，或者降低 replay_ratio 到 0.0390625 做保守版。

3. MPPI
MPPI 图最重要不是单看 Max Reward，而是看：

MPPI/Max Reward 是否上升
MPPI/Average Reward 是否也上升
MPPI/Std Reward 是否爆
MPPI/Average Value 是否跟着变好
MPPI/Return 是否稳定
红色的问题是：早期 Max Value/Max Reward 可能变好，但后期 Average Value/Average Reward 掉，Std Value/Std Reward 变大。这说明 MPPI 找到少数高分轨迹，但整体 rollout 分布很差。planner 可能被 learned reward 的假高分骗了。

紫色也类似，只是没有红色那么剧烈。MPPI/Mean Policy Influence 很低，说明 learned policy prior 对 MPPI 帮助不大，MPPI 主要靠随机 rollout 搜。full-task 下 horizon=7 又短，512 rollouts 很可能只搜到局部靠近动作，看不到抓取/对齐/插入的长程收益。

所以我推理出 horizon 是下一步该改的问题：
num_timesteps=7 对 full-task 太短。它最多解释“靠近一点”，但解释不了完整 full-task 的阶段性动作。你现在 metrics 也支持这个判断：average_pos_align_error 有改善，但 rot_align_error 仍大，success 还是 0。

结论：
下一步优先 horizon=10，也就是：

learner.planner_cfg.sampling_cfg.num_timesteps=10
learner.loss_horizon=10
4. Policy
Policy/mean_std 和 Policy/entropy 上升，说明 policy 输出分布变宽。这个不一定坏，因为 MPAIL 需要探索。但如果同时看到：

Policy entropy 高
MPPI Average Reward 低
Value loss 高
success=0
那它更像“policy 没学到可用先验，只是在保持高随机性”。紫色 S9b 就有这个倾向。

Policy/alpha 下降是正常的自动熵调节，但它没有带来明确 success。说明当前瓶颈不在 alpha 本身，不建议先调 alpha。

5. Env / Metrics

这是最终判据。三组都是：

success_rate=0
end_of_episode_success_rate=0
但紫色 S9b 的 average_pos_align_error≈0.136 比较好，说明它确实学到了一些“靠近目标”的行为。问题是 average_rot_align_error≈1.57，很大，接近完全没对齐。full-task 里只靠 position 接近不够，插入要求姿态和接触几何。

所以我的动作层面推理是：

当前策略可能能把末端/物体带近一些
但不能稳定调整姿态、抓取/保持、插入
horizon=7 看不到姿态对齐和插入的后果
reward discriminator 又太容易只按状态分布判别
我会怎么改
第一优先：

r1_coeff=0.05
reward_update_every=1
num_timesteps=10
loss_horizon=10
temperature=2.0
num_rollouts=512
num_elites=64
也就是在 S9b 基础上只改 horizon。看它是否改善：

average_rot_align_error 是否下降
MPPI/Average Reward 是否不再后期坍塌
Value/mean_loss 是否还能接受
success 是否出现非零
如果 horizon=10 后 value loss 更爆，第二步改成：

reward_update_every=2
num_timesteps=10
loss_horizon=10
如果 horizon=10 后 Max Reward 上升但 Average Reward 不升，说明采样不够或假高分太多，再试：

num_rollouts=1024
num_elites=128
我现在不建议优先改 reward_lr、gp_coeff、temperature。因为现有图已经说明 reward 不是学不动，而是学出来以后太容易让 value/MPPI 不稳定；先让 planner 看远一点，再用 reward_update_every=2 控制 reward 强度，是更符合这三组 log 的下一步。

Sunday 8:06 AM

行，现在就跑一下你说的第一优先

Sunday 8:22 AM

Worked for 44m 27s

停下来了。`pgrep` 已经没有训练进程了；这条 run 保留在：

`logs/obs22_S10_full_gp500_lr1e6_r1c0p05_rewfreq1_replay0p078_8env_h10_100iter_20260524_082253`

有 `model_0.pt` 和 `model_50.pt`，日志实际到 89/90 左右。

总判断

这次 h10 “有效果但不明显”这个感觉是对的。它解决了一部分问题：discriminator 没有像 S9b 那样很快饱和，reward 信号更平滑；但没有解决核心问题：MPPI/Value 仍然越来越不稳定，环境动作还是频繁 abnormal/timeout，success 仍然 0。

换句话说：`horizon=10` 让 planner 看远一点了，但现在失败不只是“看得不够远”，还有 高分 rollout 和真实可执行动作脱节。

Reward 图说明什么

`Reward/mean_demo_reward` 从 0 慢慢涨到约 1.1，`Reward/mean_gen_reward` 降到约 -1.3 到 -1.4。这个比 S9b 温和很多，S9b 到 100 iter 已经大概 demo 3.4、gen -3.6。

推理：h10 没让 reward 更坏，反而让 reward 分离慢了。这个是好事。

`disc_acc` 从 0.55 左右到 0.75+，`disc_entropy` 从 0.68 掉到 0.43 左右。

推理：discriminator 正在学会区分 expert/agent，但还没到完全饱和。这个区间本来应该是可用的。如果这时动作还不成功，问题就不主要是 reward 太强，而是 planner/value/action 执行链条没接上。

`gp_gradient_mean` 到 1，`gp_loss` 降到接近 0。

推理：GP 正常工作，`gp_coeff=500` 没必要优先改。

Value 图说明什么

`Value/td_target_mean` 和 `Value/mean_q_value` 都越来越负，`Value/mean_loss` 后期升到 12+。

这很关键：reward 还没极端饱和，但 value 已经开始学坏/跟不上。也就是说 longer horizon 让 target 更长、更难估，value 在学一个越来越悲观、越来越漂的目标。

推理链是：

```
horizon 7 -> 10
rollout 更长
dynamics/value 误差积累更多
MPPI 看到更远但也更容易看到假的远期高分/低分
Value loss 上升
Average Value 往负走
```

所以 h10 不是完全错，但它把 value/dynamics 的压力放大了。

Policy 图说明什么

`Policy/mean_std` 很快升到接近 0.88，`Policy/entropy` 也升到高位，`alpha` 一直下降。

这说明 policy 仍然在保持很宽的动作分布。探索是有了，但没有形成有用 prior。`Policy/mean_loss` 后期从负变正并继续上升，也说明 policy 更新方向变差了。

推理：policy 不是在收敛到 expert-like plan，而是在高熵探索里被不稳定的 value/reward 拉扯。

MPPI 图说明什么

这是最重要的一组。

`MPPI/Max Value` 和 `MPPI/Max Reward` 明显上升。

这说明 planner 在 512 条 rollout 里确实能找到“看起来很高分”的候选。h10 起作用了。

但同时 `MPPI/Average Value` 后期往负走，`MPPI/Average Reward` 也从正转负，`Std Value/Std Reward` 越来越大，`Max Return/Min Return` 大幅抖。

这说明高分只是少数候选，不是整体动作分布变好。planner 很可能在追少数假高分轨迹，真实执行时就变成异常接触、姿态错、抓不稳、timeout。

推理链：

```
Max Reward 上升
但 Average Reward 下降
Std Reward/Value 上升
=> planner 找到少数"模型里高分"的轨迹
=> 这些轨迹不稳定/不可执行
=> env 里 abn 和负 ep_ret 反复出现
```

这就是你看到“有效果但不明显”的根本：它有搜索到好东西的迹象，但没有稳定落地。

Env / train.log 说明什么

到后期：

```
disc_demo ≈ 2.2
disc_gen ≈ -2.5
dyn_loss ≈ 0.18
succ = 0
abn 经常 1-2，iter 71 甚至 abn=4
```

dyn_loss 持续下降，说明模型训练表面上没爆；但 env 还是失败。这个组合通常表示：latent dynamics 的平均 loss 下降不代表关键接触阶段预测准确。full-task 的抓取/接触/插入是稀疏且非线性的，平均 MSE 会掩盖关键失败。

下一步方向

我现在不建议继续单纯拉长 horizon。h10 已经证明“更远”有一点帮助，但 value/MPPI 抖得更明显。下一步优先改成：保留 h10，但降低 planner 被假高分骗的程度。

第一推荐：

```
horizon=10
loss_horizon=10
reward_update_every=2
r1_coeff=0.05
temperature=2.0
num_rollouts=512
num_elites=64
replay_ratio=0.078125
```

原因：h10 的 reward 分离仍在可用区间，但 value loss 上升、MPPI std 变大。reward_update_every=2 会让 reward/value 的目标变化慢一点，可能减少后期 MPPI 假高分和 value 漂移。

第二推荐，如果你想测试“是不是 elite 太容易被骗”：

```
horizon=10
num_rollouts=512
num_elites=128
temperature=2.0
reward_update_every=1 或 2
```

num_elites=64 在 512 rollouts 里偏精英化。现在 Max Reward 好但 Average 差，说明少数 top rollout 可能太支配更新。把 elites 加到 128 会让动作更保守、更平均，可能少一些 abnormal。

第三推荐，专门针对 abnormal：

```
horizon=10
raw_action_limit=[6,6,6,4,4,1.0,0.5]
```

你现在 [8,8,8,5,5,1.25,0.5] 动作很大。h10 后 planner 看得更远，可能更敢做大动作，导致接触异常。降低 action limit 可能牺牲速度，但会验证“失败是不是动作过猛”。

我会排的下一组实验

最干净的下一步是两条短跑 50-100 iter：

```
A: h10 + reward_update_every=2
B: h10 + reward_update_every=2 + num_elites=128
```

如果 A 的 Value/mean_loss 明显低、MPPI/Std Reward/Value 不再爆，就是 reward/value target 变化太快。

如果 B 的 abnormal 明显少、Average Reward 更稳，就是 elite 太尖、planner 被少数假高分骗。

如果两者都不行，再动 `raw_action_limit`，因为那就更像真实执行动作过猛/接触不稳。

Sunday 9:06 AM

你能不能帮我出四个方案依次跑来找到最好的方向（raw_action_limit不要改不包括在内）

Sunday 9:12 AM

Worked for 1m 56s

我已经给你写好了四组顺序实验脚本：

scripts/run_s11_full_h10_direction_4runs.sh

语法检查通过了。运行方式：

```
bash scripts/run_s11_full_h10_direction_4runs.sh
```

四个方案是：

```
A: h10 + reward_update_every=2 + elites=64 + temp=2 + use_terminations=False
B: h10 + reward_update_every=2 + elites=128 + temp=2 + use_terminations=False
C: h10 + reward_update_every=2 + elites=128 + temp=3 + use_terminations=False
D: h10 + reward_update_every=2 + elites=128 + temp=2 + use_terminations=True
```

raw_action_limit 全部保持不变：

```
[8,8,8,5,5,1.25,0.5]
```

这四个的逻辑是：

A 测 “S10 的 value/MPPI 漂移是不是 reward target 变太快”。

B 测 “MPPI 是不是被少数 top rollout 假高分骗了”。

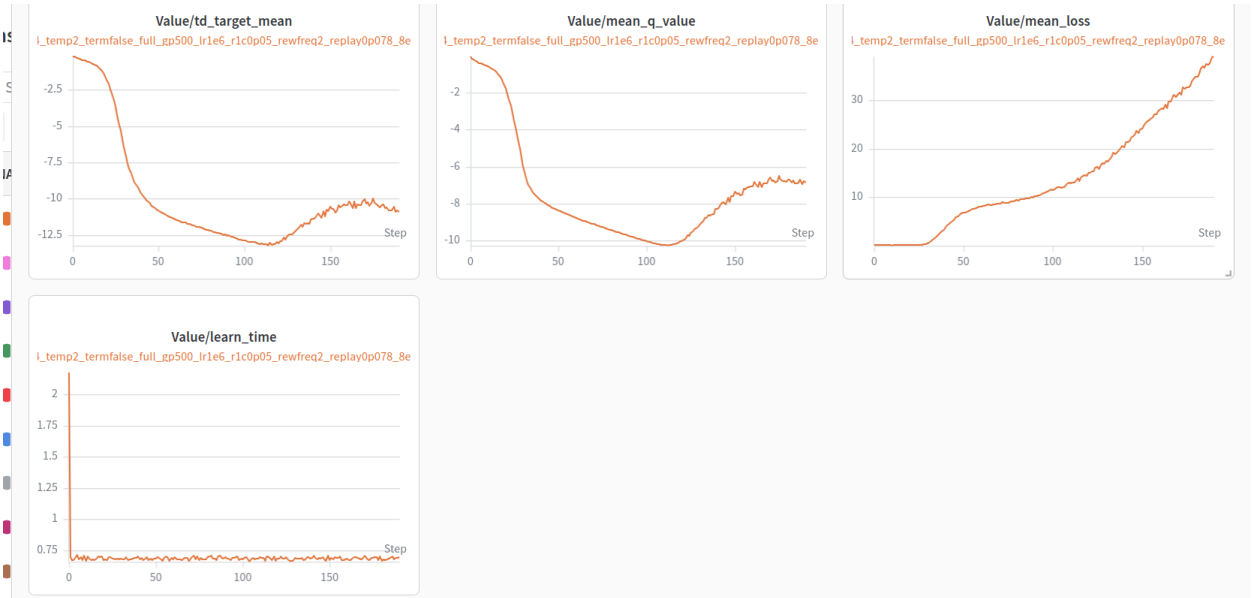
C 测 “更保守的 MPPI weighting 能不能减少抖动和 abnormal”。

D 测 “full-task 里 abnormal/terminal 语义是不是影响 value 学习”。

判断优先级看这些：

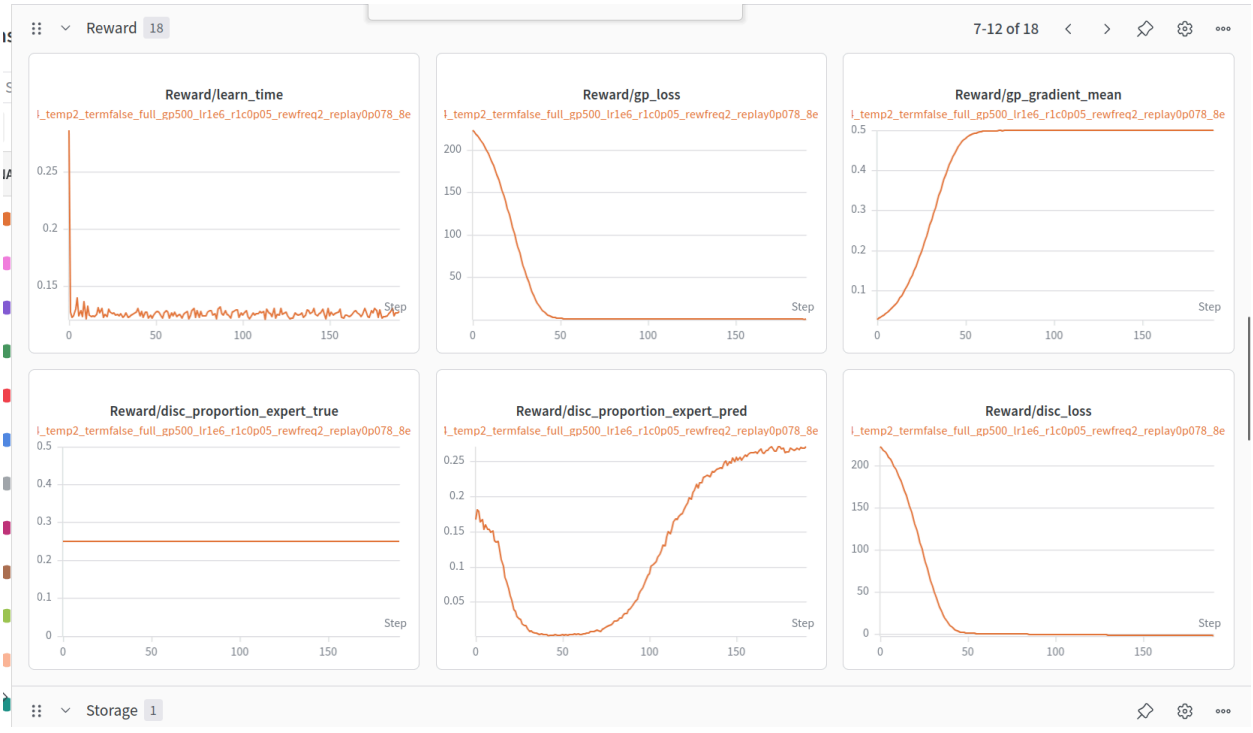
1. `success_rate / end_of_episode_success_rate` 有没有非零
2. `abnormal_robot_count` 是否下降
3. `Value/mean_loss` 是否比 `S10` 更低更平
4. `MPPI/Average Reward/Value` 是否不再后期坍塌
5. `MPPI/Std Reward/Std Value` 是否不再持续变大
6. `Reward demo-gen gap` 是否保持 `< 6~8`

我建议先让这个脚本顺序跑；如果时间紧，A 和 B 最值得先看，C/D 是确认方向。



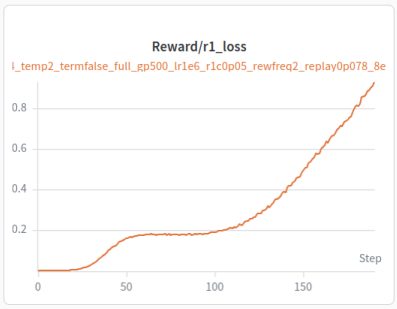
System 33

+ Add section



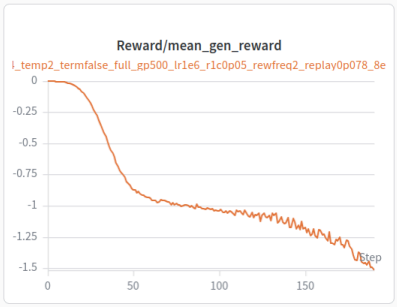
Storage 1

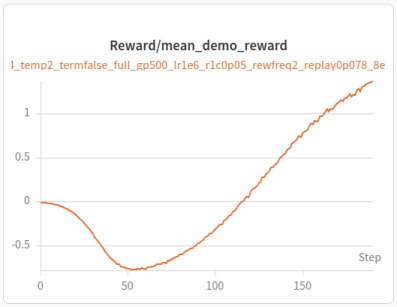
Reward 18 1-6 of 18

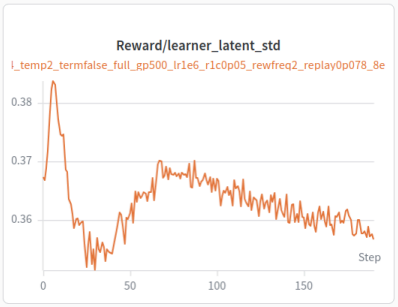






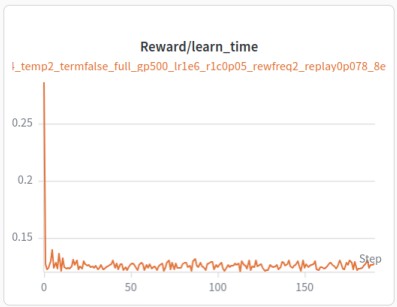


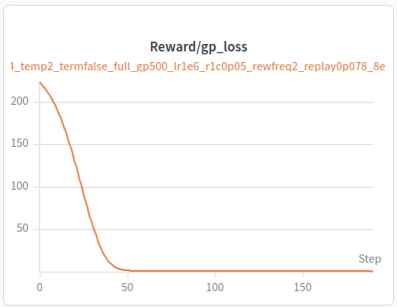


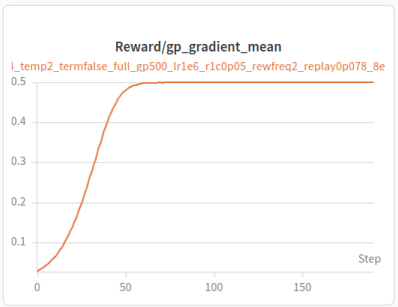


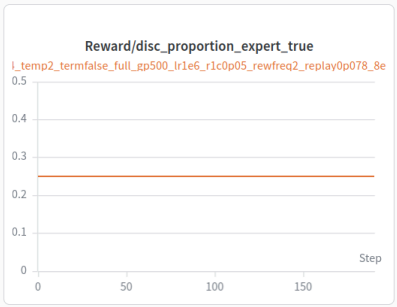
Storage 1

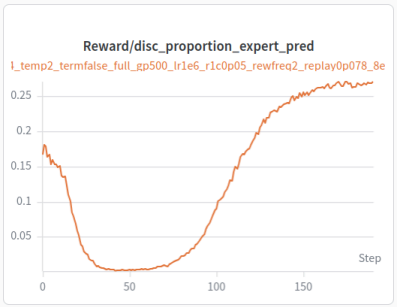
Reward 18 7-12 of 18

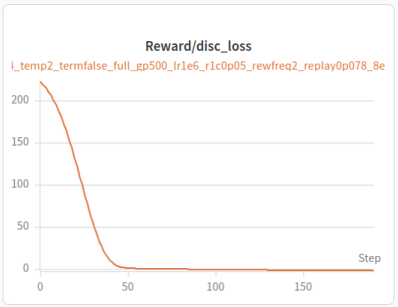






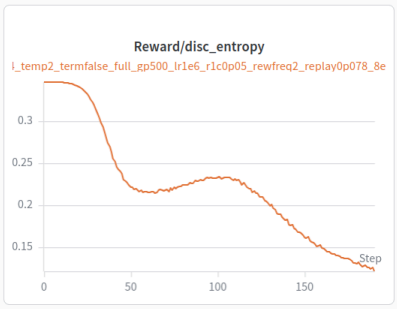


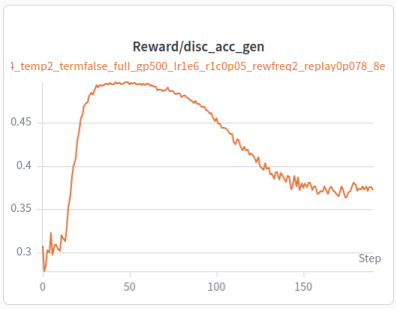


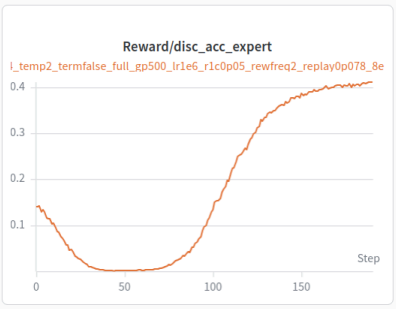


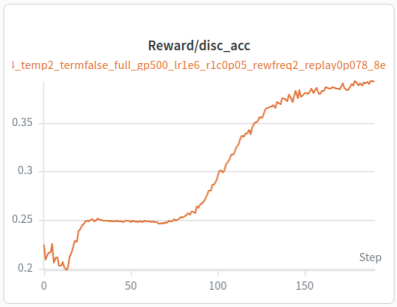
Storage 1

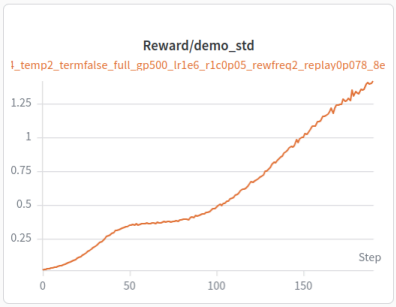
Reward 18 13-18 of 18

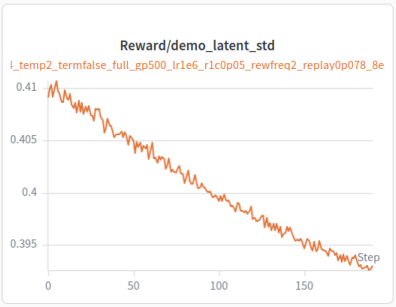






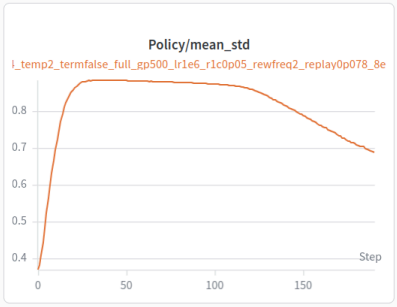


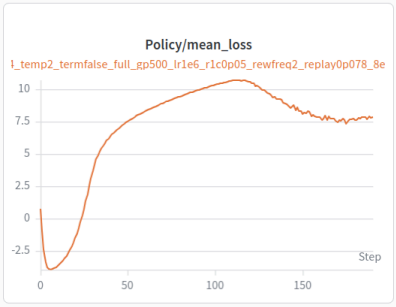


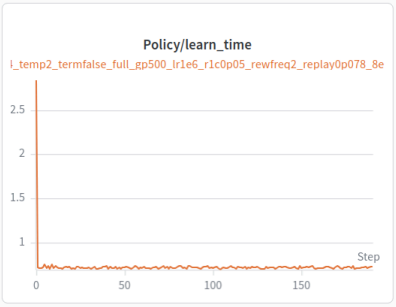


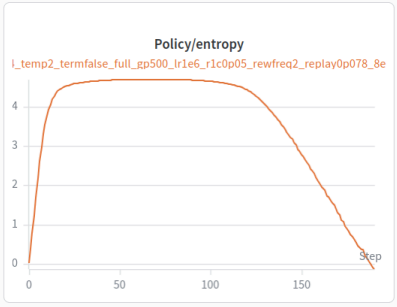
Storage 1

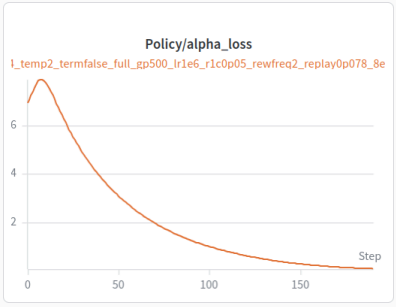
Policy 6 1-6 of 6

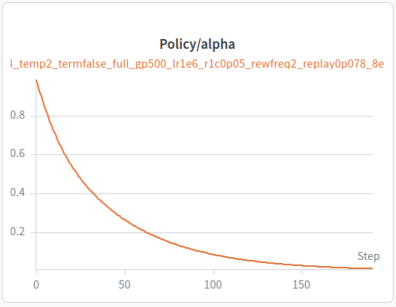












Reward 18 13-18 of 18

Dyn 3

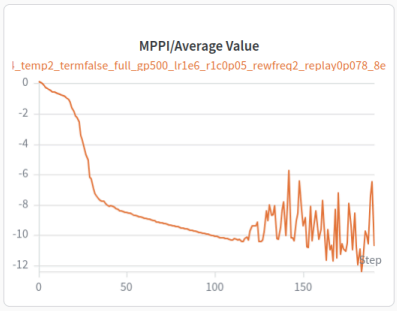
Env 11

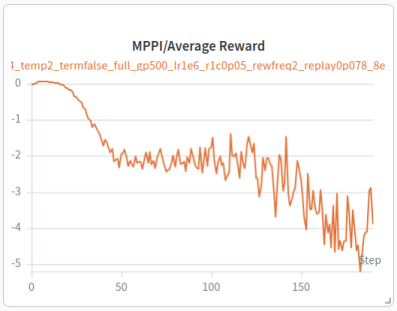
Episode_Reward 8

Episode_Termination 2

Metrics 5

MPPI 20 19-20 of 20

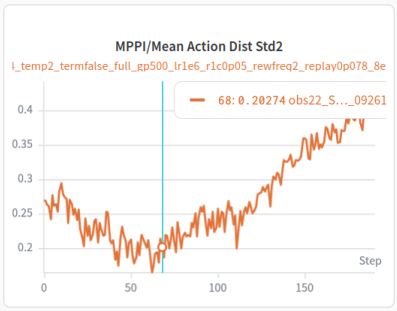


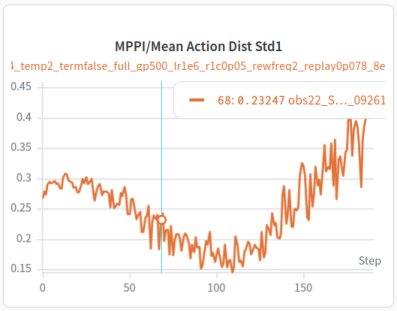


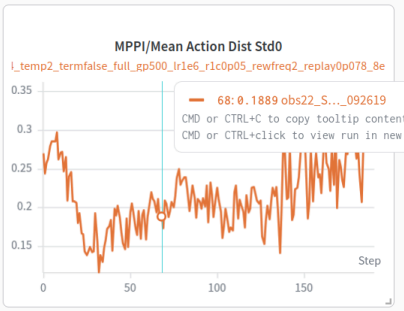
Perf 3

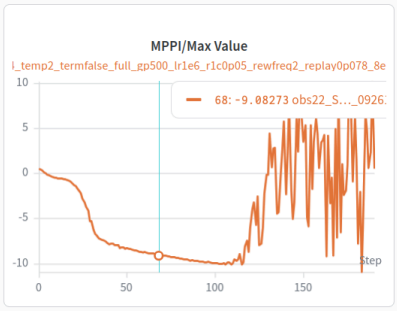
Policy 6 1-6 of 6

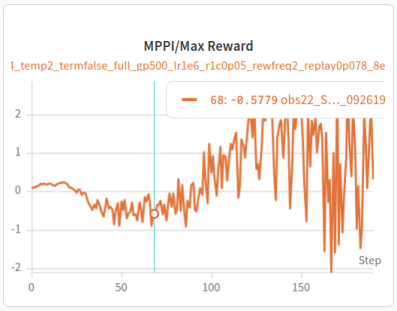
MPPI 20 13-18 of 20

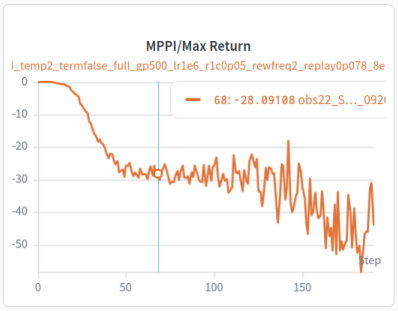












Perf 3



a跑了150tier,效果很不好，你分析一下（其他几个看来也不会好，帮我重新判断方向，看来reward_update_every=2了之后更差了，不应该这个方向，你一定要结合之前的训练仔仔细细的究其根本，到底是什么问题，我现在乱调小参数已经很久了，感觉不是微调参数的问题吧，你仔细读代码仔细的通过深入理解算法来判断问题在哪里？

,鞯改的东西调整的东西有很多很多，要从算法的根本逻辑分析是什么问题，（也可以参考仓库里面其他已有的别人的wrapper）

结合之前的 红色 FullTask 绿色 S9 紫色 S9b 一起，仔细的分析，现在这个状态，像是什么导致的学习不成功，问题处在那，要讲解的详细易懂，每个log都要分析原因，哪几个现象证明问题是什么，这个问题代码逻辑由什么造成的，有什么解决办法，不要看表象，要看本质代码逻辑问题，算法逻辑导致的问题，哪一步出现了问题？我不能改算法本身的逻辑，还是用原来算法，但是可以做调整

而且并不是fulltask的问题，因为我把任务分段减半也是一样的问题学不会，
你仔细读代码逻辑和论文（/home/fanz232602.24121v1.pdf），详细讲讲现在的问题可以通过算法训练的哪一步的哪里调整来改善？
Show less
Sunday 10:42 PM
Worked for 5m 8s

结论先说清楚：这不像 `reward_update_every`、elite、temperature 这种小参数的问题。现在更像是 MPAIL2 的核心闭环被代码/数据分布破坏了：reward 不是在学“任务进展”，而是在学“demo latent 分布 vs learner 旧数据分布”；value 又把这个不可靠 reward bootstrap 放大；MPPI 最后开始找 reward/value 的假高分漏洞。

最关键的代码问题我找到了：replay buffer 填满以后不会 FIFO 更新。

在 mpail2/storage.py (line 224)，`_add_new_data_to_buffer()` 满了以后返回 `0`，但 process_replay_buffer (line 254) 里马上判断：

```
if self.num_trajectories_stored <= self.replay_dim_size:
    return
```

`num_trajectories_stored` 永远不会大于 `replay_dim_size`，所以后面的 FIFO shift 分支永远到不了。也就是说 replay 到 `100000` transitions 后，新经验不再进入训练。S9b 的 wandb summary 已经显示 `Storage/replay_size=100000`，这之后 reward/value/policy 都在旧数据上训练，当前 MPPI 却已经跑到新分布了。

这正好解释 A 的图：大概 80 iter 以后 buffer 接近/达到满，110-150 以后 `Value/mean_loss` 爆炸，`MPPI/Std Value`、`MPPI/Std Reward` 爆，`MPPI/Max Reward/Value` 出现假高分，动作分布 std 又开始变大。这不是学会了，而是 planner 在利用过期 reward/value 的漏洞。

第二个问题是 fixed normalizer。日志里有：

```
[FixedObsNormalizer] policy: std range [0.000, 4.038]
```

我查了 demo compact obs，gripper 维度 std 只有 `0.000397`，几乎常量。现在 FixedObsNormalizer (line 62) 会用 demo std 做除法再 clip 到 `[-10,10]`。这会让 learner 只要在 gripper 或某些低方差维度上偏一点，就变成极端 normalized value。reward 很容易学成“像不像 demo 支撑集”，不是“有没有完成插入/接近/抓取”。所以你分段减半也学不会，这不是 full task 长的问题。

第三个问题是 `reward_update_every=2/4` 为什么更差。代码里 reward 被跳过在 mpail2/learner.py (line 330)，但 value 和 policy 仍然每次更新：value 在 learner.py (line 343)，policy 在 learner.py (line 354)。所以你不是“让系统更稳”，而是让 value/policy 多次追一个更旧的 reward。reward 本身已经是错的/过期的，慢更新只会让误差传播更久。

对应每个 log 的现象：

FullTask 红色：`disc_demo` 和 `disc_gen` 很快被拉开，但 success 还是 0，abnormal 很多。说明 discriminator 变强不等于 task reward 变对了。它在区分 demo/learner 分布。

S9 绿色：`reward_update_every=4` 后 reward 变化慢，短期看不爆，但 planner 没有真正学到可执行方向；本质是 reward 太滞后，value/policy 仍在更新。

S9b 紫色：`reward_update_every=1`，`disc_demo≈3.4`、`disc_gen≈-3.6`、`disc_acc≈0.83`，看起来 reward 学得很好，但 success 仍是 0，`Value/mean_loss≈39.8`，`td_target_mean≈-11.7`。这是最强证据：判别器赢了，控制没赢。

A 橙色：horizon 10 + reward_every 2 后，`td_target_mean` 和 `mean_q_value` 先掉再反弹，`Value/mean_loss` 持续爆，MPPI max value/reward 后期大幅震荡，policy alpha 掉到接近 0。这个形状就是 reward/value 被 planner exploit，不是真进步。

论文里 MPAIL2 的关键假设是：reward 要用 off-policy replay 覆盖“所有经历过的状态”，因为 MPPI 会在线随机/策略采样大量候选动作。现在我们的 replay 满后冻结，直接违反这个假设；reward 只在旧分布上准确，MPPI 在新分布上自然会钻空子。

优先解决顺序我建议这样：

1. 先修 replay FIFO。这是代码 bug，不是算法改动。
2. 改 normalizer：至少给 std 加 floor，比如 `std = max(std, 0.02~0.05)`，或先去掉/单独处理 gripper 这个近常量维度。
3. 暂时回到 `reward_update_every=1`，不要再往 2/4 走。
4. 降低 reward/value 被 exploit 的机会：先用 h=5 或 h=7，降低 raw action limit，等 reward/value 稳住后再加 horizon。
5. 增加诊断日志：replay 是否真的在滚动、sample 的 transition age、reward 在 newest replay vs oldest replay vs demo 上的分数。

我现在的判断：根因第一是 replay buffer 不更新导致 off-policy reward 失去覆盖；第二是 demo-normalizer/compact obs 让 reward 学到支撑集差异；第三才是 value/MPPI 放大这个错误。`reward_update_every=2` 不是方向，应该先修训练数据闭环。

Sunday 10:47 PM

根据你写的优先解决顺序我建议这样：
先修 replay FIFO。这是代码 bug，不是算法改动。
改 normalizer：至少给 std 加 floor，比如 std = max(std, 0.02~0.05)，或先去掉/单独处理 gripper 这个近常量维度。
暂时回到 reward_update_every=1，不要再往 2/4 走。
降低 reward/value 被 exploit 的机会：先用 h=5 或 h=7，降低 raw action limit，等 reward/value 稳住后再加 horizon。

增加诊断日志：replay 是否真的在滚动、sample 的 transition age、reward 在 newest replay vs oldest replay vs demo 上的分数。
给我三个修改方案让我我分别短跑50tier，（要可撤回），给出来直接跑
Sunday 11:38 PM
Worked for 10m 7s

已经尽力直接跑了，但当前机器的 NVIDIA driver/GPU 不可用，Isaac 启动时报：

```
RuntimeError: No CUDA GPUs are available
```

我已经做了两件事：

1. 三个脚本都实际尝试启动过：
   - `S12a` run dir: `logs/obs22_S12a_fifo50k_stdfloor005_h7_raw6..._20260524_234719`
   - `S12b` run dir: `logs/obs22_S12b_fifo50k_stdfloor005_h5_raw5..._20260524_234729`
   - `S12c` run dir: `logs/obs22_S12c_fifo50k_skiplowstd005_h5_raw5..._20260524_234735`
2. 我启动了一个非沙箱后台等待队列，GPU 一恢复就自动重跑三组：
   - PID: `15728`
   - 日志: `logs/s12_launcher/s12_abc_50_wait_gpu_20260524_234807.log`

它现在还活着，正在每 60 秒等 `nvidia-smi -L` 成功。GPU/driver 一可见，它会按 `S12a -> S12b -> S12c` 顺序跑起来。

Edited 9 files+336-16

Undo

Review

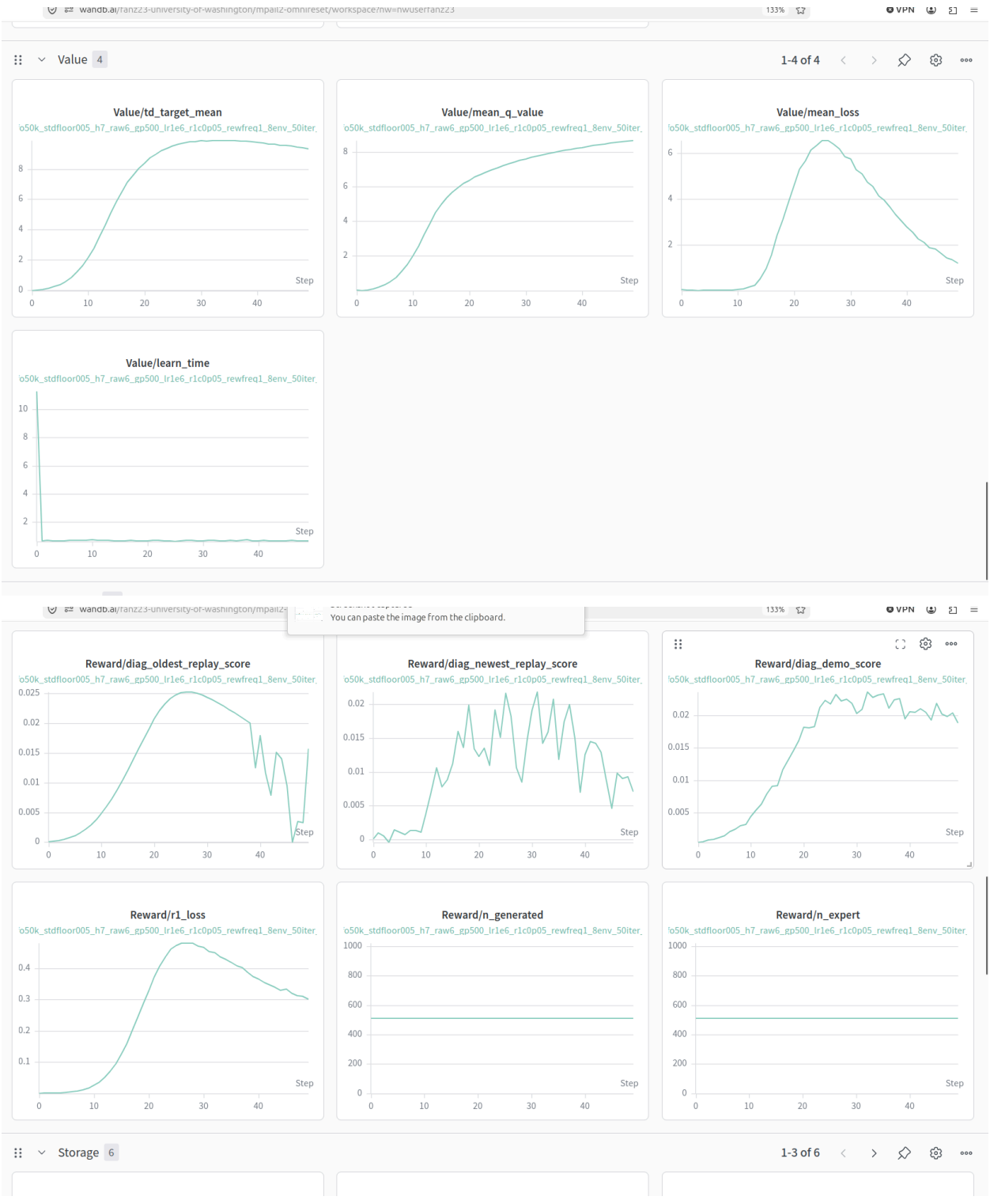
mpail2/configs/cfgs.py

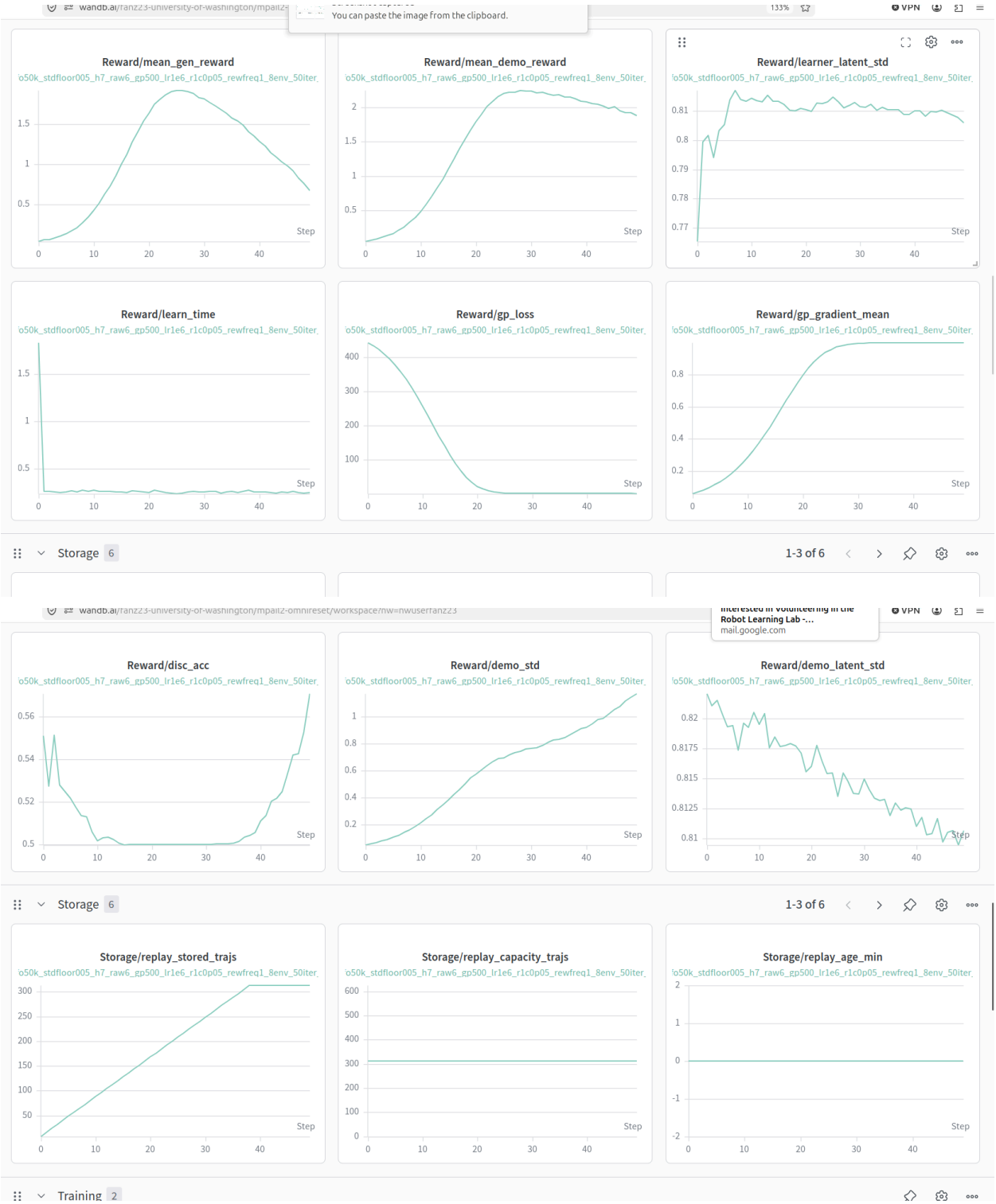
mpail2/utils/obs_normalizer.py

mpail2/learner.py

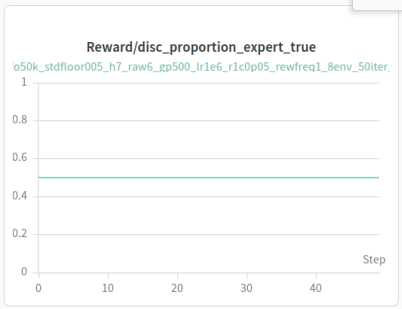
Show 6 more files

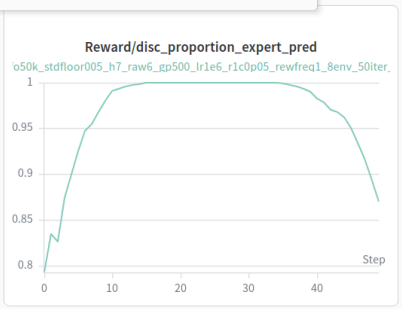
Sunday 11:48 PM

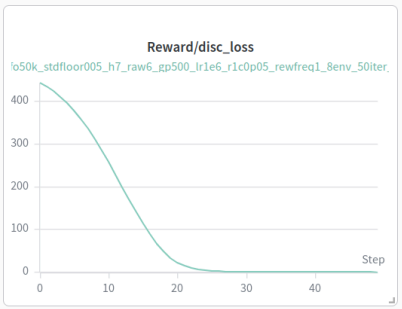

Value 4
1-4 of 4
Value/td_target_mean
Value/mean_q_value
Value/mean_loss
Value/learn_time
Step
Reward/diag_oldest_replay_score
Reward/diag_newest_replay_score
Reward/diag_demo_score
Reward/r1_loss
Reward/n_generated
Reward/n_expert
Storage 6
1-3 of 6

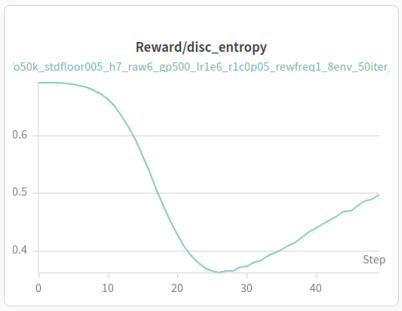

You can paste the image from the clipboard.
Reward/mean_gen_reward
Reward/mean_demo_reward
Reward/learner_latent_std
Reward/learn_time
Reward/gp_loss
Reward/gp_gradient_mean
Storage 6
1-3 of 6
Reward/disc_acc
Reward/demo_std
Reward/demo_latent_std
Storage 6
1-3 of 6
Storage/replay_stored_trajs
Storage/replay_capacity_trajs
Storage/replay_age_min
Training 2

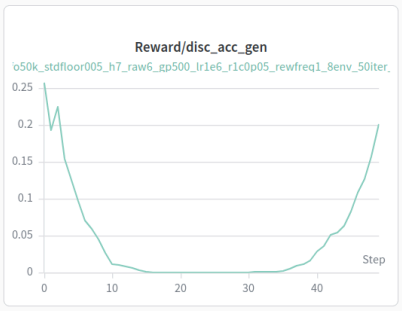
You can paste the image from the clipboard.







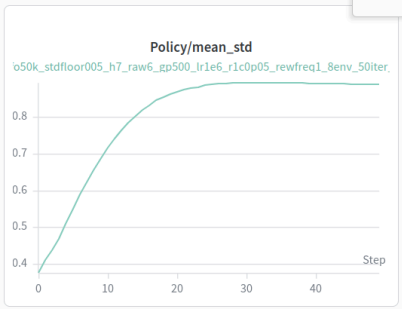


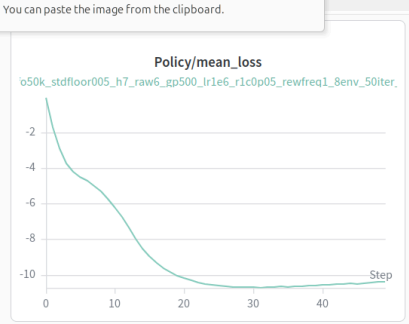


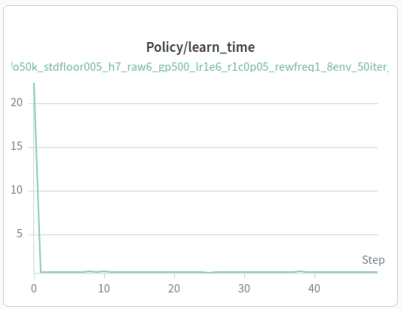


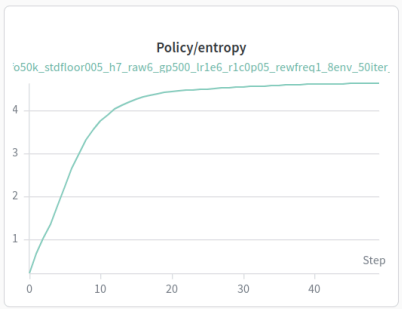
Storage 6 1-3 of 6

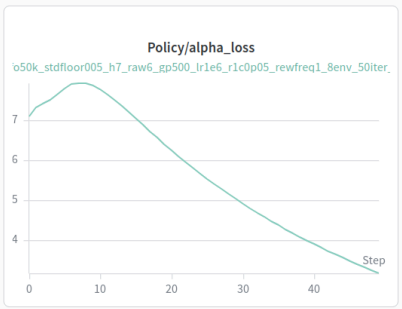
You can paste the image from the clipboard.

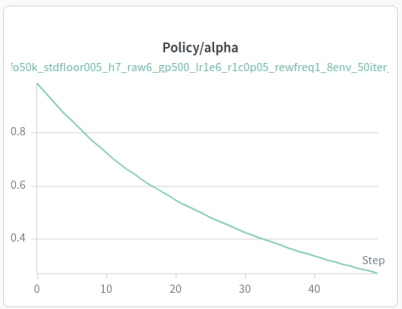












Reward 21 13-18 of 21

You can paste the image from the clipboard.

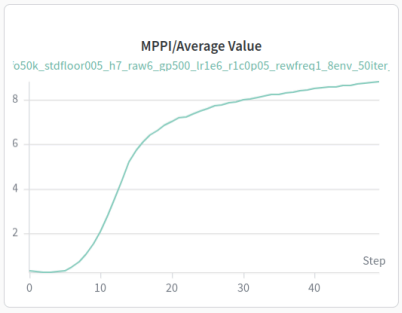
Env 11

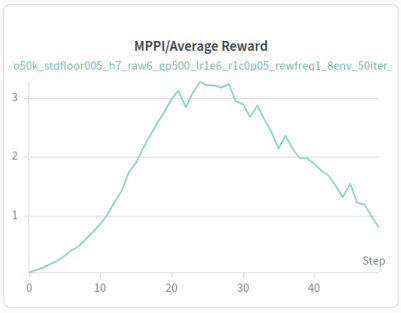
Episode_Reward 8

Episode_Termination 2

Metrics 5

MPPI 20 19-20 of 20

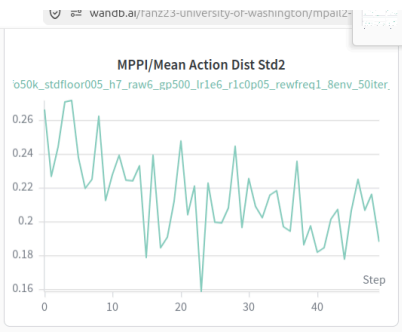


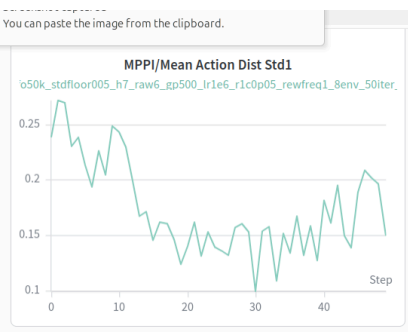


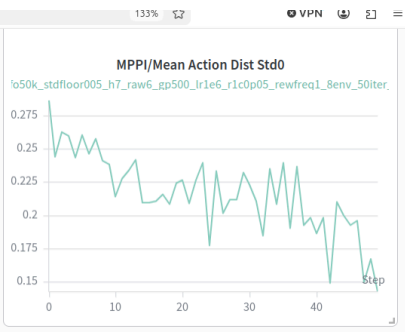
Perf 3

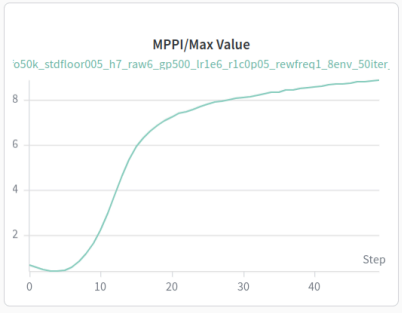
Policy 6 1-6 of 6

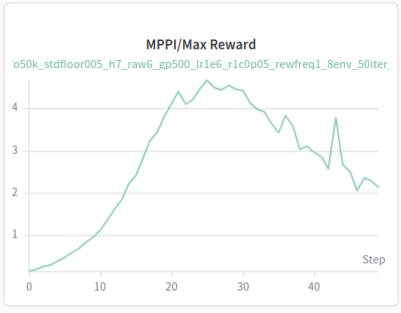
You can paste the image from the clipboard.

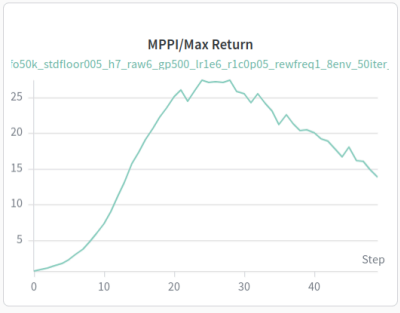












Perf 3

Policy 6 1-6 of 6



Perf 3

Policy 6 1-6 of 6



Perf 3

Policy 6 1-6 of 6

三组都是 50 iter / 8 env / replay_size=50000 / replay_ratio=0.078125 / reward_lr=1e-6 / gp=500 / r1=0.05 / reward_update_every=1 / num_rollouts=512 / num_elites=64 / obs22 / FIFO replay。

共同改动（针对 codex 怀疑的两类问题）

改动	对应 codex 担心的问题
`replay_size=50000`（之前 100000）	让 FIFO 滚动得更快，新经验更快进入训练分布
`std_floor=0.05`	normalizer 不再被 demo 上接近常量的维度（gripper 等）放大
`reward_update_every=1`	不踩 codex 提的"reward 慢更新但 value/policy 仍在追"那个陷阱

三组的差异

	S12A	S12B	S12C
`loss_horizon` / MPPI `num_timesteps`	7	5	5
`raw_action_limit`	[6,6,6,4,4,1.0,0.5]	[5,5,5,3,3,0.8,0.4]	[5,5,5,3,3,0.8,0.4]
`skip_low_std_dims`	False	False	True

每组验证什么

- **S12A**：基线方向。h=7 + 较松动作 + std_floor=0.05。验证"只加 std_floor、回到 reward_every=1、缩小 replay 让它 FIFO"这一组合是否就能让 reward/value 不被 MPPI exploit。如果 A 已经稳了，说明根因就是 normalizer + replay churn，h=7 不是问题。
- **S12B**：A → B 同时缩 horizon（7→5）和收紧动作上限。把 MPPI 找漏洞的搜索空间砍小。如果 A 还是 exploit·B 稳了，说明在 reward/value 真正稳之前，必须先压住 MPPI 的搜索强度。**注意 A→B 是 horizon 和 action_limit 一起变了**，这是个混淆对照——不能单独归因到 horizon 还是 action limit。
- **S12C**：在 B 的基础上把 `skip_low_std_dims=True`。低 std 维度不做 normalize、原值通过。验证比 std_floor 更激进的处理是否进一步好。如果 C 比 B 更稳，说明那 7 个低 std 维度即便有 0.05 floor 仍然在制造问题；如果 C 跟 B 差不多，floor 已经够了。

我已经跑完a了，bc严重崩溃了就不给你看了，a的这个情况证明什么，感觉也不太健康但是我找不到原因，你要结合代码算法逻辑深度解析

而且我看了a的50tier视频，动的非常非常慢动作非常非常小，几乎不可见，非常不正常，这个我也不知道是什么逻辑造成的

2:49 AM

Worked for 3m 26s

A 这组证明的核心不是“已经健康了”，而是：

FIFO bug 不是唯一根因；修好 replay 后，主要问题暴露成 MPPI/value 的静止高分 exploit。

几个最关键证据：

1. `Storage/replay_stored_trajs` 到容量后停住，`Storage/replay_age_min=0`
   说明 FIFO 已经在滚动，新数据确实进 replay 了。旧的“replay 满后永久训练旧数据”不是 A 这次的主因。
2. `Reward` 没有像之前那样完全崩掉
   `Reward/mean_demo_reward` 和 `mean_gen_reward` 都先升后降，但 demo 始终高于 gen；`gp_gradient_mean` 接近 1，`disc_entropy` 后期回升。说明 discriminator/reward 不是纯发散。
3. 真正不正常的是：`MPPI/Average Value` 单调升到 8+，但 `MPPI/Average Reward` 25 tier 后开始掉。
   这非常关键。MPPI 不是在找高 reward 的轨迹，而是在找 value 觉得高的轨迹。代码里 planner.py (line 157) 的 `td_return` 最后一步直接用 value 替代 reward：

```
rewards[..., -1] = self.value(...)
```

所以只要 value 对“接近当前状态/小动作状态”变乐观，MPPI 就会选静止或极小动作。

4. `MPPI/Mean Policy Influence ≈ 0`
   这说明 policy rollout 几乎没有成为 elite。实际控制基本来自 MPPI noise/CEM，不是 policy network

。于是 `Policy/entropy`、`Policy/std` 升高不代表机器人真的探索，它只是 policy 学得很散，但 MPPI 不采纳它。

5. 视频慢动作、小动作的直接逻辑链是：

`reward/value` 对小动作附近状态给高分

→ MPPI top-k elite 选小动作轨迹

→ `update_plan_dist_from_elites()` 把均值/std 往小动作收缩

→ 下一步又以上一步 opt_controls 为中心采样

→ 机器人越来越像“慢慢蹭/几乎不动”。

不是 raw limit 单独造成的。A 的 raw limit `[6,6,6,4,4,1.0,0.5]` 确实偏保守，但 `MPPI/Mean Policy Influence≈0` 和 `Average Value↑ / Average Reward↓` 才是本质证据。

还有一个我会修的日志问题：`Reward/diag_*` 现在被除以 `actual_updates` 了，所以图上 0.02 不是原始 reward 分数。它应该和 `mean_demo_reward≈2` 同尺度，诊断日志目前低估了约 100 倍。

下一步方向

不要再往更小 raw limit、更短 horizon 走了。B/C 崩溃符合预期：动作更小、horizon 更短，只会让 MPPI 更难靠真实动作产生进展，更容易依赖 value 幻觉。

我建议下一步不是微调 reward freq，而是补两个诊断/约束：

- 记录真实执行动作：`Action/norm`、每维 normalized/raw action mean/std/max。
- 记录 dynamics action sensitivity：同一个 z 下 action=0 和 random action 预测的 next latent 差多少。

如果 action sensitivity 很小，就说明 dynamics 学成“动作不重要的 latent drift”，MPPI 自然会学静止高分。这就是现在最像的根因。

所以需要我重新跑一遍吗，你对现在MPPI/Mean Policy Influence≈0 和 Average Value↑ / Average Reward↓ 这两个严重问题的原因有什么看法吗，算法逻辑里面这个是什么原因呢？
3:04 AM
Worked for 1m 4s

不建议原样重跑 A。A 已经说明问题不在 `reward_update_every=2/4`，也不主要是 FIFO 了；现在的核心是 MPPI 正在被 value 带进“静止高分”的局部解。

两个现象怎么连起来

`MPPI/Mean Policy Influence≈0` 的意思是：MPPI 采样里有一部分来自 policy network，但这些 policy proposal 几乎没有进入 elite。代码在 sampling.py (line 245) 把 noise rollout 和 policy rollout 拼起来；然后 planner.py (line 277) 只按 return 选 top-k。Influence≈0 就是 policy 的轨迹分数打不过 noise/CEM 轨迹，实际控制基本不是 policy 在主导。

`Average Value↑ / Average Reward↓` 更严重。MPPI 的 return 里，前面步是 reward，但最后一步被 value 替代：

```
rewards[..., -1] = self.value(...)
```

所以它会选“短期 reward 下降、但终点 value 很高”的轨迹。A 的曲线就是这个：reward 25 tier 后掉，value 继续涨。视频慢动作/小动作就很像 value 学会了给“几乎不动、保持当前安全状态”的 latent 高分，MPPI 于是选择小动作，因为这类轨迹终端 value 高、风险低、还不容易触发异常。

算法逻辑里的根因

这是 model-based imitation 很典型的闭环问题：

1. reward 只区分 demo transition 和 learner transition，不直接奖励“动作幅度/任务进展”。
2. value 用 bootstrapped target 学：
   `Q(s,a) = r + gamma * V(model_rollout(policy(next_s)))`
3. policy 又用 value 更新，去找高 Q 的动作。
4. MPPI planning 再用 learned dynamics + reward + value 排序 rollout。

一旦 value 对某类状态过乐观，policy/MPPI 都会追它；而这些状态往往是 replay 里大量出现的“没成功但也没坏”的慢动作状态。于是算法不是不会优化，而是在优化一个假的高分盆地。

我现在最怀疑三件事，按优先级：

1. value overestimation / terminal value dominate
   MPPI 里 terminal value 太强，而且 planner 用 `return_type='avg'`，不是 ensemble min。value 一乐观，MPPI 就吃它。
2. dynamics 学到 action sensitivity 太弱
   如果 latent dynamics 觉得不同 action 造成的 next latent 差别很小，那么大动作没有预测收益，MPPI 更偏向小动作/稳定轨迹。
3. policy proposal 不进入 elite
   policy influence≈0 表明 policy 学出来的 proposal 对 MPPI 没帮助，甚至被 MPPI 排斥。后面 policy entropy/std 升高也没意义，因为 planner 不采用它。

所以不是“再跑一遍 A”。下一步应该跑一个有诊断的短实验：

- log 实际执行动作 `Action/norm`, `Action/raw_norm`, 每维 mean/std/max。
- log `Q(action=0)` vs `Q(executed action)` vs `Q(random/demo-like action)`。
- log dynamics sensitivity：同一个 z 下 `||f(z,a)-f(z,0)||`。
- 把 planner terminal value 临时改成 ensemble `min` 做 50 tier 对照，减少 value exploit。

如果这个对照里动作变大、`Average Reward` 不再和 `Average Value` 分叉，就基本坐实：问题是 value terminal optimism，不是 reward freq，也不是 fulltask 长度。

好的，就按照你说的做，你直接帮我来跑

3:12 AM

Worked for 5m 53s

已经帮你跑起来了。

S15 正在跑，真实训练进程 PID 是 `31283`。
Run 目录是 S15 log dir，启动日志在 launcher log。

这次不是重跑 A，而是按我们刚才判断的核心问题来验证：

- `planner_cfg.value_return_type=min`：MPPI terminal value 从 ensemble avg 改成 min，降低 value 高估被 exploit 的机会。
- `h=7`、`raw_action_limit=[8,8,8,5,5,1.25,0.5]`、`std_floor=0.03`。
- 保留 `reward_update_every=1`、FIFO replay、reward diagnostics。
- 新增诊断日志：`Action/*`、`ValueDiag/q_zero_mean`、`ValueDiag/q_exec_mean`、`ValueDiag/q_rand_mean`、`Dyn/action_sensitivity_*`。

当前日志已经进入训练：

```
[iter    0/50] fps=36 rew=-1.5837 ep_ret=-1.584 succ=0.000 done=8 abn=1 to=7 disc_demo=-0.082 disc_gen=-0.103 dyn_loss=1.6983
```

你重点看这几个指标：

- `MPPI/Mean Policy Influence`：如果还是接近 0，说明 policy proposal 仍然完全没被 planner 采纳。
- `MPPI/Average Value` vs `MPPI/Average Reward`：如果 value 还涨、reward 还掉，说明 value exploit 仍存在。
- `Action/raw_norm` 和 `Action/env_norm`：验证视频里“动作几乎不可见”到底是不是实际动作幅度被压小。
- `ValueDiag/q_zero_mean` 对比 `q_exec_mean/q_rand_mean`：如果 zero-action Q 明显偏高，就基本坐实“不动策略被 value 奖励”。
- `Dyn/action_sensitivity_exec_vs_zero`：如果很低，说明 learned dynamics 对动作不敏感，MPPI 怎么采样都像在同一个状态附近打转。

Edited 6 files+122-2

Undo

Review

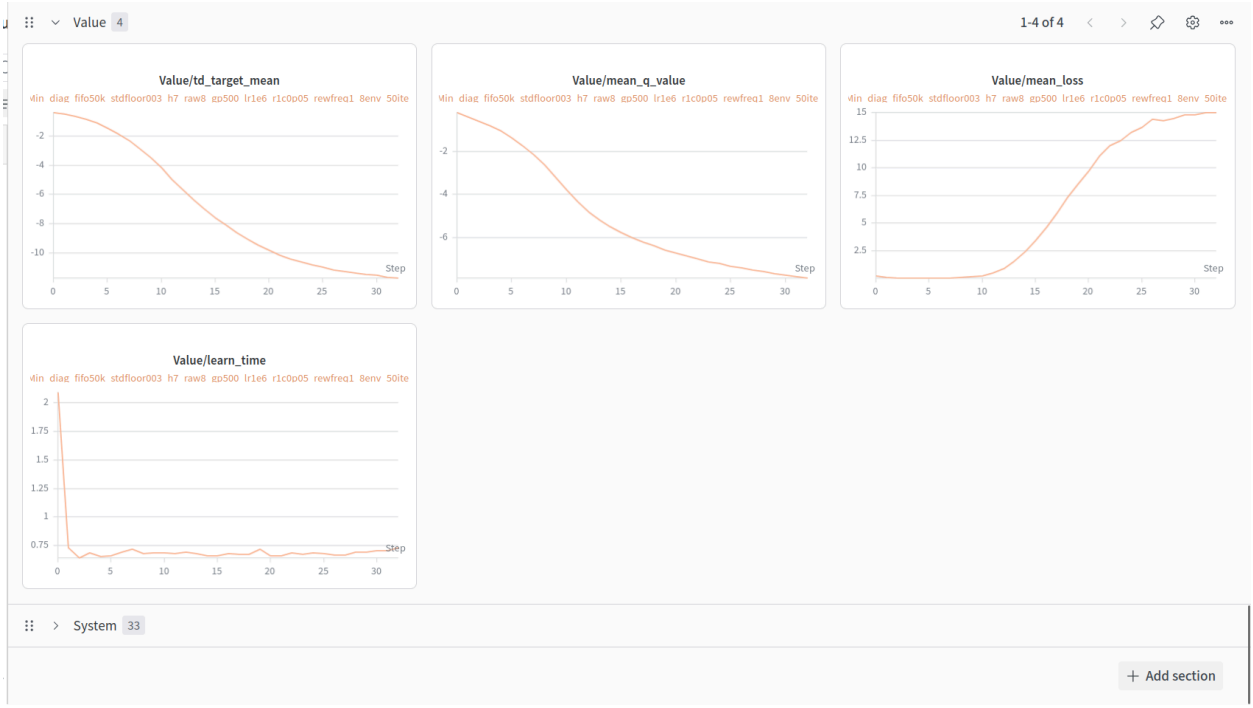
mpail2/configs/cfgs.py

mpail2/configs/defs.py

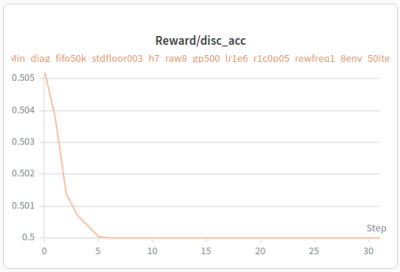
mpail2/planner.py

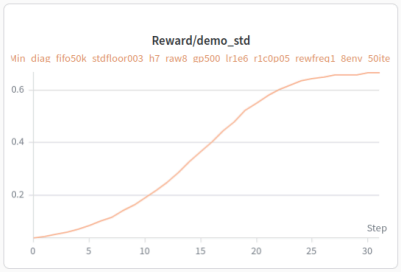
Show 3 more files

3:17 AM



Reward 21 19-21 of 21







Storage 6

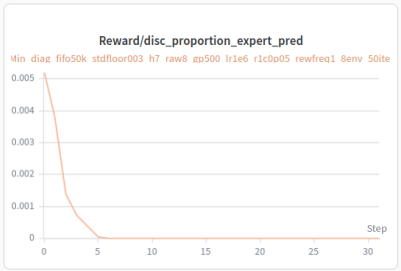
Training 2



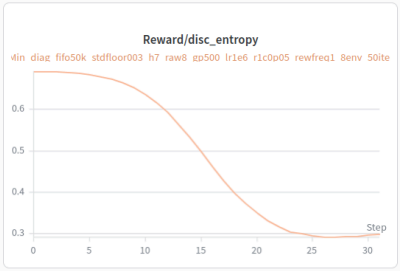


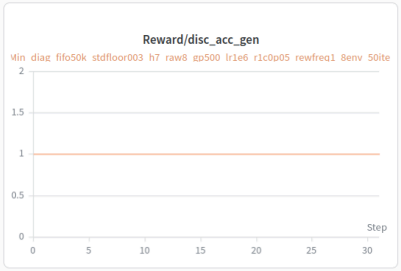
Reward 21 13-18 of 21

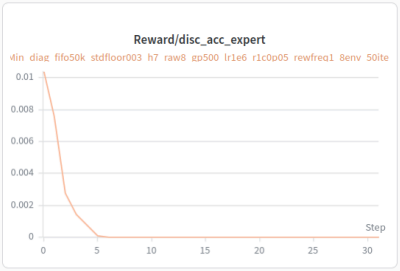








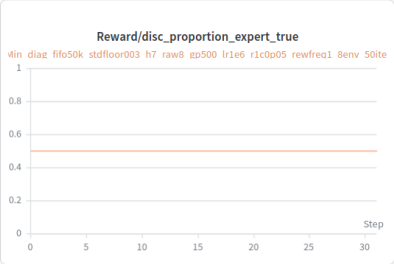


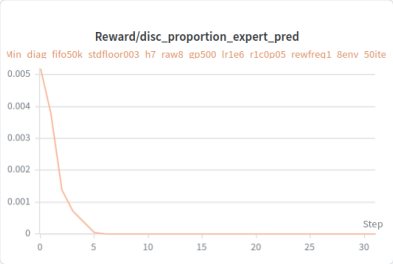


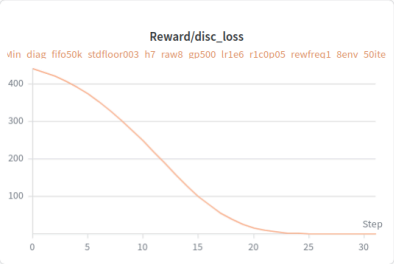
Storage 6

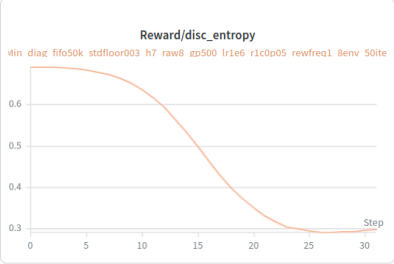
Training 2

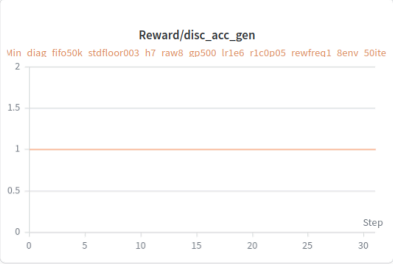
Reward 21 13-18 of 21

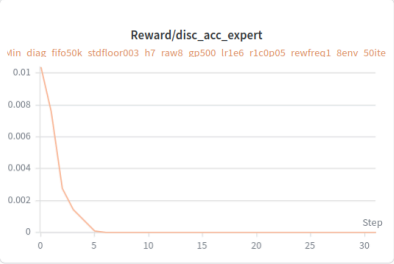








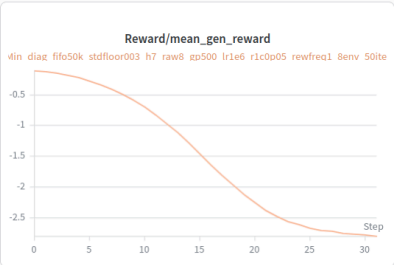


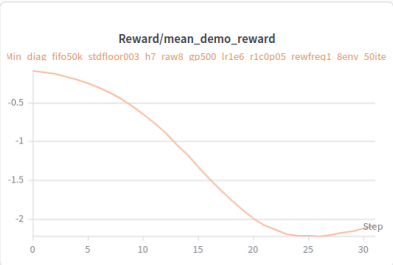


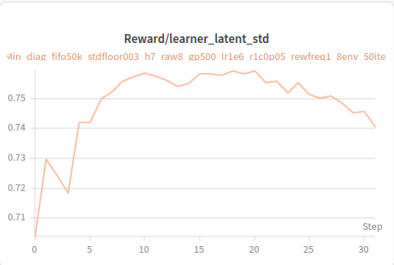
Storage 6

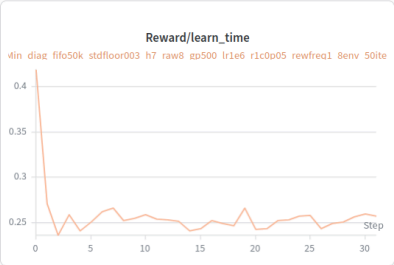
Training 2

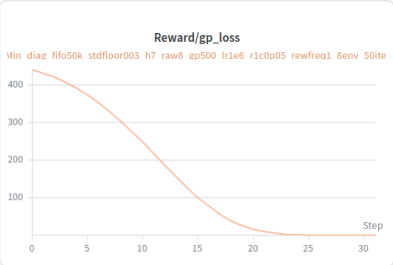
Reward 21 7-12 of 21

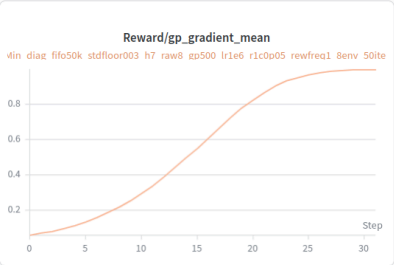








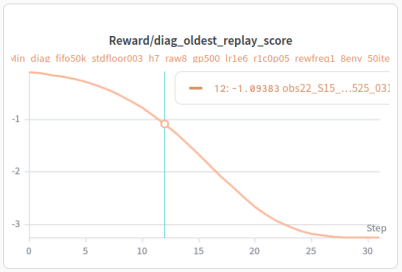


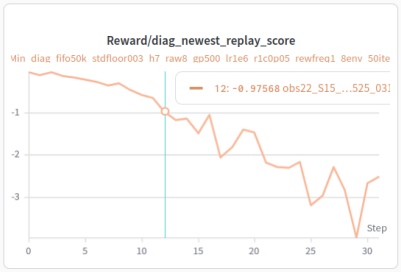


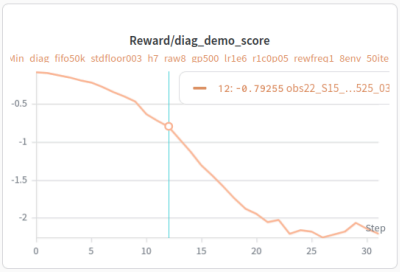
Storage 6

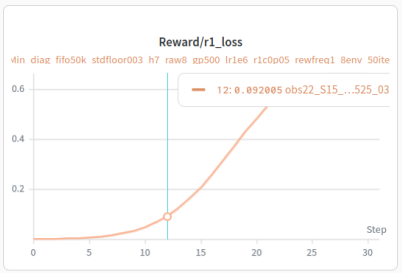
Training 2

Reward 21 1-6 of 21













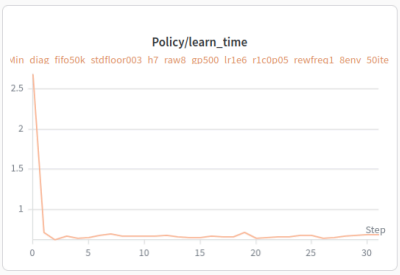
Storage 6

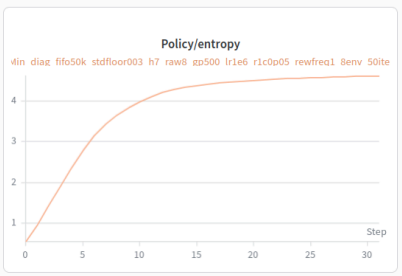
Training 2

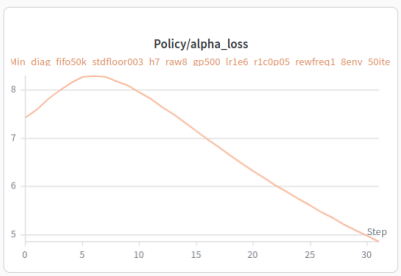
Policy 6 1-6 of 6

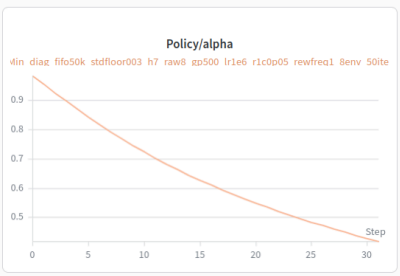












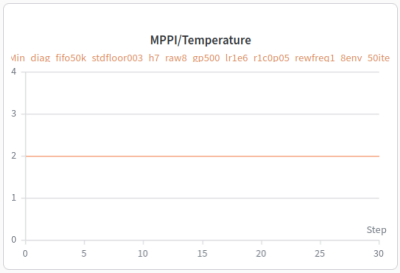
Reward 21

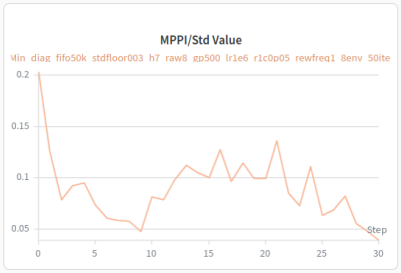
Storage 6

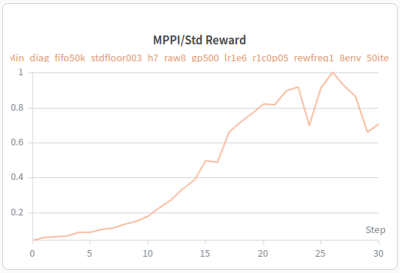
Episode_Termination 2

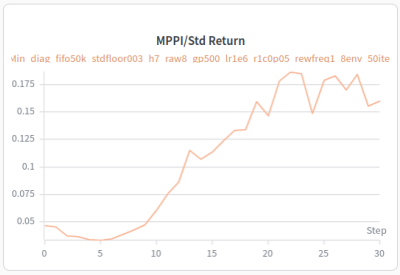
Metrics 5

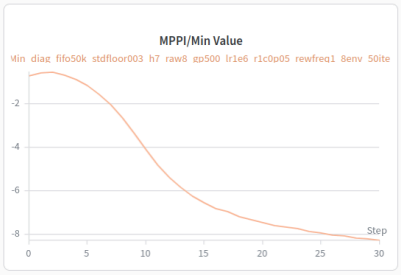
MPPI 20 1-6 of 20

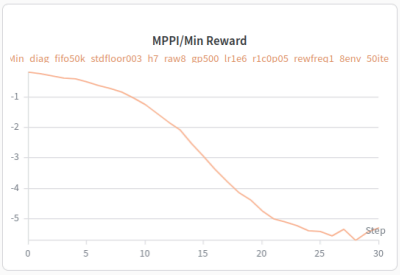










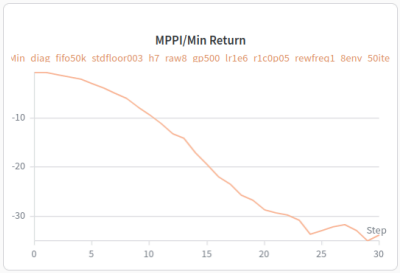


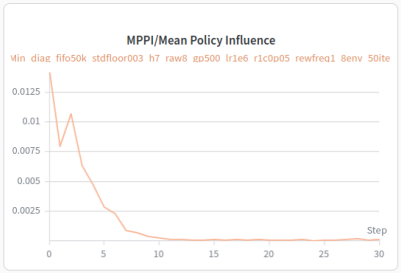
Perf 3

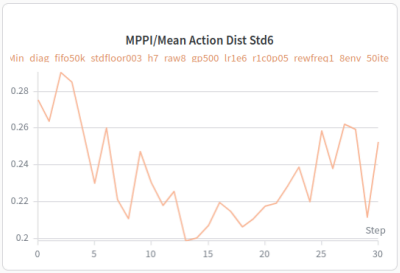
Episode_Termination 2

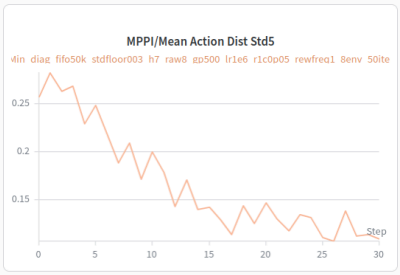
Metrics 5

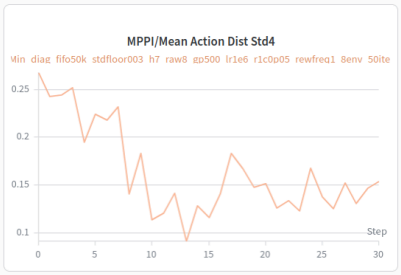
MPPI 20 7-12 of 20

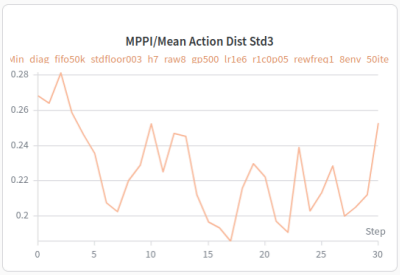










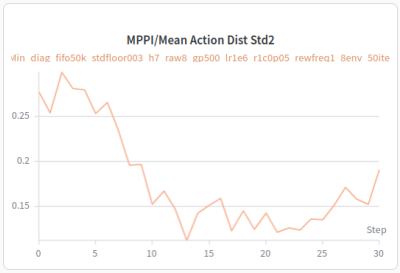


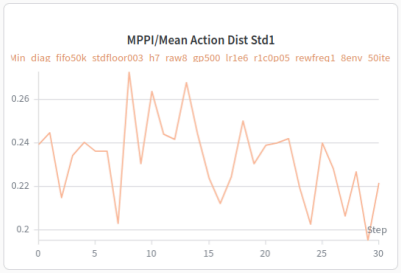
Perf 3

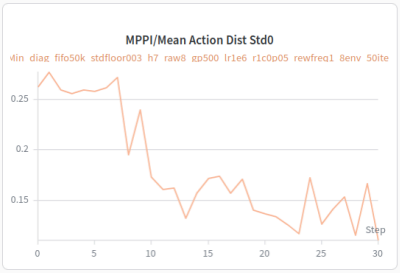
Episode_Termination 2

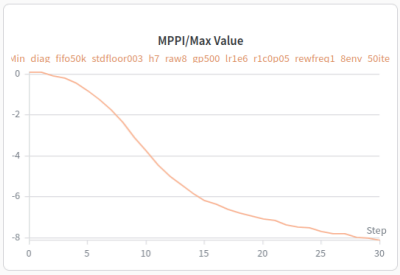
Metrics 5

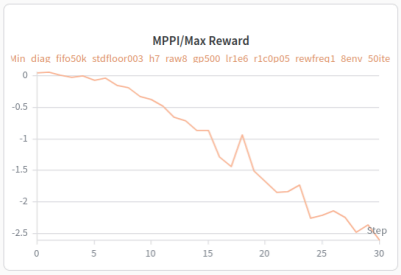
MPPI 20 13-18 of 20

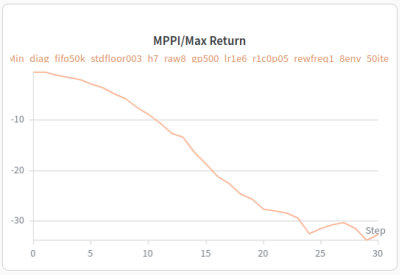












Perf 3

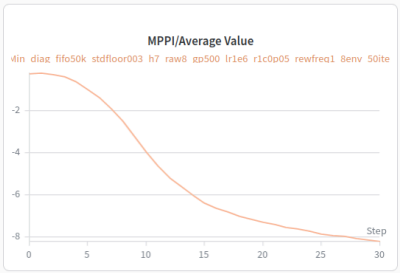
Env 11

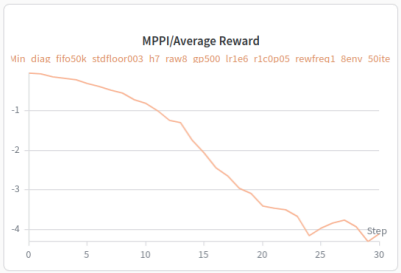
Episode_Reward 8

Episode_Termination 2

Metrics 5

MPPI 20 19-20 of 20





Perf 3

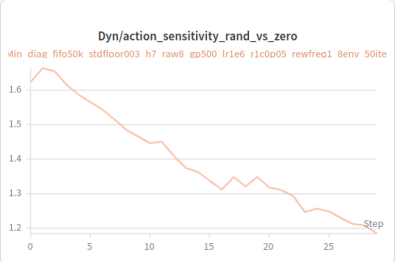
Policy 6

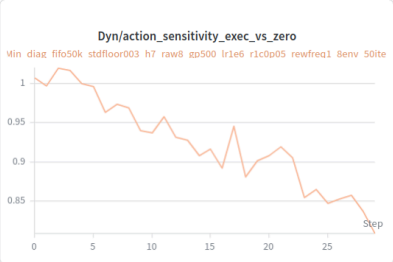
Reward 21

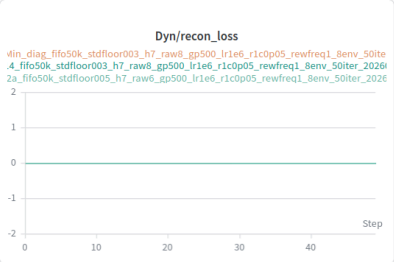
Storage 6

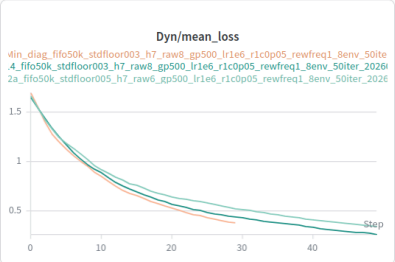
Dyn 5

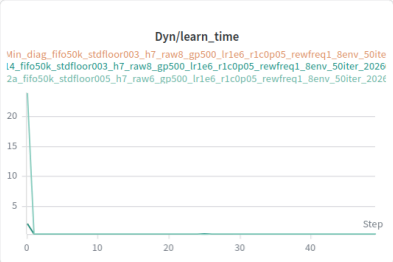
1-5 of 5









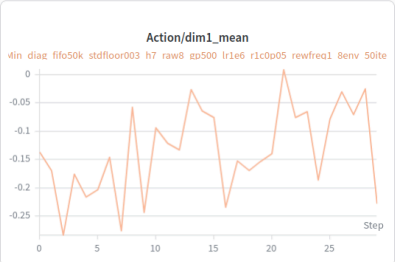


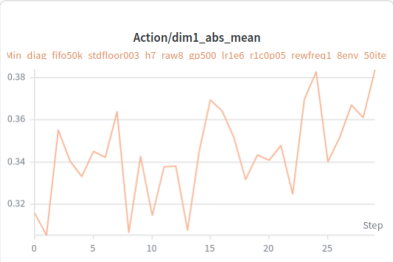
Env 11

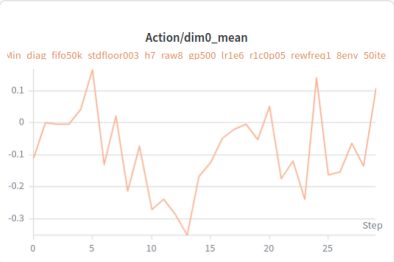
Episode_Reward 8

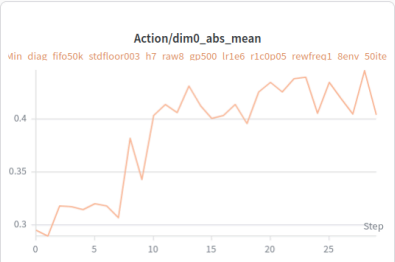
Action 30

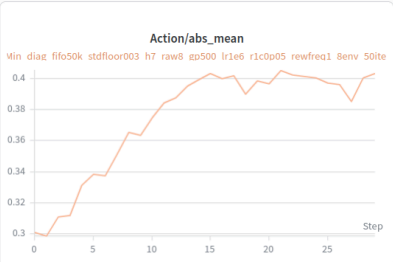
25-30 of 30

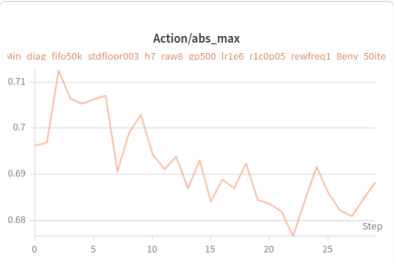








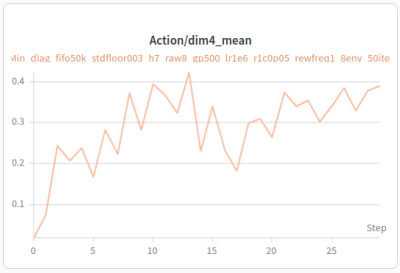


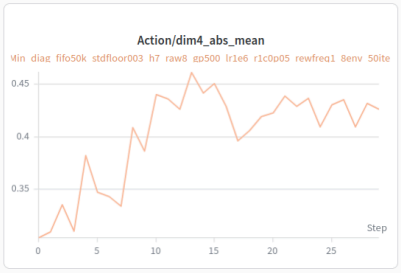


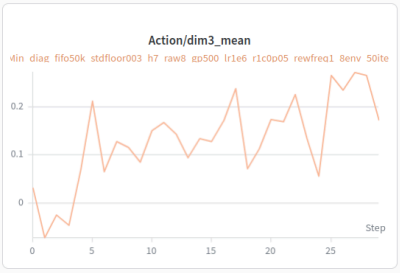
Charts 2

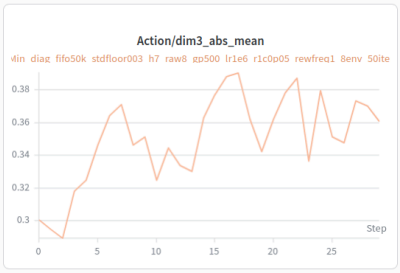
Action 30

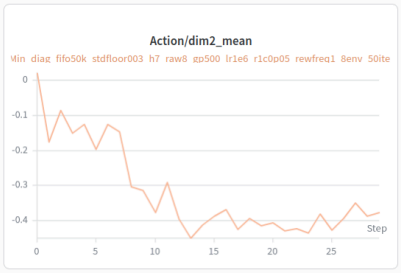
19-24 of 30

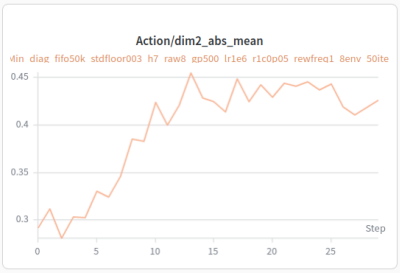








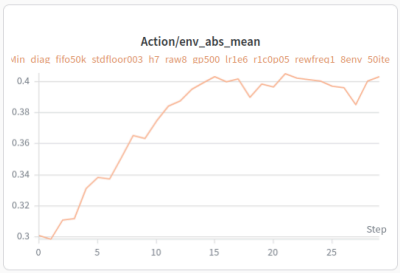


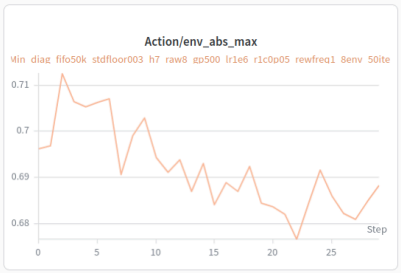


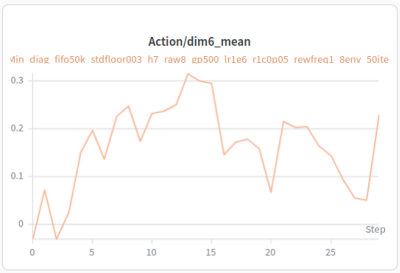
Charts 2

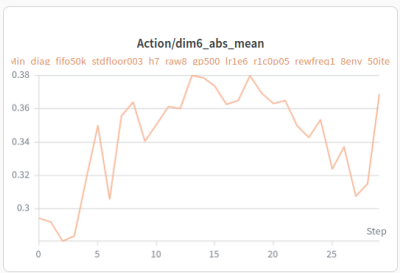
Action 30

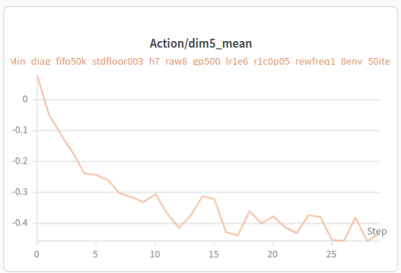
13-18 of 30

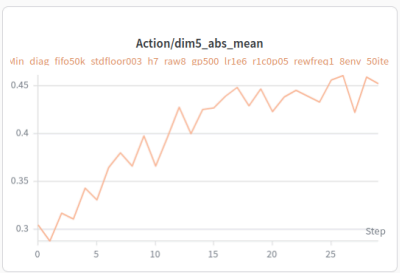








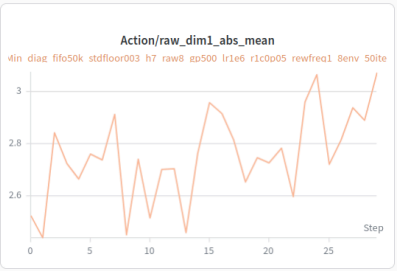


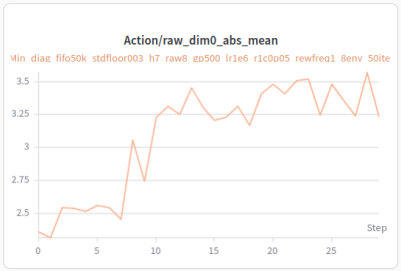


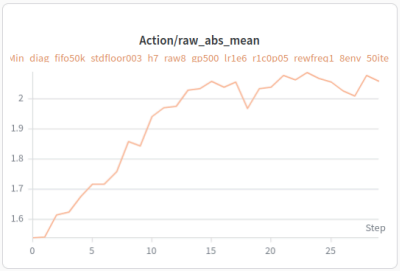
Charts 2

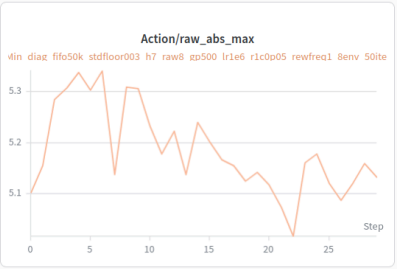
Action 30

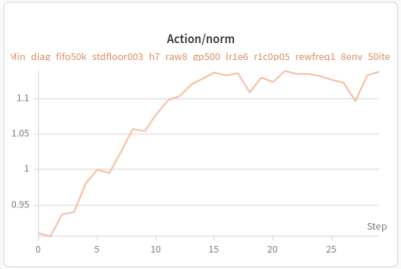
7-12 of 30

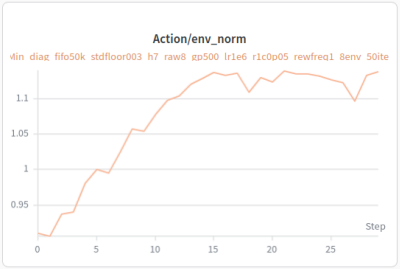








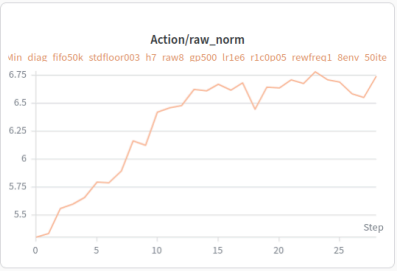


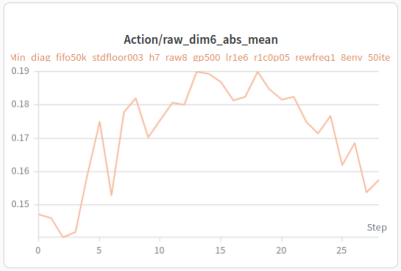


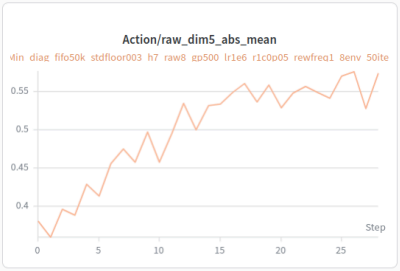
Charts 2

Action 30

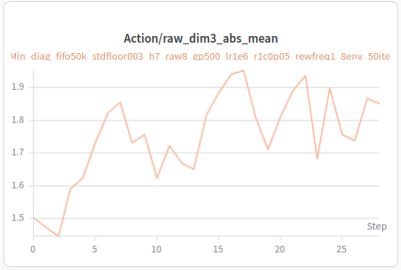
1-6 of 30













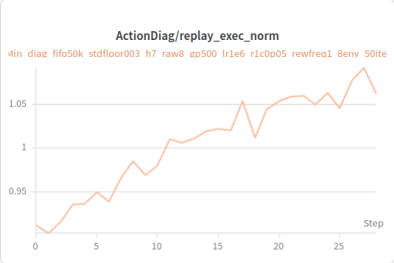
Charts 2

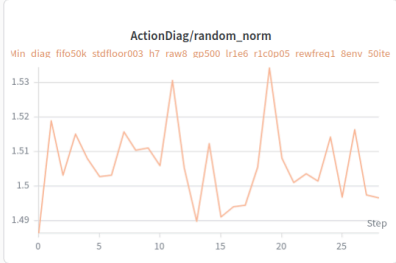
Search panels with regex

Settings | New report | Add panels

ValueDiag 7

ActionDiag 2





Action 30

Charts 2

Data 2

Dyn 5

Env 11

Episode_Reward 8

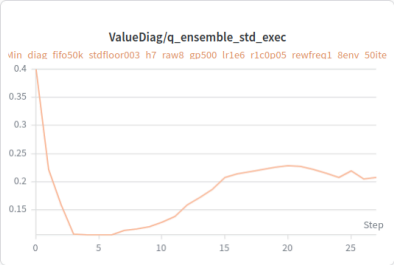
Episode_Termination 2

Search panels with regex

Settings | New report | Add panels

ValueDiag 7

7-7 of 7



ActionDiag 2

Action 30

Charts 2

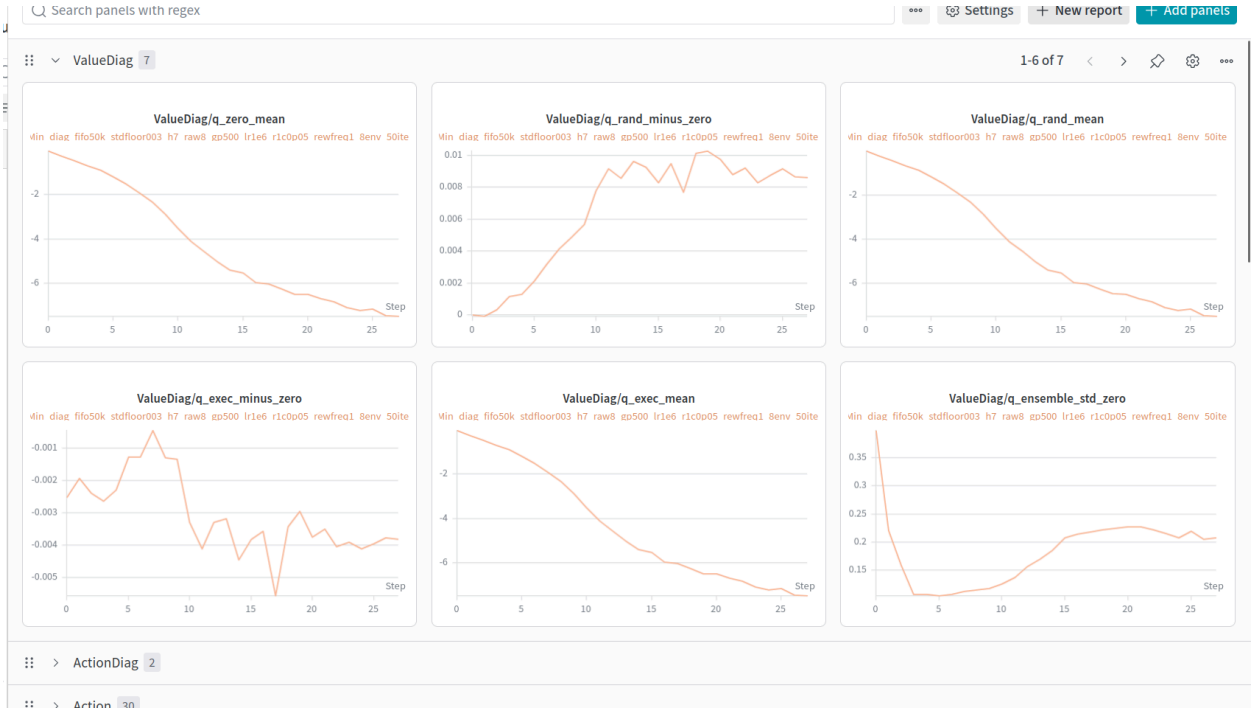
Data 2

Dyn 5

Env 11

Episode_Reward 8

Episode_Termination 2



不用接着跑了这次效果非常不好，但是从log里面你能获得有用的信息吗

3:33 AM

Worked for 4m 55s

我已经把 S15 停掉了，没有残留训练进程。

这次 log 很有用，而且结论挺明确：问题不是单纯 “ensemble avg value 高估导致不动”。我把 terminal value 改成 `min` 以后，`MPPI/Average Value` 和 `Value/mean_q_value` 都一路变得更负，`MPPI/Average Reward` 也一路变差。也就是说，保守 value 没有救回来，反而证明更根本的问题在 reward/value/policy 三者的训练信号已经坏掉了。

最关键的现象：

1. `Reward/disc_proportion_expert_pred = 0`，`disc_acc_gen = 1`，`disc_acc_expert = 0`
   这说明 discriminator/reward 基本在把所有东西都判成 generated，包括 demo。
   `Reward/mean_demo_reward` 和 `mean_gen_reward` 都一起往负数掉，只是 demo 稍微没那么差。这不是健康的 imitation reward。健康状态应该是 demo reward 相对高、gen reward 相对低，而不是两边一起塌成负值。
2. `Reward/diag_demo_score` 从接近 0 掉到约 `-2`，newest replay 掉到约 `-3`
   这个诊断特别关键：reward 网络不是只讨厌 agent，它连 expert demo 也越来越讨厌。于是 value 的 TD target 必然越来越负，policy/MPPI 得到的训练信号就是“所有未来都很差”。
3. `MPPI/Mean Policy Influence` 很快掉到接近 0
   代码里这个值来自 planner.py (line 324)，意思是 MPPI 最终权重里 policy proposal 占多少。接近 0

说明 learned policy 采样完全不被 planner 选中，控制主要由 MPPI/CEM 噪声分布决定。也就是说 policy 学出来的东西没有进入实际执行闭环。

4. `Action/raw_norm` 反而从约 `5.3` 涨到 `6.7`，`Action/env_norm` 也在涨
   所以视频里的“慢、几乎不可见”不是因为代码实际输出了接近 0 的动作。更像是动作数值很大，但对任务状态无效：可能动作方向/尺度/控制器映射不对，或者 MPPI 选的是会让 latent reward/value 看起来“没那么坏”的动作，但物理上没有产生有效接近/插入。
5. `ValueDiag/q_exec_minus_zero` 约为负，`q_rand_minus_zero` 只有很小正值
   这也很有信息：value 并没有明显觉得当前 executed action 比 zero action 好。也就是说 critic 没学到“执行某些动作能推进任务”，它基本只在学一个随 reward 整体下沉的坏势函数。

本质判断：现在失败链条是
reward discriminator 退化为几乎全判 generated -> demo/gen reward 一起负漂移 -> value TD target 全面变负 -> MPPI 用 learned reward/value 评估时只是在一堆坏轨迹里选相对不坏的噪声动作 -> policy influence 归零 -> 行为看起来无效。

所以接下来不要再优先调 horizon / raw action limit / value avg-vs-min 了。最该查的是 reward 训练本身：label/sign、reward 输出是否需要 centering/normalization、GP/R1 是否把 reward 压成全负、以及 demo/replay latent 分布是否因为 encoder/normalizer 导致 reward 无法稳定区分。这个 S15 基本把方向从“MPPI value exploit”推进到了“reward 学习目标退化”。

所以接下来应该怎么做啊？
3:44 AM
Worked for 1m 22s
下一步不要再跑完整训练了。现在最该做的是 先把 reward 标定问题单独拆出来，因为 S15 已经说明：planner/value/policy 都是在吃一个绝对值不断负漂移的 reward。

我建议按这个顺序做：

1. 先做 reward centering / reward shift 诊断
现在 `update_reward()` 是 WGAN 形式：

```
loss = -demo_reward.mean() + gen_reward.mean()
```

这个目标只关心 `demo - gen` 的差，不关心整体加一个常数。也就是说 demo=-2、gen=-3 在 WGAN 看来是“能区分”的，但对 MPPI/value 来说，所有 reward 都是负的，TD target 就会一路往负数滚。
所以 S15 里 `disc_acc_expert=0` 不只是分类失败，更重要的是 reward 的绝对零点漂了。

下一步要加两个可撤回选项：

- 只用于 planner/value：`reward_center = reward - running_demo_reward_mean`
- 或更稳一点：`reward_center = reward - 0.5 * (demo_mean + gen_mean)`

这不是改算法核心，是 reward calibration。WGAN/AIRL 类 reward 本来就有 reward shaping/constant ambiguity，直接把 raw score 当 planner reward 很容易炸。

2. 把 GP 从 500 大幅降下来

`gp=500` 现在太像把 reward 网络训练成“平滑但整体漂移”的形状了。S15 里 `gp_gradient_mean` 接近 1，但 reward margin 没有变成可用的正负分界，反而 demo/gen 一起掉。

我下一组会先试：

```
gp=10 或 gp=50
r1=0.01
reward_lr=1e-6 或 3e-7
```

不要再用 `gp=500` 作为主方向。

3. 先别优化 policy，让 MPPI/reward/value 单独站稳

`MPPI/Mean Policy Influence≈0` 说明 policy proposal 基本没被采纳。此时 policy loss 怎么变都不重要，policy 只是跟着坏 planner 学。

下一步短跑应该看：

```
Reward/diag_demo_score 是否稳定高于 replay
MPPI/Average Reward 是否不再一路下坠
Value/td_target_mean 是否不再单调负漂移
Action/raw_norm 高但行为无效是否改善
```

4. 我建议下一次只跑一组 30-50 tier：S16

配置方向：

```
replay FIFO 保持
std_floor=0.03
reward_update_every=1
h=5 或 h=7
value_return_type=avg 先改回去
gp=10
r1=0.01
加 reward_center_for_planning=True
```

核心验证问题只有一个：reward 被中心化后，MPPI 是否还会 Average Reward↓、Value↓、Policy Influence≈0。

如果 S16 仍然失败，但 reward demo/gen margin 是正常的，那问题再转向 dynamics/action mapping。

如果 S16 明显稳住，那根因就基本坐实：之前不是任务难，也不是 fulltask，不是 horizon，而是 WGAN reward 的绝对尺度/零点漂移被直接喂给 value/MPPI，导致整个闭环学歪。

3:45 AM

好，就跑这个
replay FIFO 保持
std_floor=0.03
reward_update_every=1
h=7

value_return_type=avg 先改回去
gp=10
r1=0.01
加 reward_center_for_planning=True

你开始搞吧
3:55 AM
Worked for 4m 31s

S16 已经跑起来了，首个 iteration 正常，没有配置错误。

运行信息：

- 训练 PID：`37733`
- W&B run：`h20a36d2`
- run 目录：obs22_S16_rewardCenter...
- 启动日志：s16_launcher_20260525_035831.log

我做的改动是可撤回的：

- 加了 `reward_center_for_planning` / `reward_center_mode`
- reward 网络仍然按原来的 WGAN loss 训练
- 只在 value target 和 MPPI rollout reward 上使用 `reward - center`
- `center = 0.5 * (demo_reward_mean + gen_reward_mean)`

首个日志看起来至少方向对了：

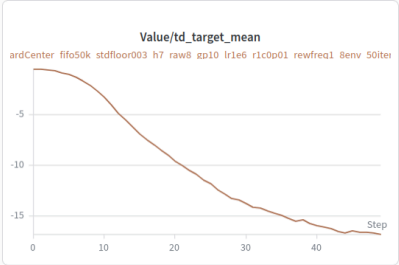
```
[iter 0/50] rew=1.1659 ep_ret=1.166 succ=0.000 done=8 abn=0 to=8
disc_demo=0.023 disc_gen=-0.009 dyn_loss=1.6780
```

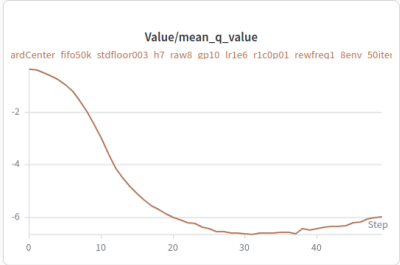
重点盯这几个：

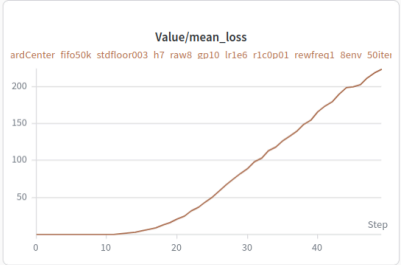
- `Reward/center`
- `Reward/diag_demo_score` **vs** `diag_newest_replay_score`
- `MPPI/Average Reward`
- `MPPI/Average Value`
- `MPPI/Mean Policy Influence`
- `Value/td_target_mean`
- `Action/raw_norm`

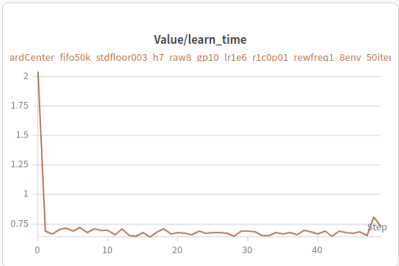
如果这次 `MPPI/Average Reward` 不再一路塌，value 也不再跟着负漂移，那基本就坐实 reward 零点漂移是主因。

Value 4 1-4 of 4





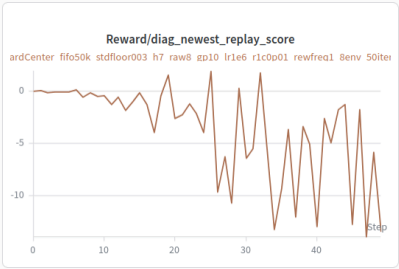


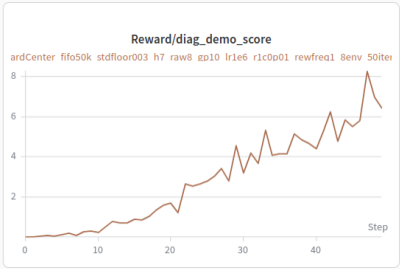


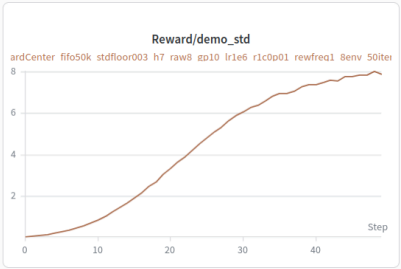
System 33

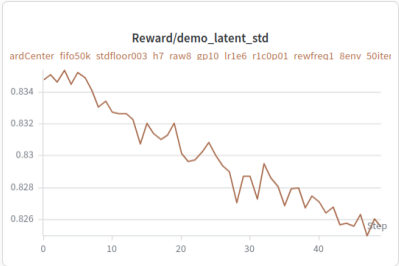
+ Add section

Reward 22 19-22 of 22





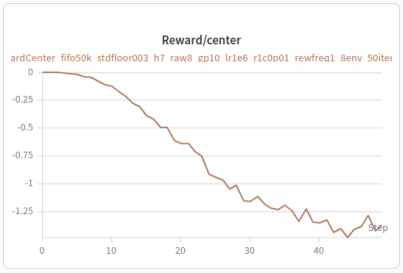


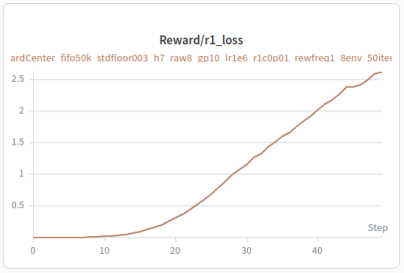


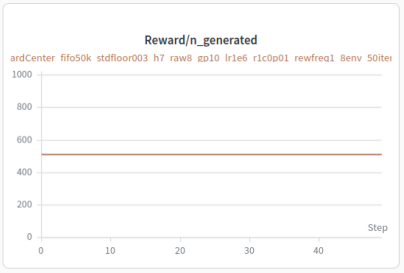
Storage 6

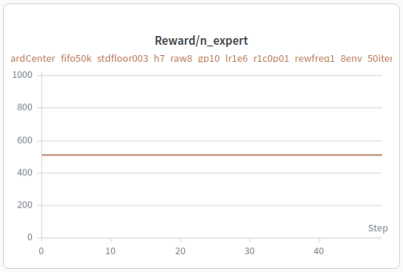
Training 2

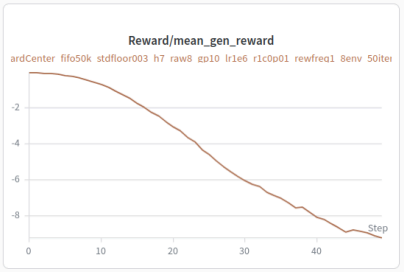
Reward 22 1-6 of 22

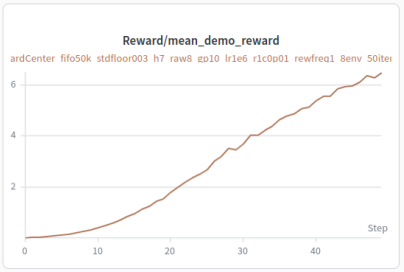








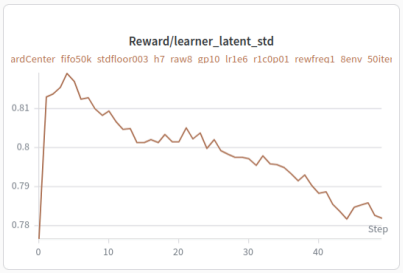


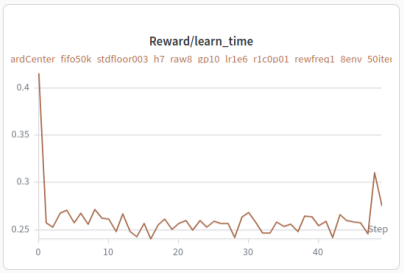


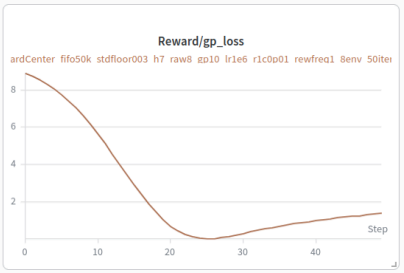
Storage 6

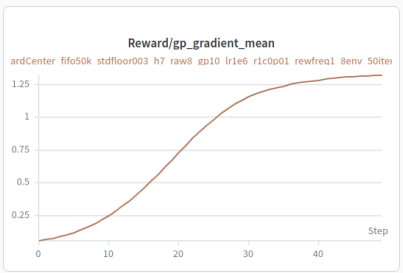
Training 2

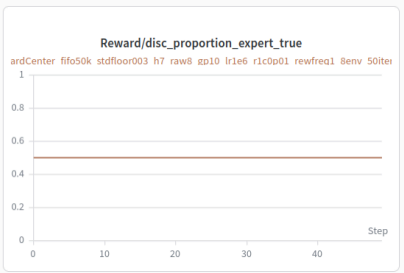
Reward 22 7-12 of 22









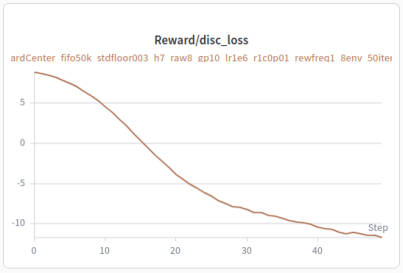


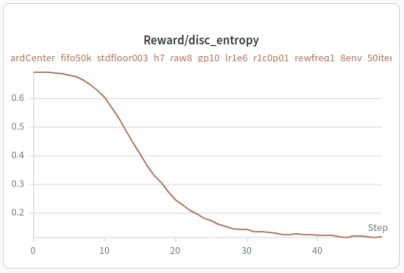


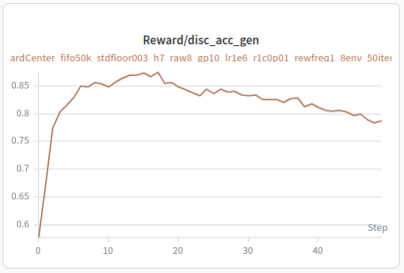
Storage 6

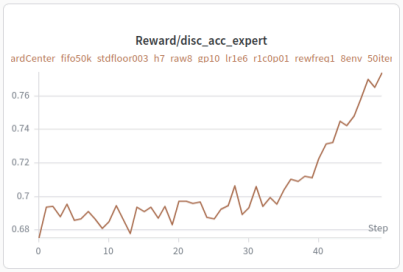
Training 2

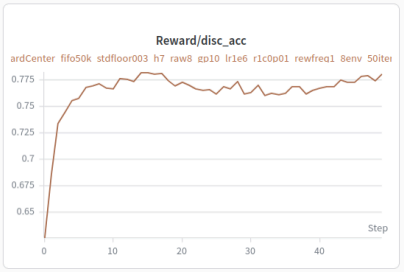
Reward 22 13-18 of 22









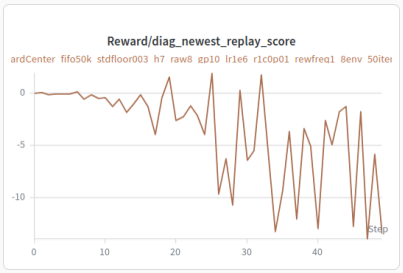


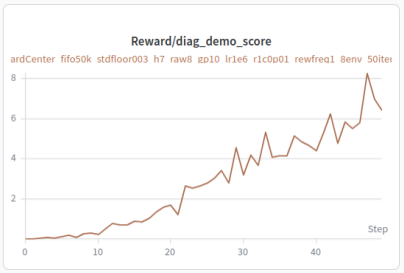


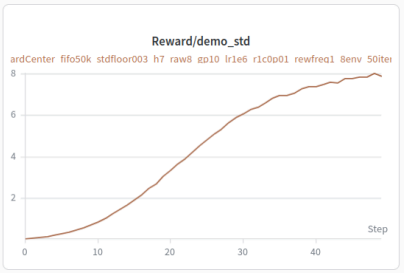
Storage 6

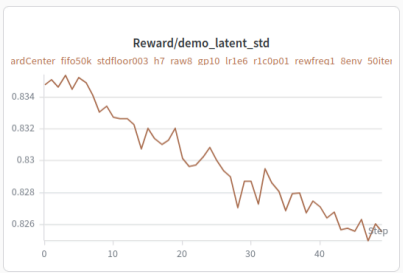
Training 2

Reward 22 19-22 of 22





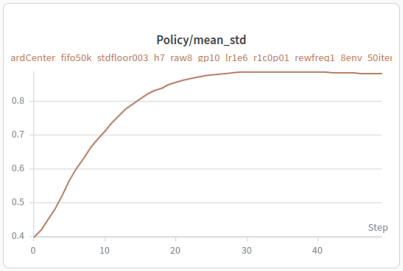


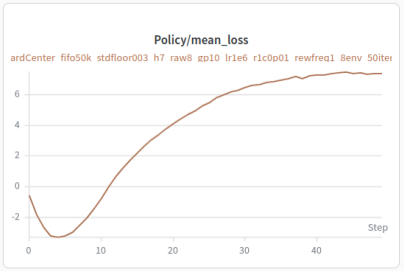


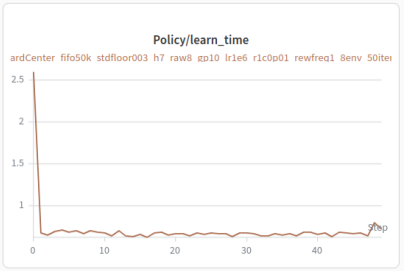
Storage 6

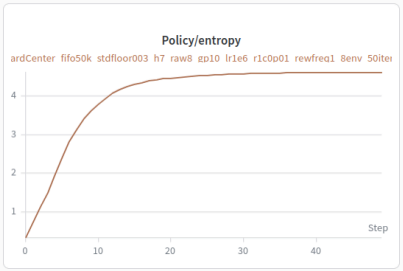
Training 2

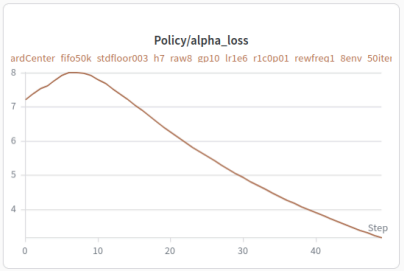
Policy 6 1-6 of 6

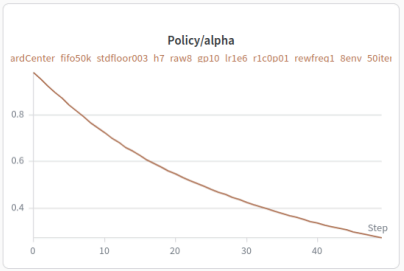










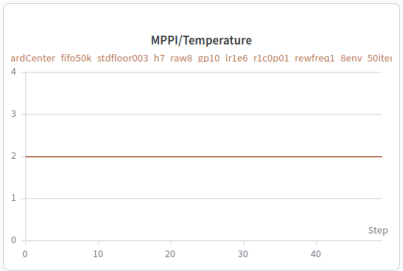


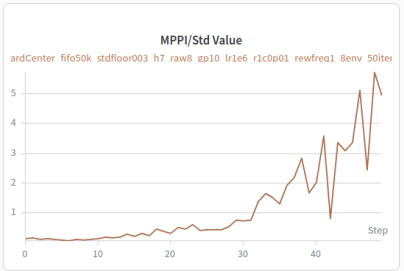
Reward 22 19-22 of 22

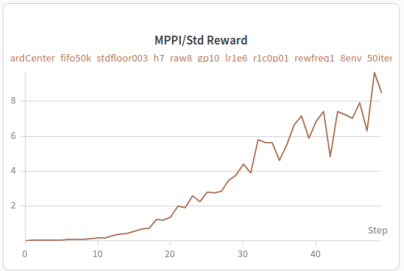
Episode_Termination 2

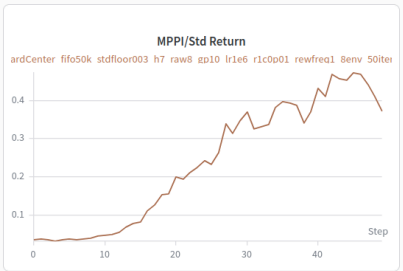
Metrics 5

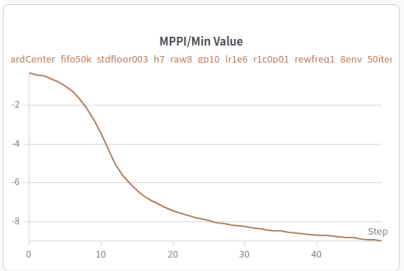
MPPI 20 1-6 of 20

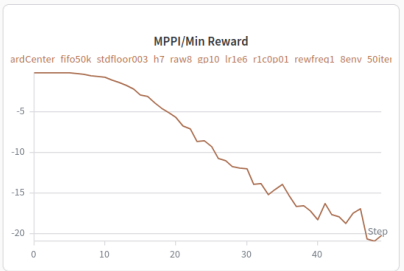










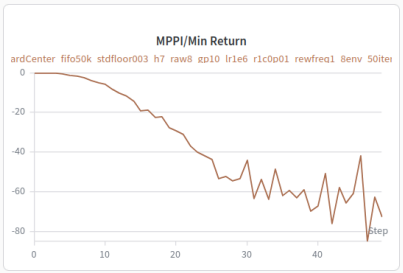


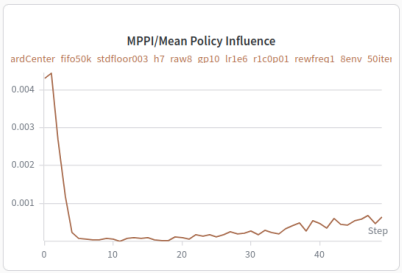
Perf 3

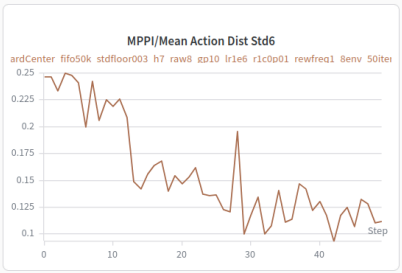
Episode_Termination 2

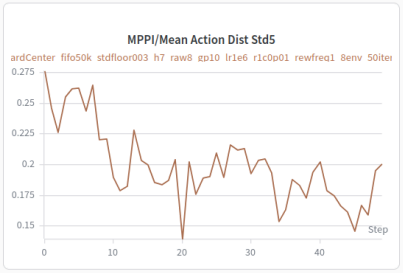
Metrics 5

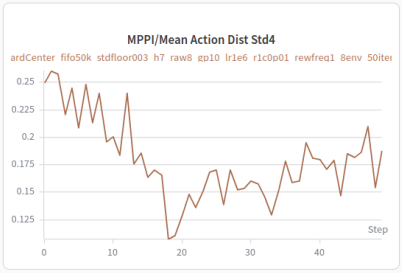
MPPI 20 7-12 of 20

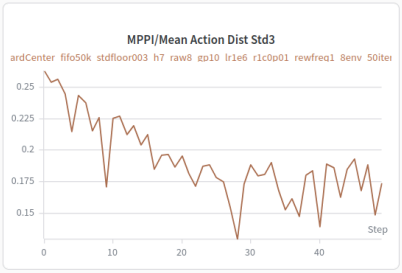








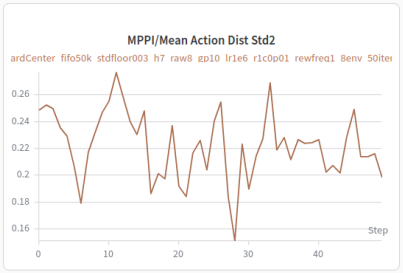


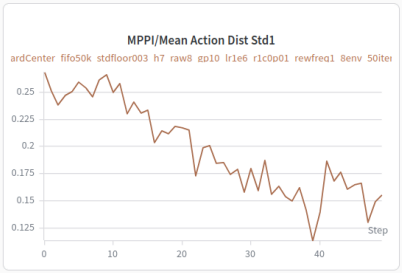


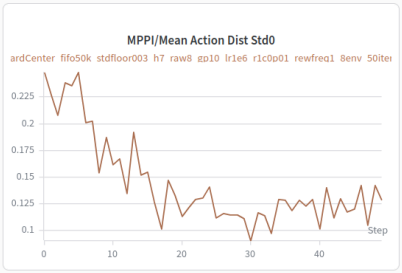
Episode_Termination 2

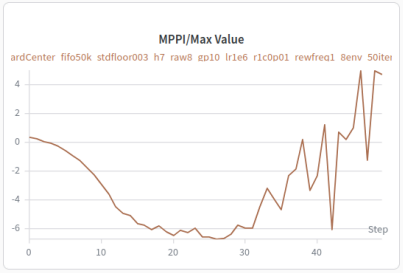
Metrics 5

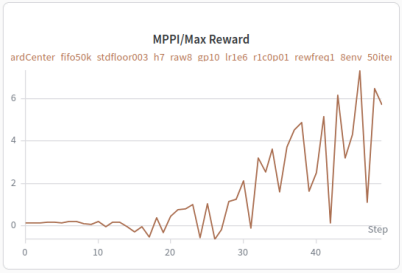
MPPI 20 13-18 of 20

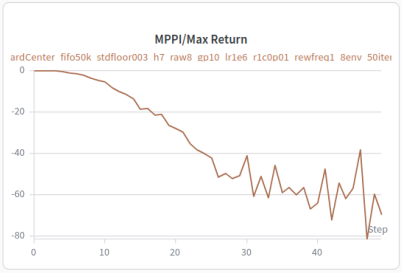




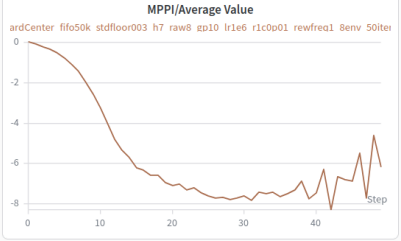




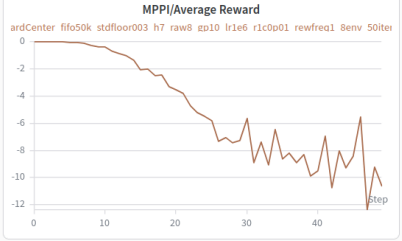




MPPI/Average Value



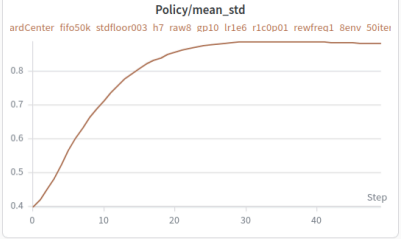
MPPI/Average Reward



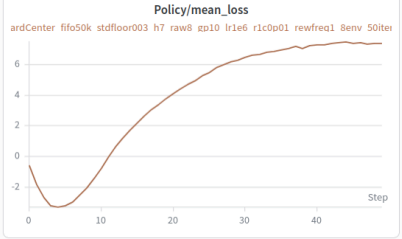
Perf 3

Policy 6 1-6 of 6

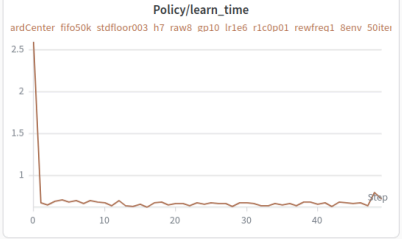
Policy/mean_std



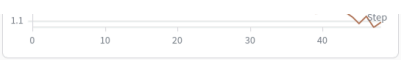
Policy/mean_loss



Policy/learn_time



Policy/entropy



Policy/alpha_loss

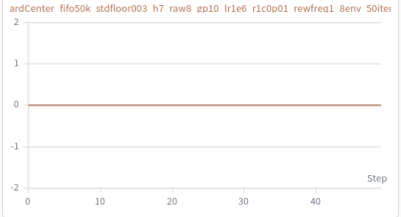


Policy/alpha

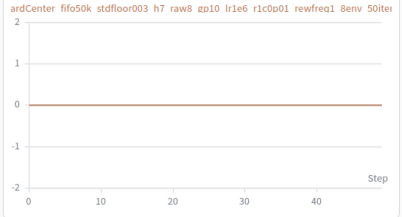
Env 11

Episode_Reward 8 1-6 of 8

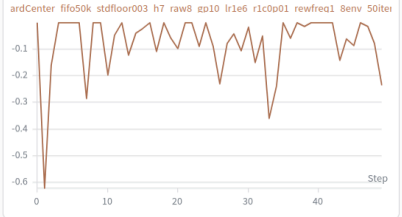
Episode_Reward/success_reward



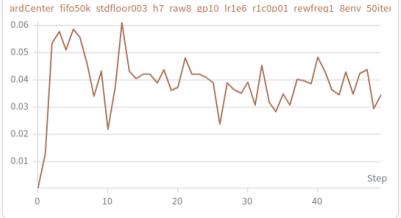
Episode_Reward/progress_context



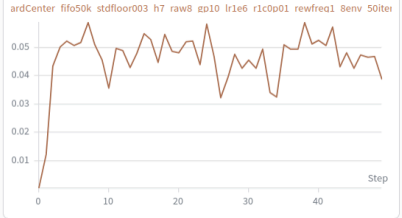
Episode_Reward/joint_vel



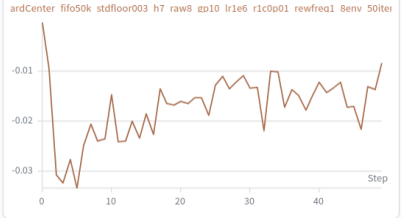
Episode_Reward/ee_asset_distance



Episode_Reward/dense_success_reward

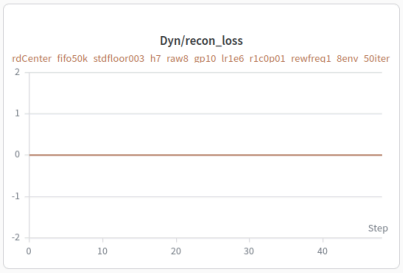


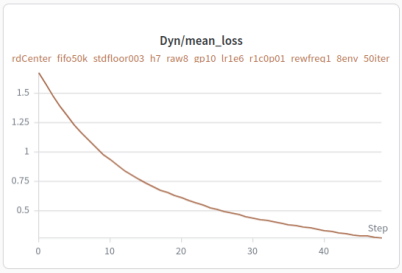
Episode_Reward/action_rate

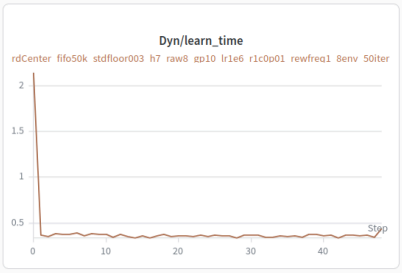


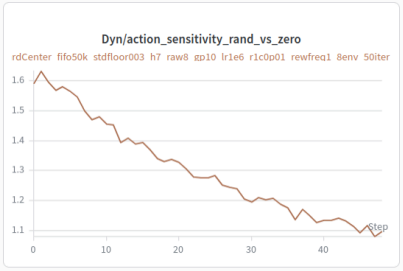
Data 2

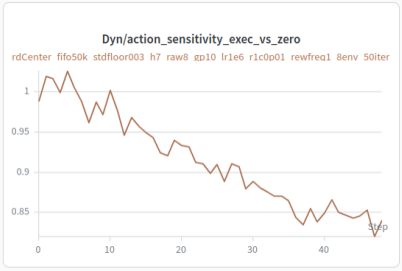
Dyn 5 1-5 of 5









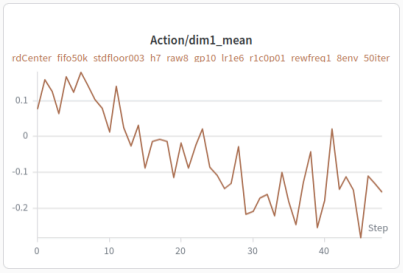


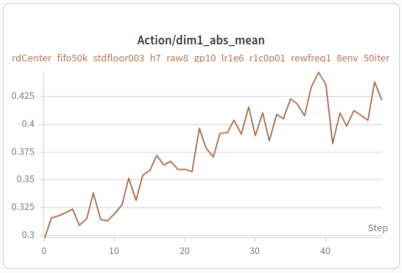
Env 11

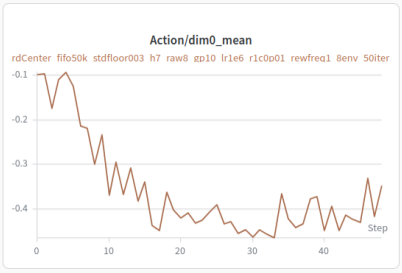


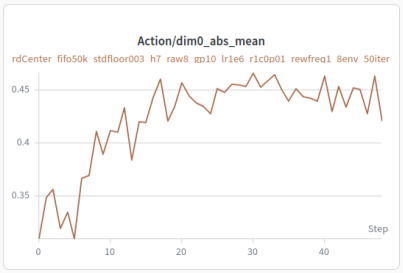


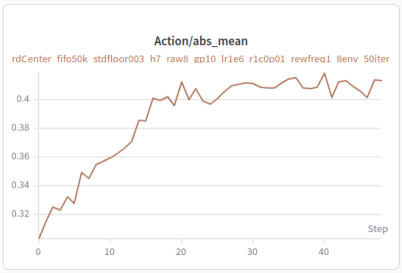
Action 30 25-30 of 30

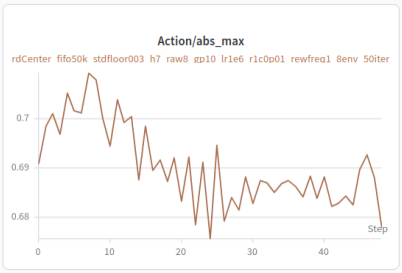








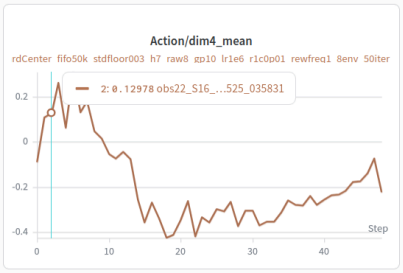


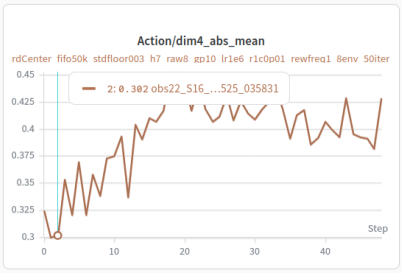


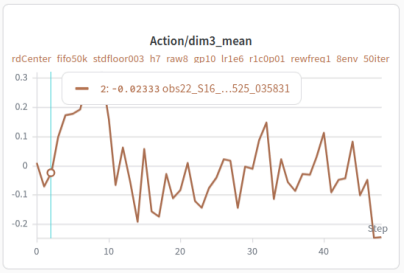


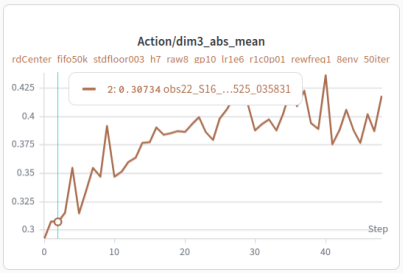


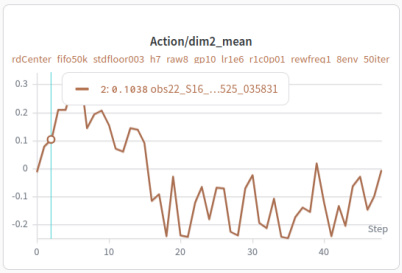
Action 30 19-24 of 30

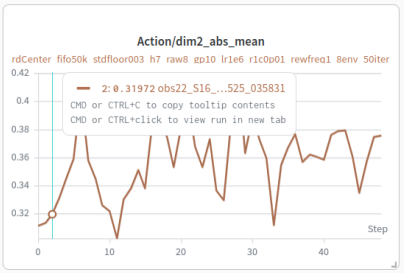








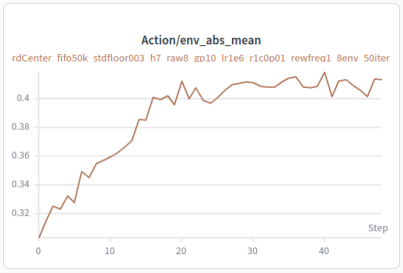


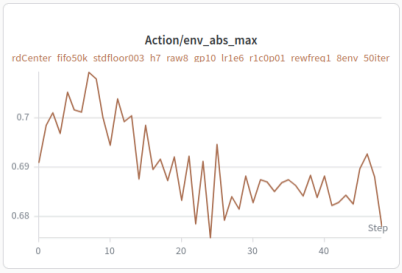


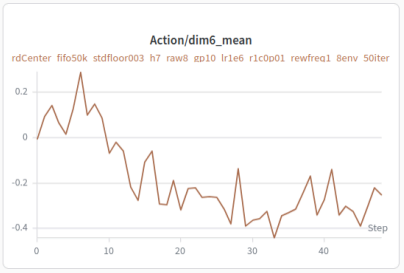


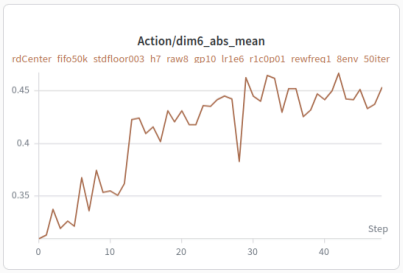


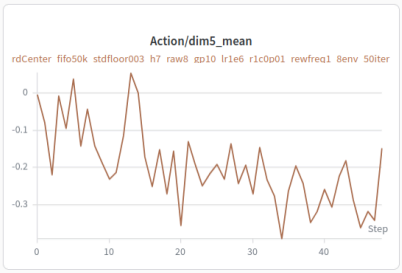
Action 30 13-18 of 30

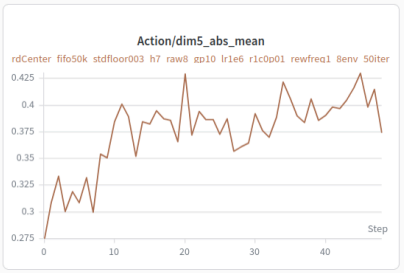










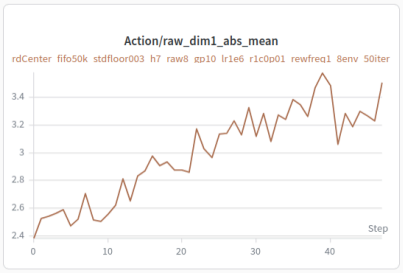


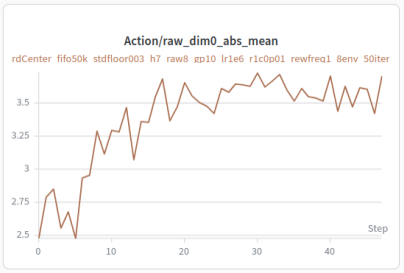


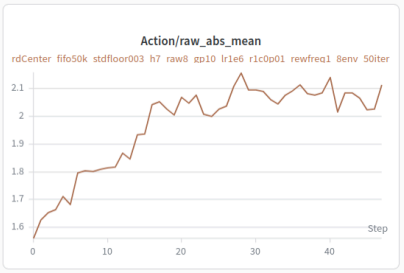


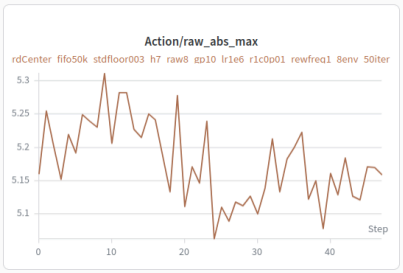
Action 30

7-12 of 30

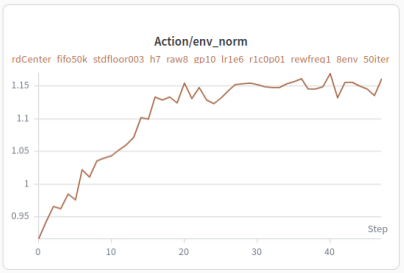




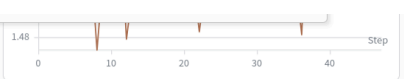










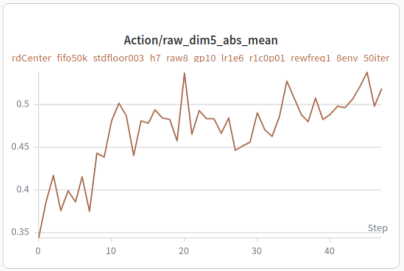


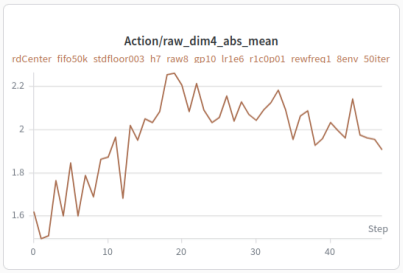
Action 30

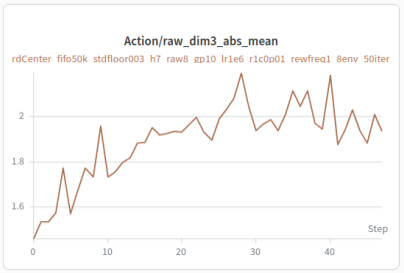
1-6 of 30

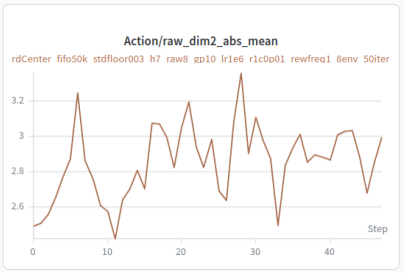










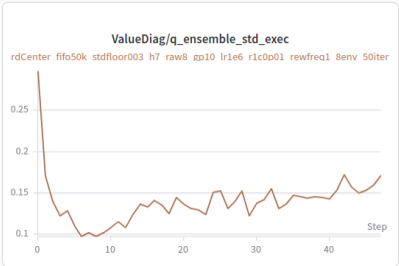


Search panels with regex

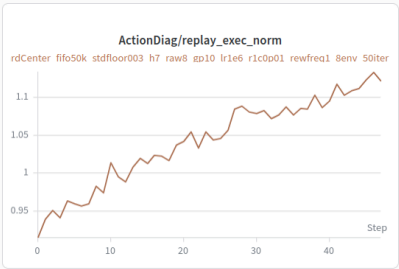
Settings | New report | Add panels

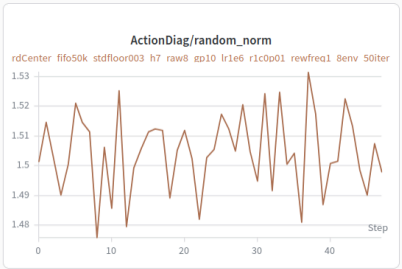
ValueDiag 7

7-7 of 7



ActionDiag 2





Action 30

Search panels with regex

Settings | New report | Add panels

ValueDiag 7

7-7 of 7

ValueDiag/q_ensemble_std_exec
rdCenter fifo50k stdfloor003 h7 raw8 gp10 lr1e6 r1c0p01 rewfreq1 8env 50iter
0.25
0.2
0.15
0.1
Step
0
10
20
30
40

ActionDiag 2

Action 30

Charts 2

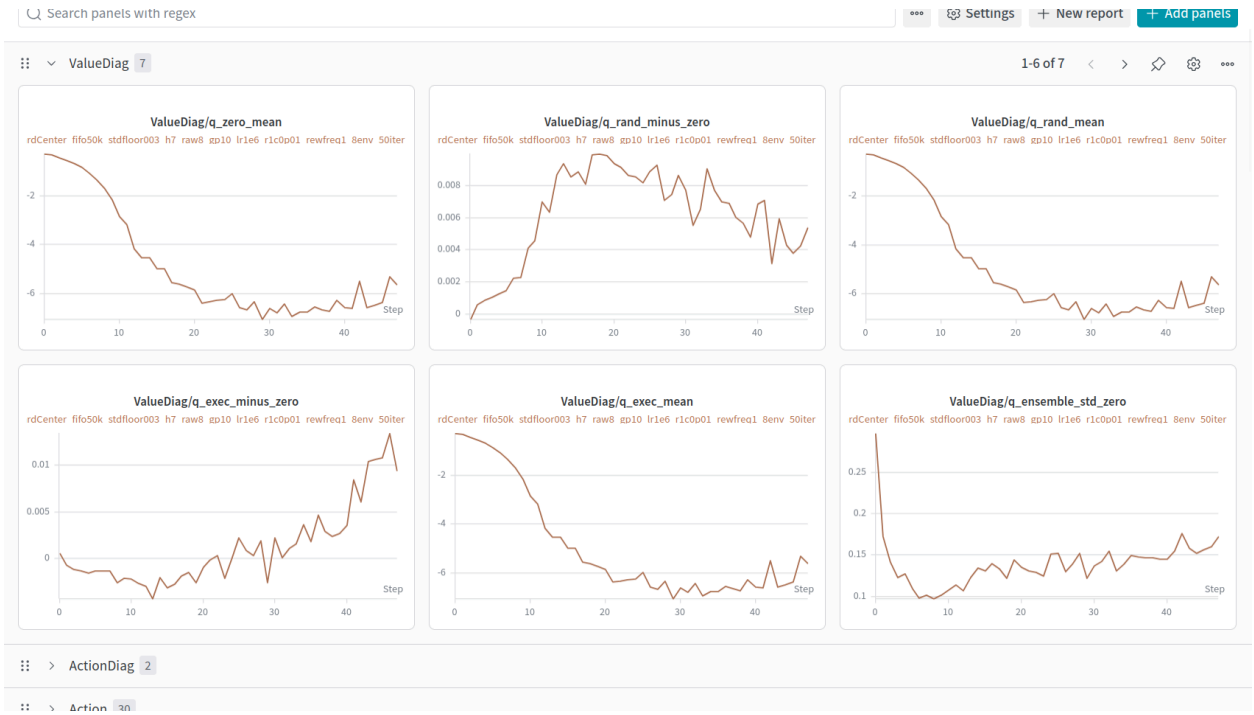
Data 2

Dyn 5

Env 11

Episode_Reward 8

Episode_Termination 2



你看log和代码算法逻辑训练逻辑详细分析这次实验效果

4:23 AM

Worked for 1m 4s

S16 跑完了，结论很有价值：reward centering 方向验证了一半，但这次实现/配置没有救训练，反而暴露出 value target 还有一条未中心化路径，critic 被打爆了。

最重要的现象

1. Reward 终于能区分 demo/gen 了

   S15 是 demo/gen 一起掉；S16 变成 `mean_demo_reward ↑ 到 6+`，`mean_gen_reward ↓ 到 -9`，`disc_acc≈0.77`，`disc_acc_expert≈0.77`，`disc_acc_gen≈0.78`。

   这证明之前“reward 学不出排序”不是绝对的，降低 `gp=10` 后 discriminator 有区分能力了。

2. 但 value 彻底崩了

   `Value/td_target_mean` 掉到约 `-17`，`Value/mean_loss` 涨到 `200+`。这不是普通不稳定，是 TD target 跑飞。

   关键代码在 learner.py (line 681)：我只 center 了第一步 `rewards_batch`。但后面的 `_n_step_return_lambda(...)` 在 learner.py (line 799) 里又直接调用 `reward_fn(...)`，这部分还是 raw reward，没有 subtract center。

   所以 value target 其实是：第一步 centered reward + 后续 raw reward + bootstrapped Q。这会把 raw gen reward 的大负数继续灌进 critic。

3. MPPI 仍然没被 policy 影响

   `MPPI/Mean Policy Influence` 很快接近 0。说明 policy proposal 仍不进 elites，实际控制还是 MPPI 噪声/CEM 分布。policy loss 和 entropy 看着动，但对实际行为没贡献。

4. MPPI reward/value 仍然恶化
   `MPPI/Average Reward` 掉到约 `-10`，`Average Value` 掉到约 `-8`。这说明 planner 的 rollout 评估仍被坏 value/raw reward 牵着走。centering 没有形成“可优化的正向 imitation signal”。
5. 动作不是太小
   `Action/raw_norm` 仍在 `6.5` 左右，`env_norm≈1.15`。所以现在不是“输出动作幅度小”的问题，而是 动作有效性/方向不对 + planner 评估信号坏。

本质判断
S16 证明两件事：

- 对：reward 的零点/尺度确实是核心问题之一。降低 GP 后 demo/gen margin 出来了。
- 但：只做 midpoint centering 不够，而且我这版 centering 没覆盖 value 的 n-step future reward，所以 critic 被 raw negative future rewards 打爆。

下一步应该改什么
先修实现，不急着换大方向：

1. `_n_step_return_lambda` 里的 `_all_rewards` 也必须 subtract `reward_center`。
2. `planner.td_return()` 的 terminal value 不需要 center，但 value 本身要先稳定，否则 terminal value 还是坏的。
3. `gp=10` 可能太弱，reward margin 爆太大；下一组建议 `gp=50, r1=0.05`，不是回 500。
4. 加 reward scale/clip，比如 planner/value 使用：
   `centered_reward = clamp(raw_reward - center, -5, 5)`
   这还是 reward calibration，不是改算法主体。

所以 S17 我建议：修完整 centering + reward clamp + gp=50/r1=0.05 + h=7 继续 50 tier。S16 已经说明方向比 S15 更接近根因，但现在 critic 这条链还没被真正救住。

好，继续帮我按这个方向实验
4:25 AM
Worked for 4m 29s

S17 已经跑起来了，首个 iteration 正常。

这次我做了两件关键修正：

1. 补全 reward calibration 覆盖范围
   S16 只 center 了 value 的第一步 reward，漏掉了 `_n_step_return_lambda()` 里的 future rewards。现在这三处都统一走 `center + clip`：
   - MPPI planner rollout reward
   - value TD 第一跳 reward
   - value lambda-return future reward

2. 加入 reward clip
   planner/value 使用的 reward 现在是：
3. `calibrated_reward = clamp(raw_reward - center, -5, 5)`
   reward 网络本身仍然按 WGAN loss 训练，没有改 reward 学习目标。

S17 配置：

- `std_floor=0.03`
- `reward_update_every=1`
- `h=7`
- `value_return_type=avg`
- `gp=50`
- `r1=0.05`
- `reward_center_for_planning=True`
- `reward_center_mode=midpoint`
- `reward_clip_for_planning=5.0`
- replay FIFO 保持

运行信息：

- 训练 PID：`41518`
- W&B run：`vxc8xvrb`
- run 目录：S17 run dir
- 启动日志：s17_launcher_20260525_042839.log

首个 iteration：

```
[iter 0/50] rew=1.2264 ep_ret=1.226 succ=0.000 done=8 abn=0 to=8
disc_demo=-0.024 disc_gen=-0.018 dyn_loss=1.6950
```

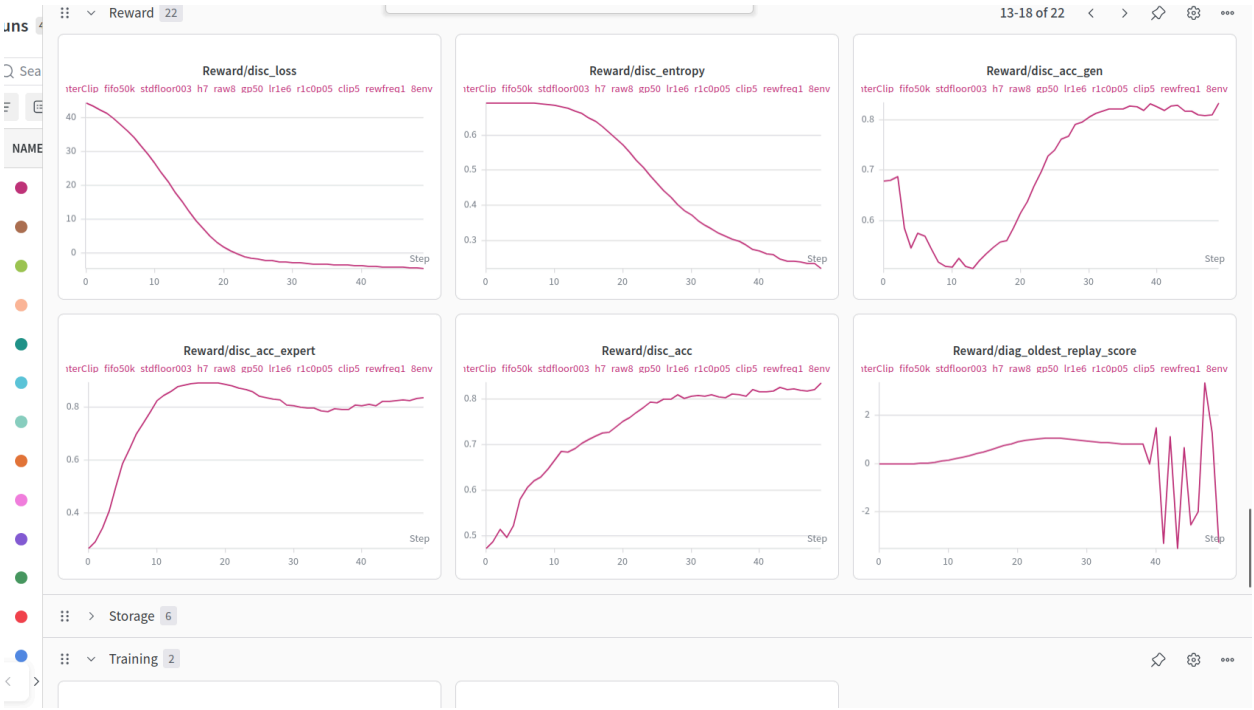
这次最该看：

- `Value/mean_loss` 是否不再像 S16 一样炸到 200+
- `Value/td_target_mean` 是否不再单调掉到 -17
- `MPPI/Average Reward` 是否不再崩到 -10
- `Reward/mean_demo_reward - mean_gen_reward` 是否有 margin 但不过度爆炸
- `MPPI/Mean Policy Influence` 是否能从 0 附近抬起来一点

如果 S17 value 稳住但 policy influence 还是 0，那下一步就转向 planner sampling/policy proposal 的问题；如果 value 仍崩，那说明 reward scale 还要更强约束，或者 value target 的 lambda/horizon 本身在这个任务上太激进。

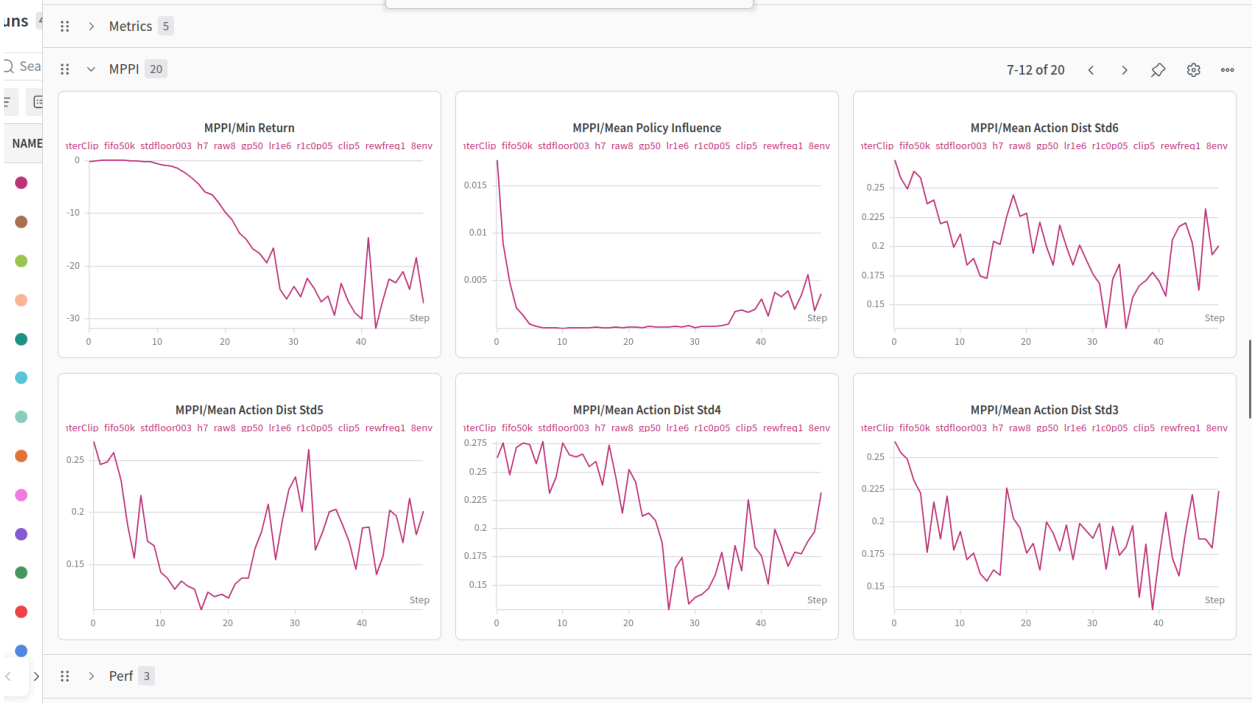



Reward 22
7-12 of 22
Reward/learner_latent_std
Reward/learn_time
Reward/gp_loss
Reward/gp_gradient_mean
Reward/disc_proportion_expert_true
Reward/disc_proportion_expert_pred
Step
Storage 6
Training 2


Reward 22
13-18 of 22
Reward/disc_loss
Reward/disc_entropy
Reward/disc_acc_gen
Reward/disc_acc_expert
Reward/disc_acc
Reward/diag_oldest_replay_score
Step
Storage 6
Training 2

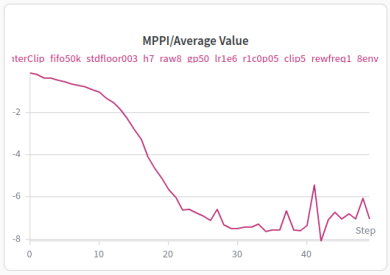

Reward 22
19-22 of 22
Reward/diag_newest_replay_score
Reward/diag_demo_score
Reward/demo_std
Reward/demo_latent_std
Storage 6
Training 2
Perf 3
Policy 6
1-6 of 6
Policy/mean_std
Policy/mean_loss
Policy/learn_time
Policy/entropy
Policy/alpha_loss
Policy/alpha
Step

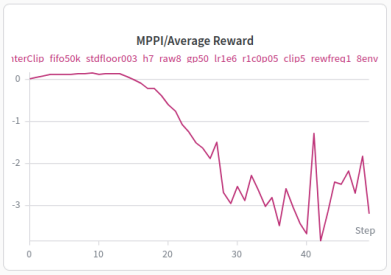

Metrics 5
MPPI 20
1-6 of 20
MPPI/Temperature
MPPI/Std Value
MPPI/Std Reward
MPPI/Std Return
MPPI/Min Value
MPPI/Min Reward
Step
Perf 3


Metrics 5
MPPI 20
7-12 of 20
MPPI/Min Return
MPPI/Mean Policy Influence
MPPI/Mean Action Dist Std6
MPPI/Mean Action Dist Std5
MPPI/Mean Action Dist Std4
MPPI/Mean Action Dist Std3
Step
Perf 3

Metrics 5

MPPI 20 19-20 of 20

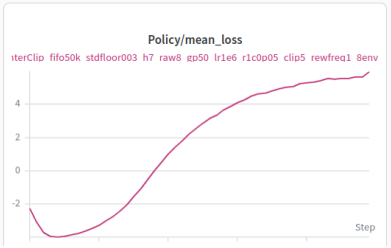


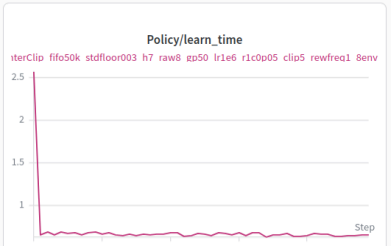


Perf 3

Policy 6 1-6 of 6

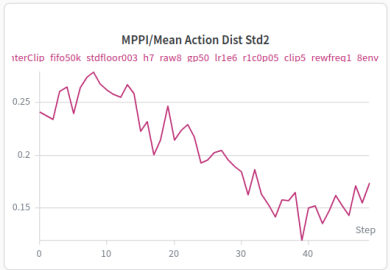


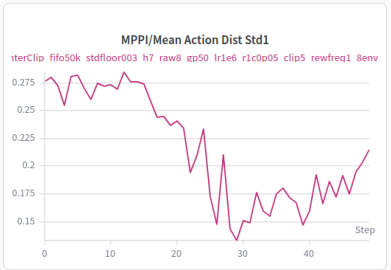


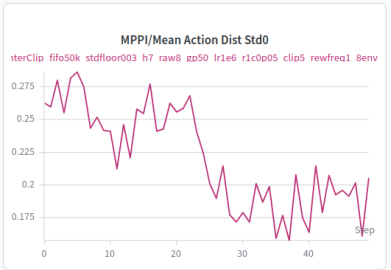


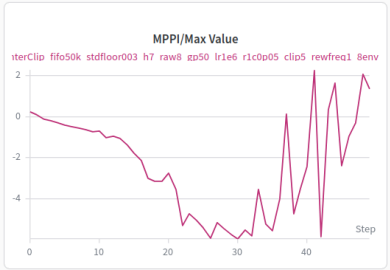
Metrics 5

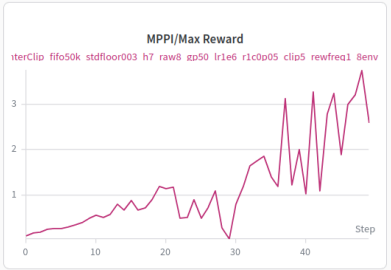
MPPI 20 13-18 of 20

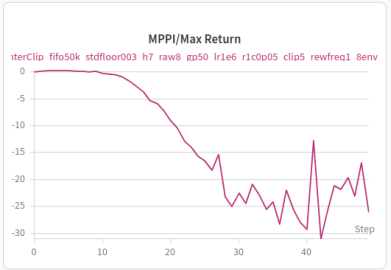










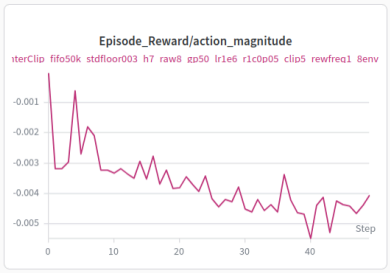


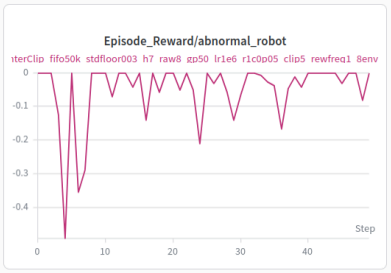
Perf 3

Env 11

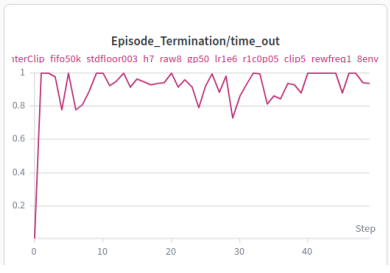
Episode_Reward 8

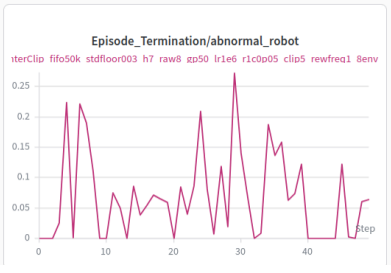
7-8 of 8





Episode_Termination 2



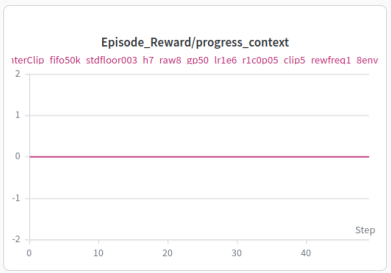


Env 11

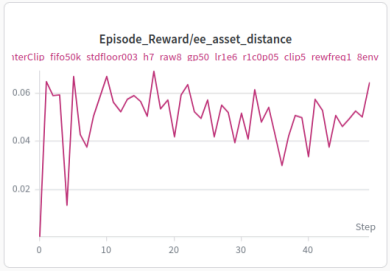
Episode_Reward 8

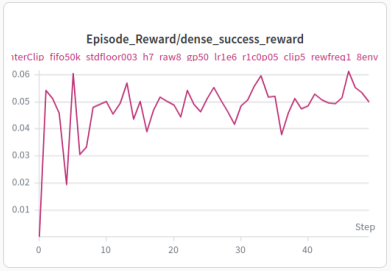
1-6 of 8

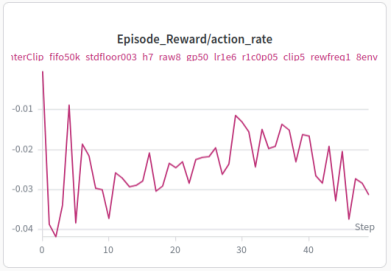








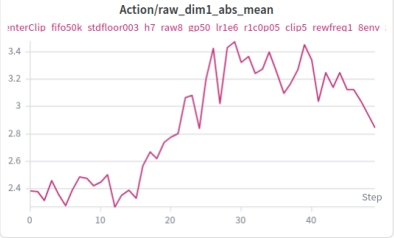


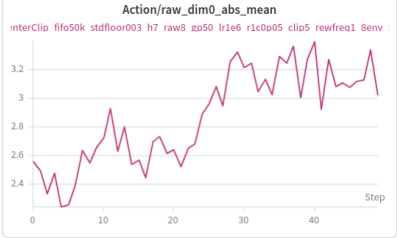


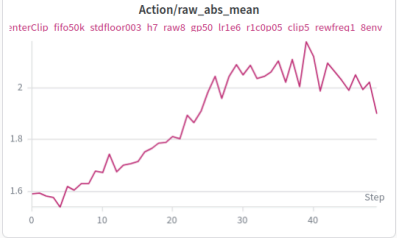
Episode_Termination 2

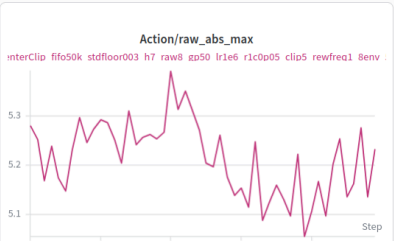

Data 2
Dyn 5
1-5 of 5
Dyn/recon_loss
Dyn/mean_loss
Dyn/learn_time
Dyn/action_sensitivity_rand_vs_zero
Dyn/action_sensitivity_exec_vs_zero
Env 11
Episode_Reward 8
1-6 of 8
Action 30
1-6 of 30
Action/raw_norm
Action/raw_dim6_abs_mean
Action/raw_dim5_abs_mean
Action/raw_dim4_abs_mean
Action/raw_dim3_abs_mean
Action/raw_dim2_abs_mean
Charts 2
Step

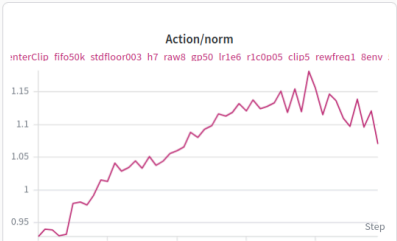
Action 30 7-12 of 30

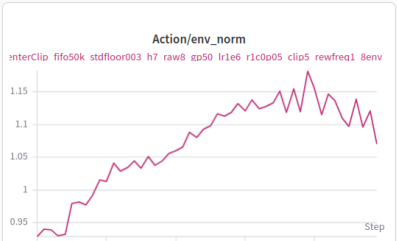






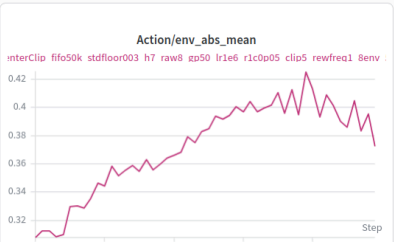


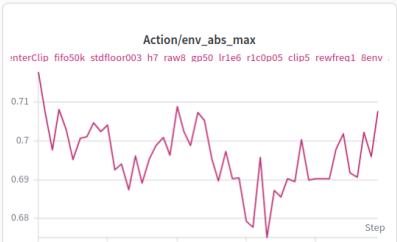


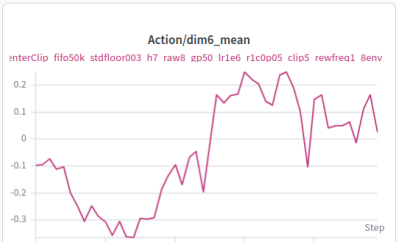


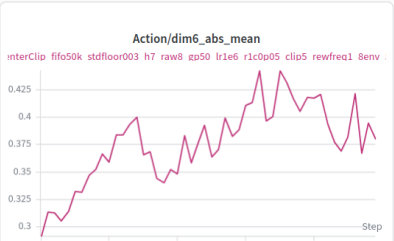
Charts 2

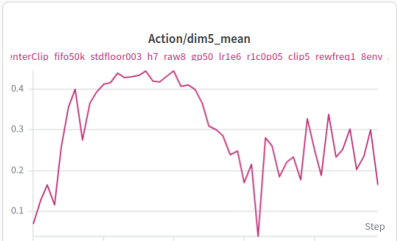
Action 30 13-18 of 30

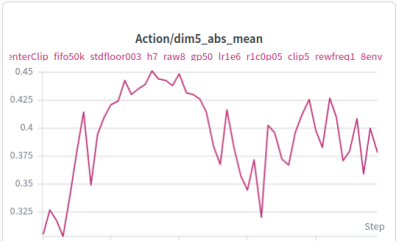








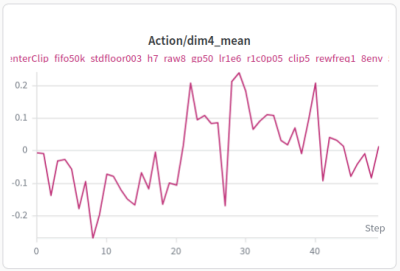


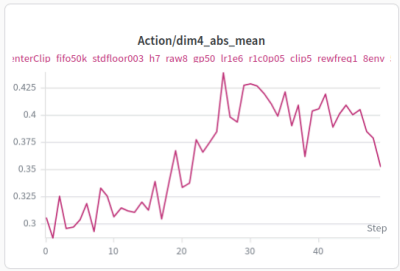


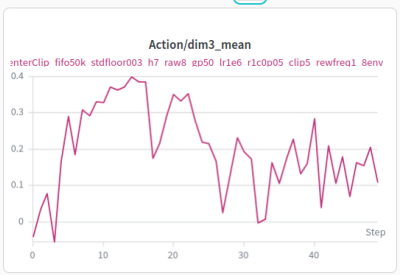
Charts 2

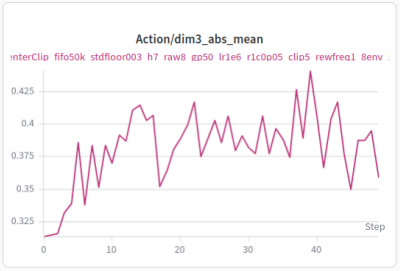
Action 30

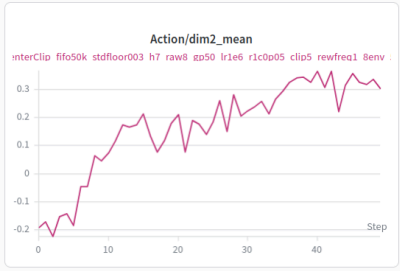
19-24 of 30

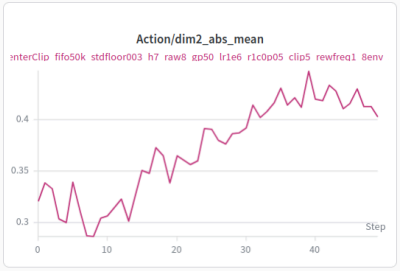










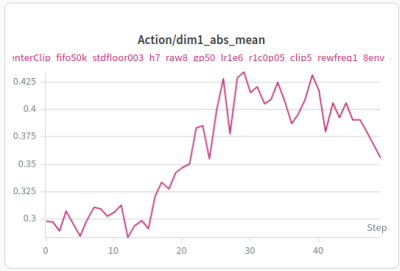


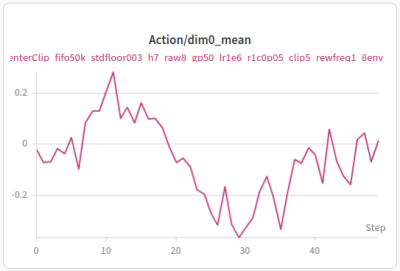
Charts 2

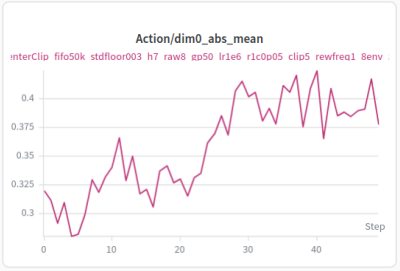
Action 30

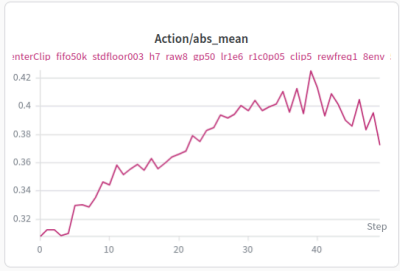
25-30 of 30

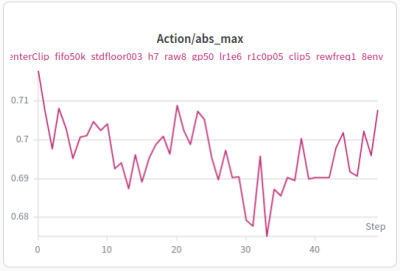












Charts 2



深入的根据算法训练逻辑分析这次的现象，证明了什么，

5:21 AM

Worked for 3m 7s

结论

这次 S17 证明了一件很关键的事：之前 S16 的“reward center 没救”不完全成立，因为当我们把 reward calibration 真正贯穿 value target 和 MPPI rollout 后，reward/value 的灾难程度明显下降了。但它也证明了更

深的问题：现在主要瓶颈已经不是 replay FIFO、不是 fulltask、不是 action 太小、也不是单纯 reward 没学到，而是 learned reward 已经能区分 demo/replay，但 value/dynamics/planner 没能把这个区分转化成可控的动作选择。

换句话说，reward 现在会说“demo 好、当前 learner 坏”，但 MPPI/value 不知道“往哪里动会变好”。

这次证明了什么

1. reward calibration 是有效的，但只解决了爆炸，不解决学习闭环

S16 里 `Value/mean_loss` 到 200+，这次 S17 到约 20 多，说明 `reward_center_for_planning + reward_clip_for_planning=5` 确实把 target 尺度压住了。

而且 `MPPI/Min Reward` 被卡在 `-5`，这说明 clip 确实生效。`Reward/mean_demo_reward` 上升到约 3，`Reward/mean_gen_reward` 下降到约 -3.8，也说明 discriminator/reward 不是完全坏的，它学会了 demo 和 replay 的差异。

所以这次不是“reward 完全没学到”。

2. 但是 value 学到的是“所有动作都差”，不是“哪个动作更好”

最关键的是 ValueDiag：

`q_zero_mean`、`q_rand_mean`、`q_exec_mean` 都一起掉到约 `-6 ~ -7`，而 `q_exec_minus_zero`、`q_rand_minus_zero` 只有 `0.005 ~ 0.01` 这种量级。

这说明 critic 的输出主要由状态决定，几乎不由动作决定。也就是：

```
Q(s, a_exec) ≈ Q(s, a_zero) ≈ Q(s, a_rand)
```

这对 MPPI 是致命的。MPPI 需要 reward/value 对不同 action sequence 排序；如果所有 action 的 Q 差不多，它只能在噪声里选“看起来没那么坏”的轨迹。

这也解释了为什么 `MPPI/Mean Policy Influence` 仍然接近 0。不是 policy 一定没输出，而是 policy proposal 在 MPPI 打分里没有优势，所以不会被 elite 选中。

3. Average Reward 下降 + Average Value 下降，说明 planner 进入了“负回报盆地”

这次 `MPPI/Average Reward` 从接近 0 掉到约 `-3`，`MPPI/Average Value` 掉到约 `-7`。比 S16 的 `-10` 好，但方向仍然错。

代码上在 planner.py (line 160) 里，MPPI rollout 先算 calibrated reward，但最后一步会被 terminal value 覆盖。也就是说，即使某些短期 reward sample 有正值，terminal value 如果整体悲观，最后 return 还是被拖下去。

这次的现象像是：reward 已经在惩罚 learner 偏离 demo，但 value 没有学出“动作可以改善未来”的结构，于是 terminal value 变成一个大范围悲观项，MPPI 被它压住。

4. action 不是太小，视频里慢/没进展不是因为 raw action limit 太低

Action log 反而显示 `Action/raw_norm` 从约 5.4 上升到 6.5，`Action/env_norm` 到 1.1 左右，动作幅度并不小。很多 raw action dim 还在变大。

所以这次跟之前“动作几乎不可见”的问题不同：这里不是动作太小，而是动作没有产生任务进展。Episode reward 也支持这一点：

`success_reward=0`，`progress_context=0`，`dense_success_reward` 基本平，`ee_asset_distance` 没明显改善，`time_out` 接近 1。

也就是 policy/MPPI 在动，但没有朝任务目标有效动。

5. dynamics loss 下降，但 action sensitivity 下降，这是很重要的坏信号

`Dyn/mean_loss` 下降到约 0.3，看起来 dynamics 在拟合。但 `Dyn/action_sensitivity_rand_vs_zero` 和 `exec_vs_zero` 都下降，说明模型越来越不敏感于动作。

这意味着 learned dynamics 可能学到了“latent state 的平均漂移/环境惯性”，但没有学到“动作如何改变关键状态”。对 MPPI 来说，这比普通 loss 高更危险，因为 planner 依赖 dynamics 展开动作后果。

如果 dynamics 对 action 不敏感，MPPI 看到的未来就会接近：

```
不同 action sequence -> 差不多的 latent rollout -> 差不多的 Q/reward
```

这正好和 `Q_exec ≈ Q_zero ≈ Q_rand` 对上了。

根本问题

当前最像的问题不是一个小超参，而是这个闭环断了：

```
reward 可以区分 demo vs learner
但 dynamics/value 对 action 的可控差异太弱
所以 MPPI 排不出有效 action
所以 policy proposal 不被选中
所以 replay 继续是坏数据
reward 更确信 learner 坏
value 更悲观
最后 MPPI 只是在负回报区域里采样
```

这也是为什么 fulltask 减半也学不会。任务长短不是主因，主因是“动作到未来 reward/value 的 credit assignment”没有建立起来。

下一步我建议不要再继续调 gp/r1 这种小参数了

下一步应该做一个更直接的诊断实验：把 terminal value 对 MPPI 的影响先关掉或大幅减小，只让 MPPI 用 clipped centered reward sum 选动作。

如果这样 `MPPI/Mean Policy Influence` 仍接近 0、Episode 仍没进展，那主因就是 dynamics/reward rollout 不能提供可控方向。

如果这样 Average Reward/episode progress 明显好转，那主因就是 value terminal 把 planner 压死了。

具体下一组我建议：

```
replay FIFO 保持
std_floor=0.03
reward_update_every=1
h=5 或 h=7
gp=50
r1=0.05
reward_center_for_planning=True
reward_clip_for_planning=5
terminal_value_scale=0
value 仍然训练，但 MPPI 暂时不用 terminal value
```

这不是最终算法形态，而是为了验证“是不是 value terminal 在 planner 里造成主导性坏信号”。从这次 log 看，这是目前最值得直接验证的点。

好，帮我跑把

5:31 AM

Worked for 27m 49s

S18 跑完了。

改动：

- 加了 `planner_cfg.terminal_value_scale`，默认 `1.0`，旧实验不变。
- S18 里设成 `0.0`，也就是 MPPI 只用 centered/clipped reward rollout，不再用 terminal value。
- 脚本：scripts/run_s18_no_terminal_value_h7_gp50_50.sh

Run：

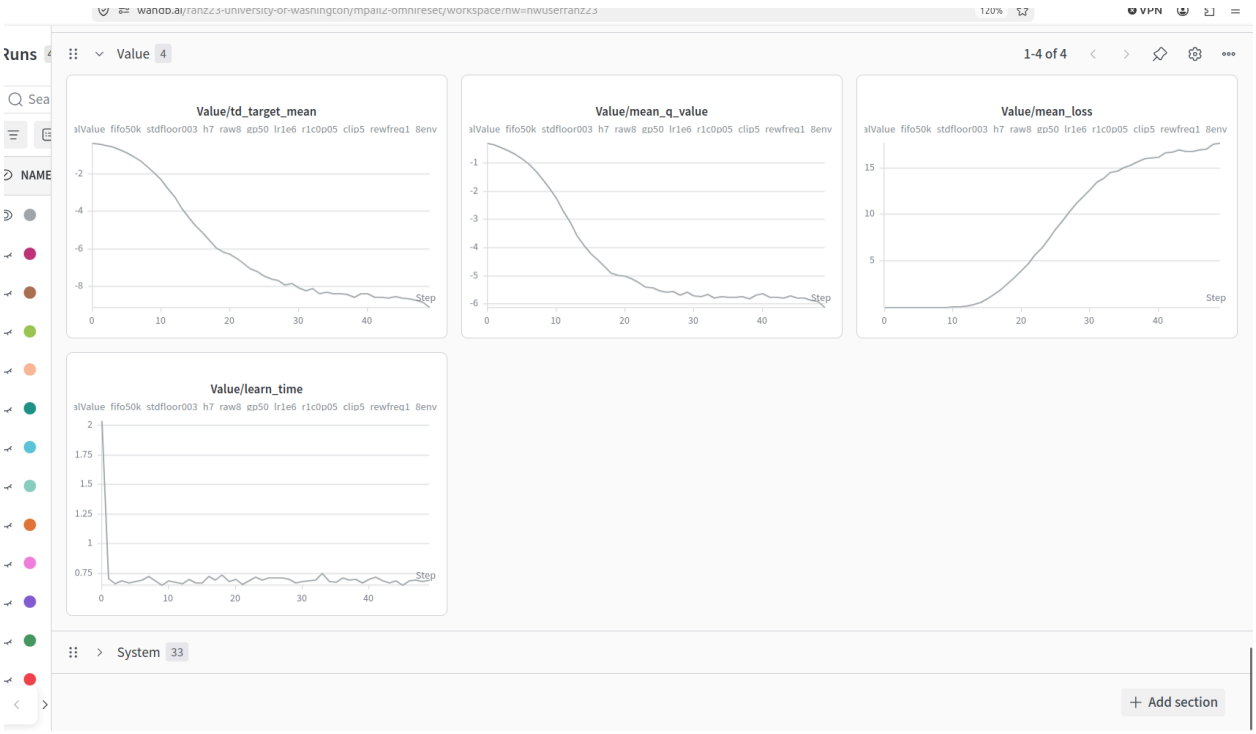
- W&B: `exsvspf2`
- run id: `obs22_S18_noTerminalValue_fifo50k_stdfloor003_h7_raw8_gp50_lr1e6_r1c0p05_clip5_rewfreq1_8env_50iter_20260525_053620`
- log: train.log

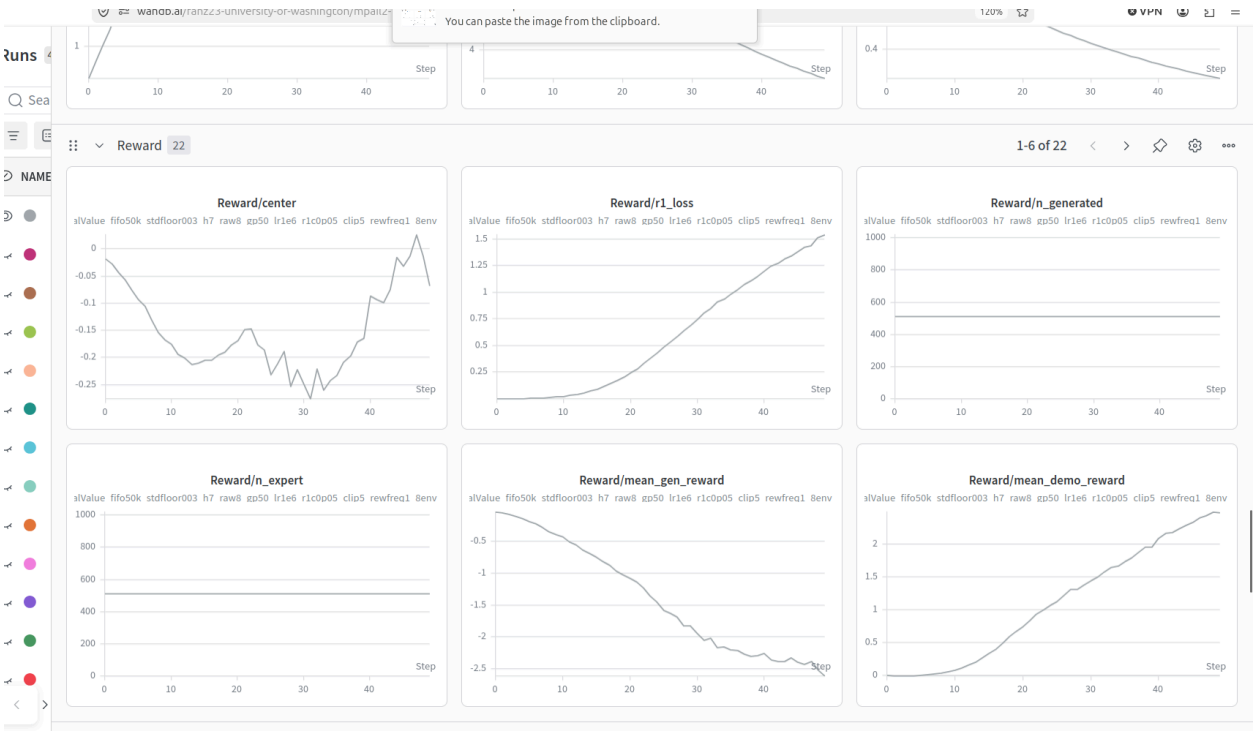
终端粗看结果：

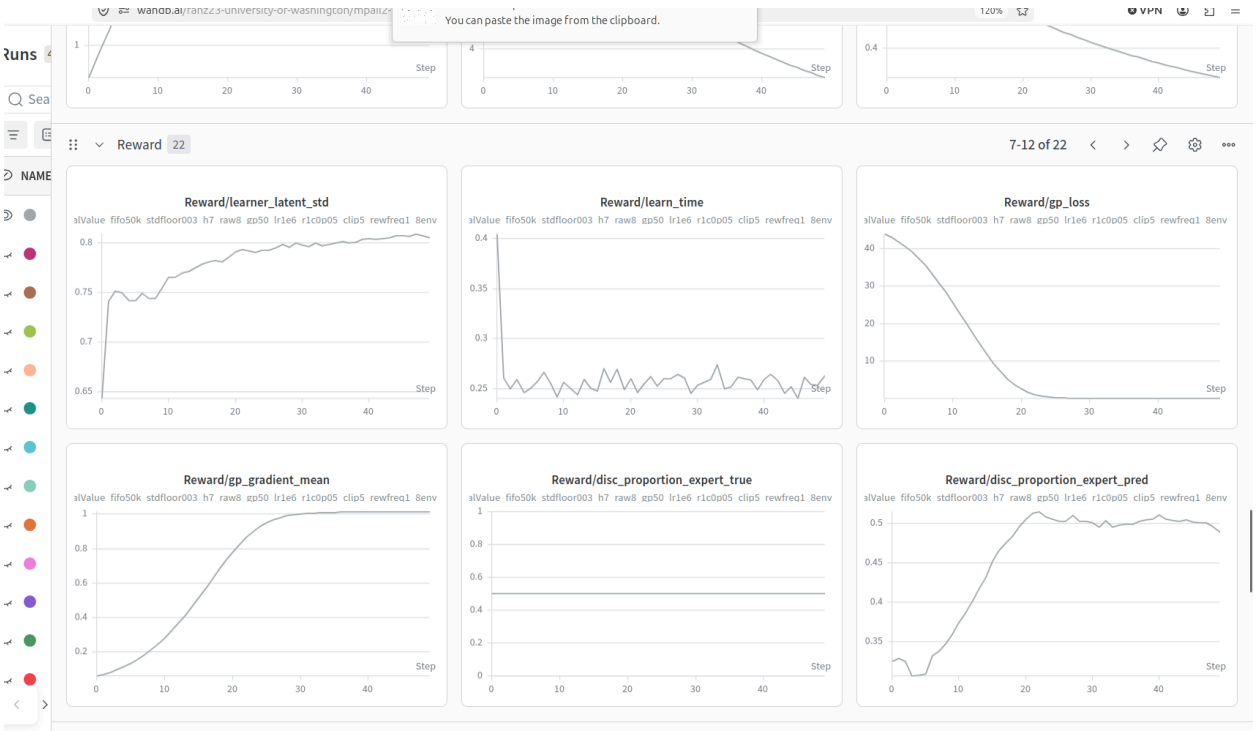
- `succ=0` 全程没有成功。
- 但后半程 `rew` 从负数变成正数，最后几步在 `0.23~0.47`，比 S17 的 MPPI reward 被 value 拖死明显好。
- `disc_demo` 到 `2.476`，`disc_gen` 到 `-2.613`，reward 分离正常。

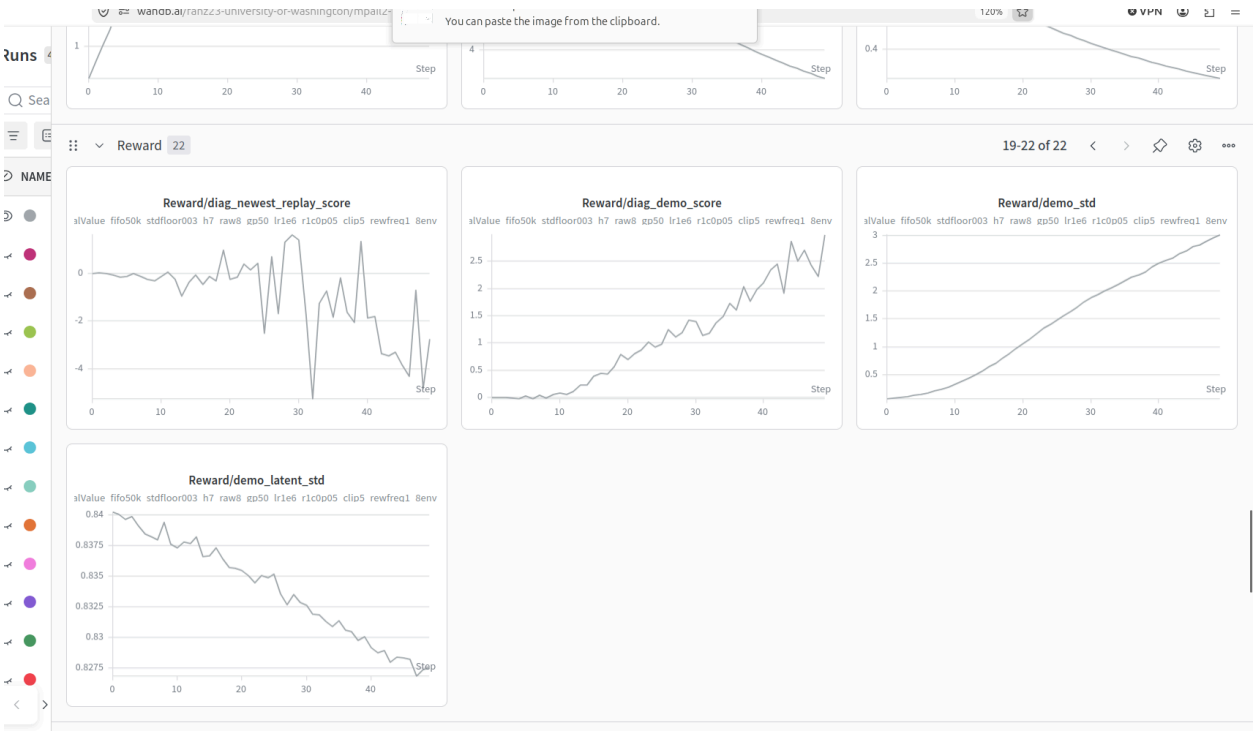
- `dyn_loss` 到 `0.3984`。
- episode return 仍然多数只是 `1.x` 或异常负数，没有任务进展突破。

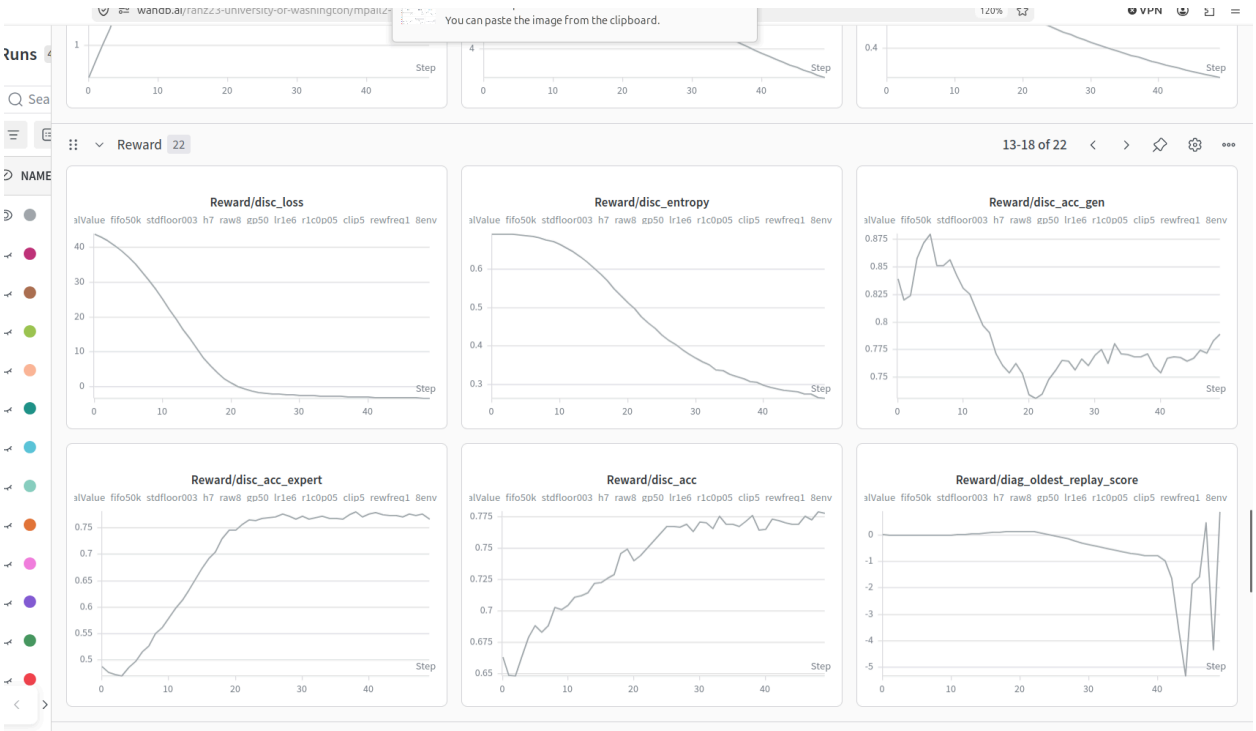
这说明关掉 terminal value 确实缓解了 planner 被 value 悲观项压住的问题，但没有解决根因：reward 能给分，MPPI 也不再完全被 value 拖死，可动作仍没转化成真实任务 progress。下一步应该重点看 S18 的 `MPPI/Mean Policy Influence`、`Average Reward`、`Episode_Reward/progress_context` 和动作/异常曲线。

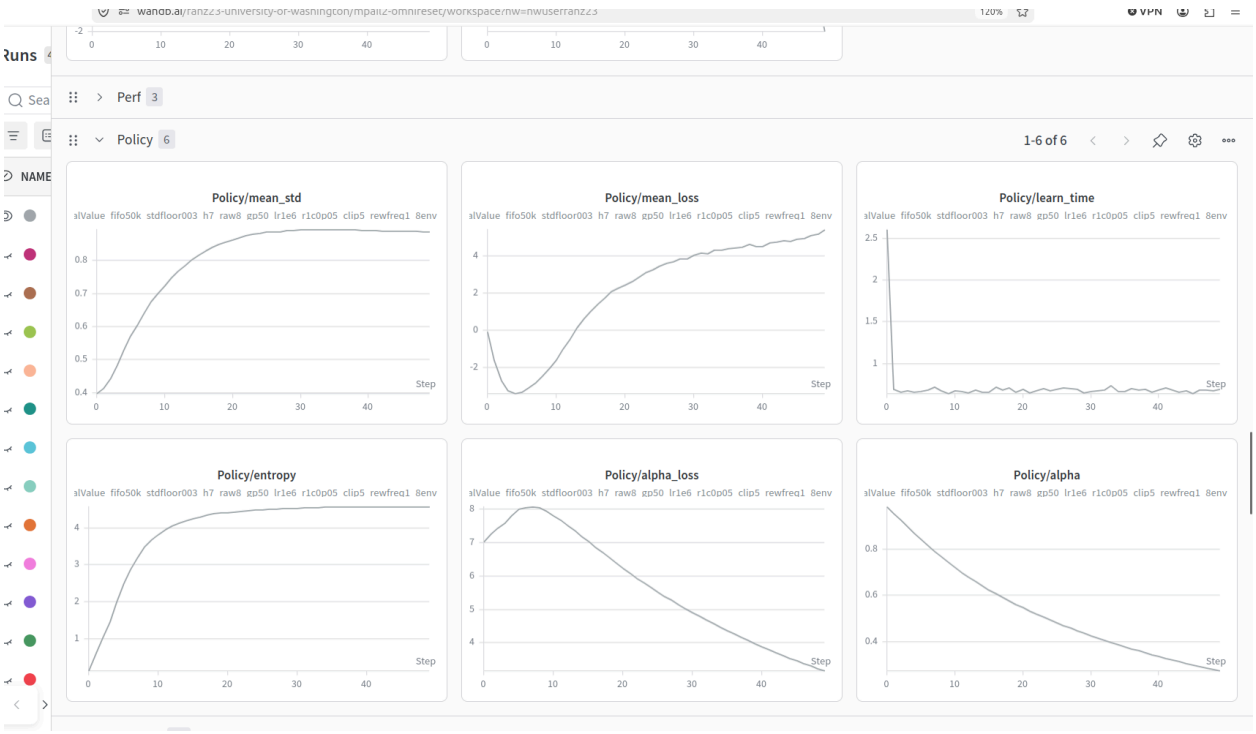



You can paste the image from the clipboard.
Runs
Reward 22
1-6 of 22
Reward/center
Reward/r1_loss
Reward/n_generated
Reward/n_expert
Reward/mean_gen_reward
Reward/mean_demo_reward
Step


You can paste the image from the clipboard.
Runs
Reward 22
7-12 of 22
Reward/learner_latent_std
Reward/learn_time
Reward/gp_loss
Reward/gp_gradient_mean
Reward/disc_proportion_expert_true
Reward/disc_proportion_expert_pred
Step


You can paste the image from the clipboard.
Runs
Reward 22
19-22 of 22
Reward/diag_newest_replay_score
Reward/diag_demo_score
Reward/demo_std
Reward/demo_latent_std
Step


You can paste the image from the clipboard.
Runs
Reward 22
13-18 of 22
Reward/disc_loss
Reward/disc_entropy
Reward/disc_acc_gen
Reward/disc_acc_expert
Reward/disc_acc
Reward/diag_oldest_replay_score
Step

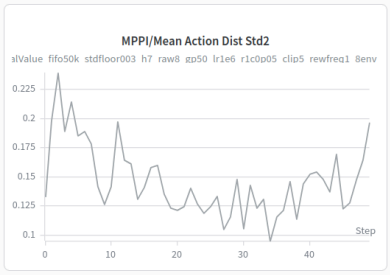


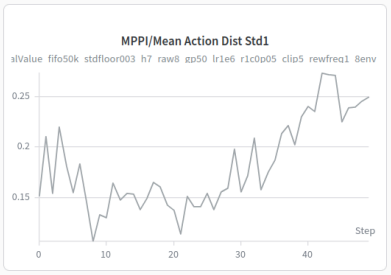
wandb.ai/fanz23-university-of-washington/mpail2-omnireset/workspace?nw=nwuserfanz23

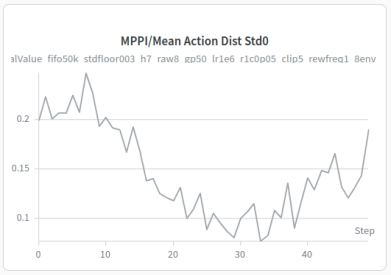


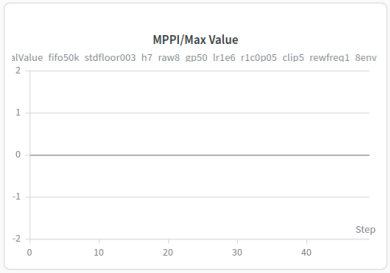
Metrics 5

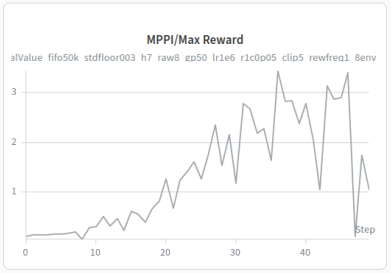
MPPI 20 13-18 of 20

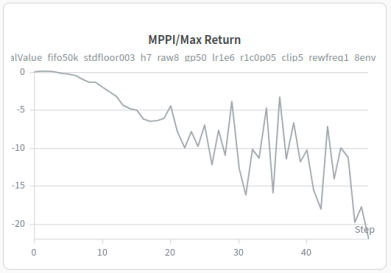






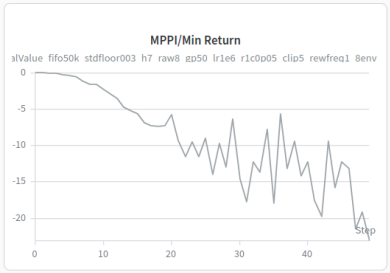


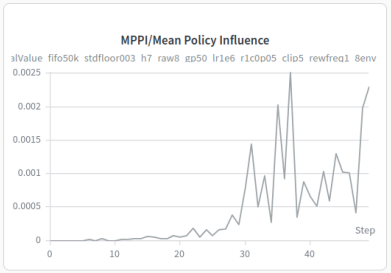


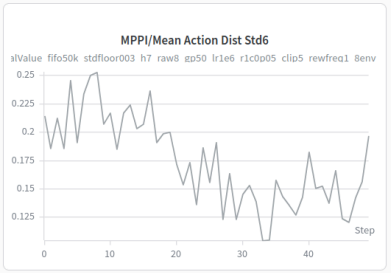


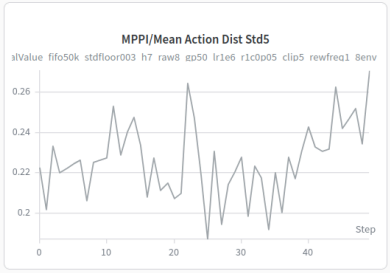
Metrics 5

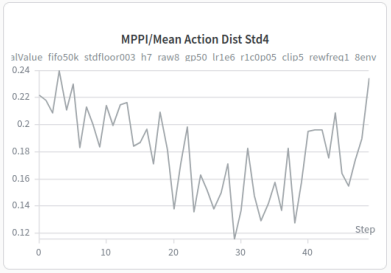
MPPI 20 7-12 of 20

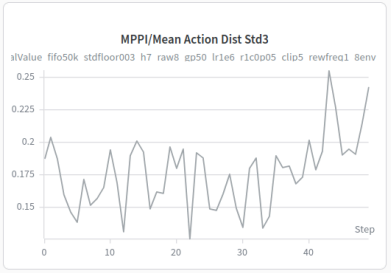






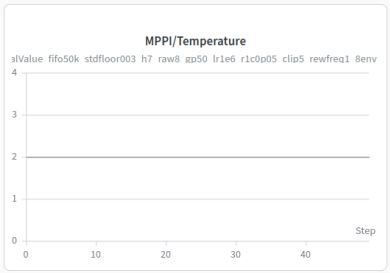


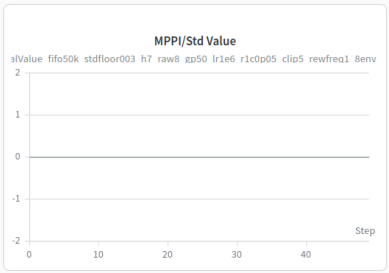


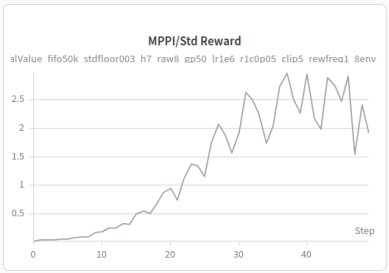


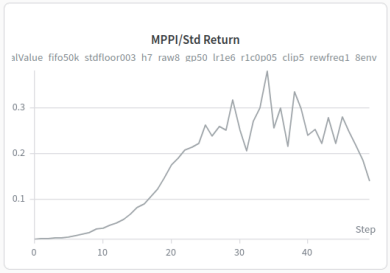
Metrics 5

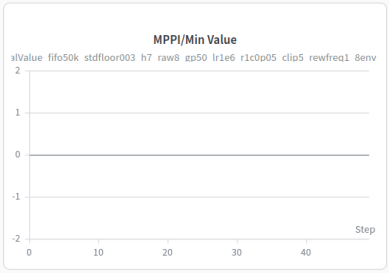
MPPI 20 1-6 of 20

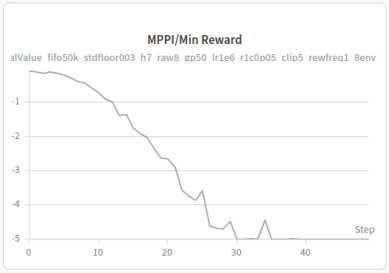




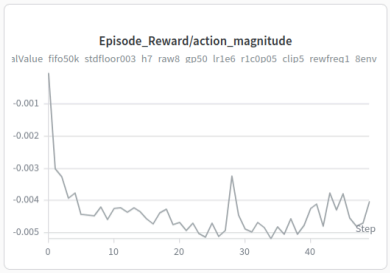


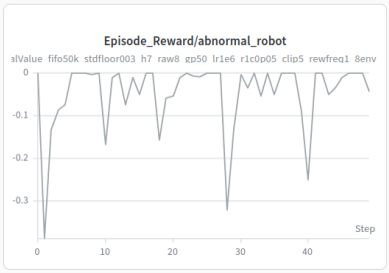




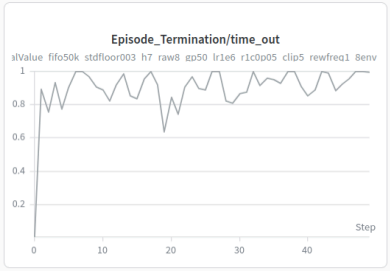


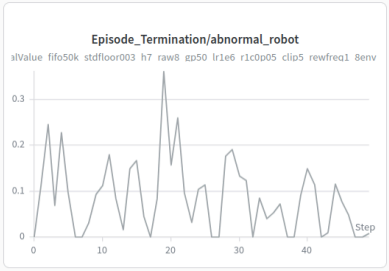
Episode_Reward 8 7-8 of 8





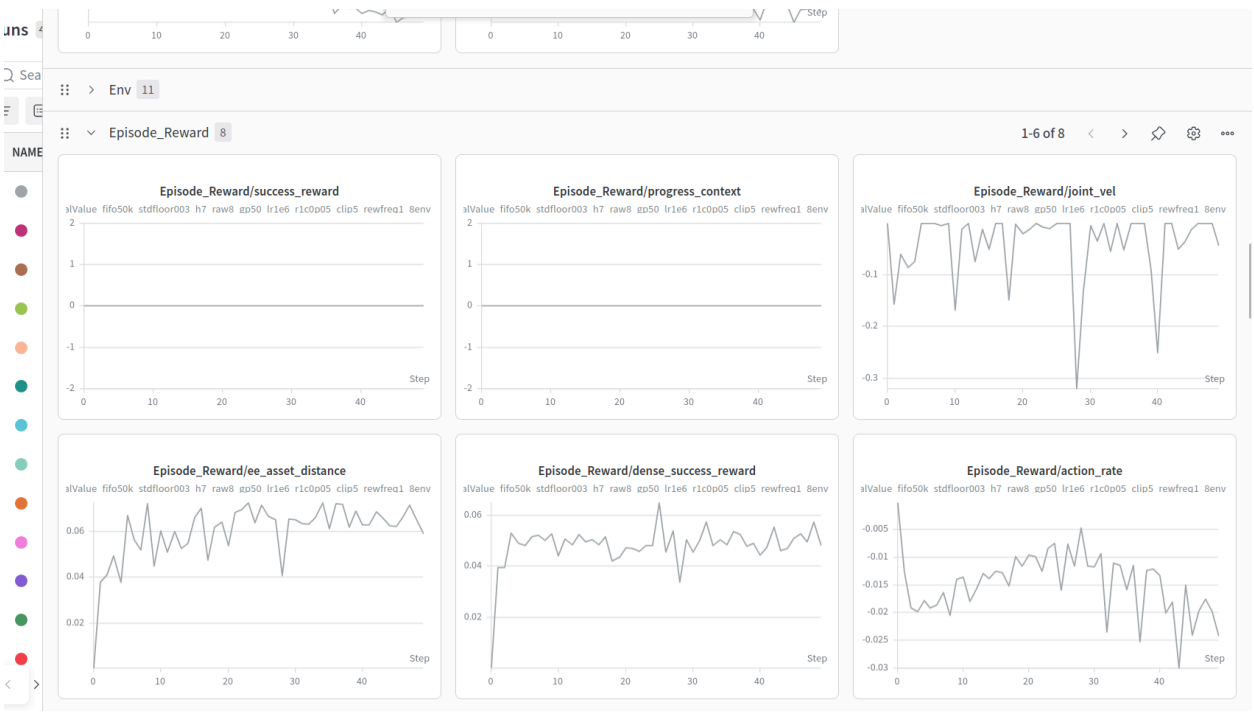
Episode_Termination 2



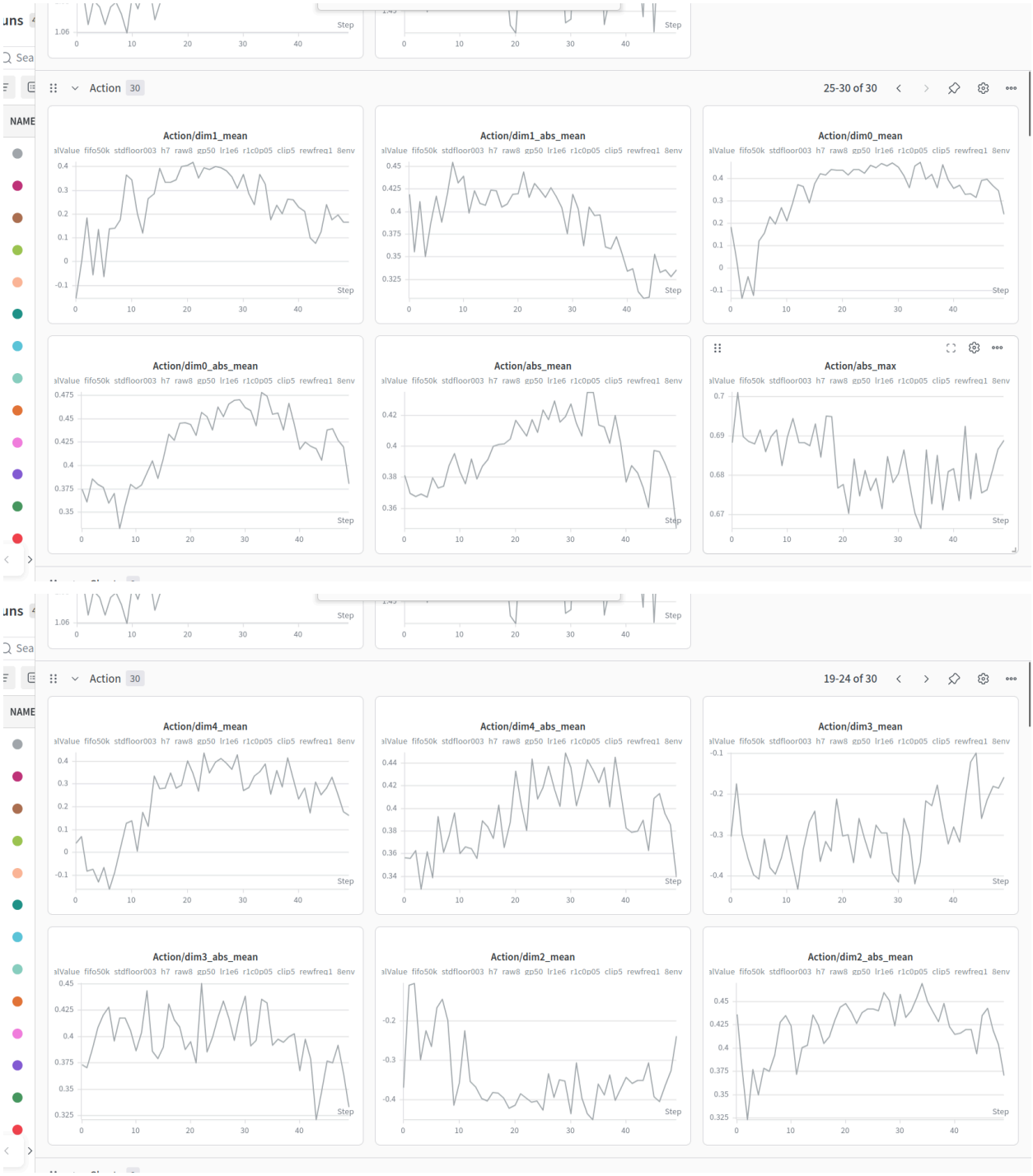


Metrics 5

MPPI 20 1-6 of 20

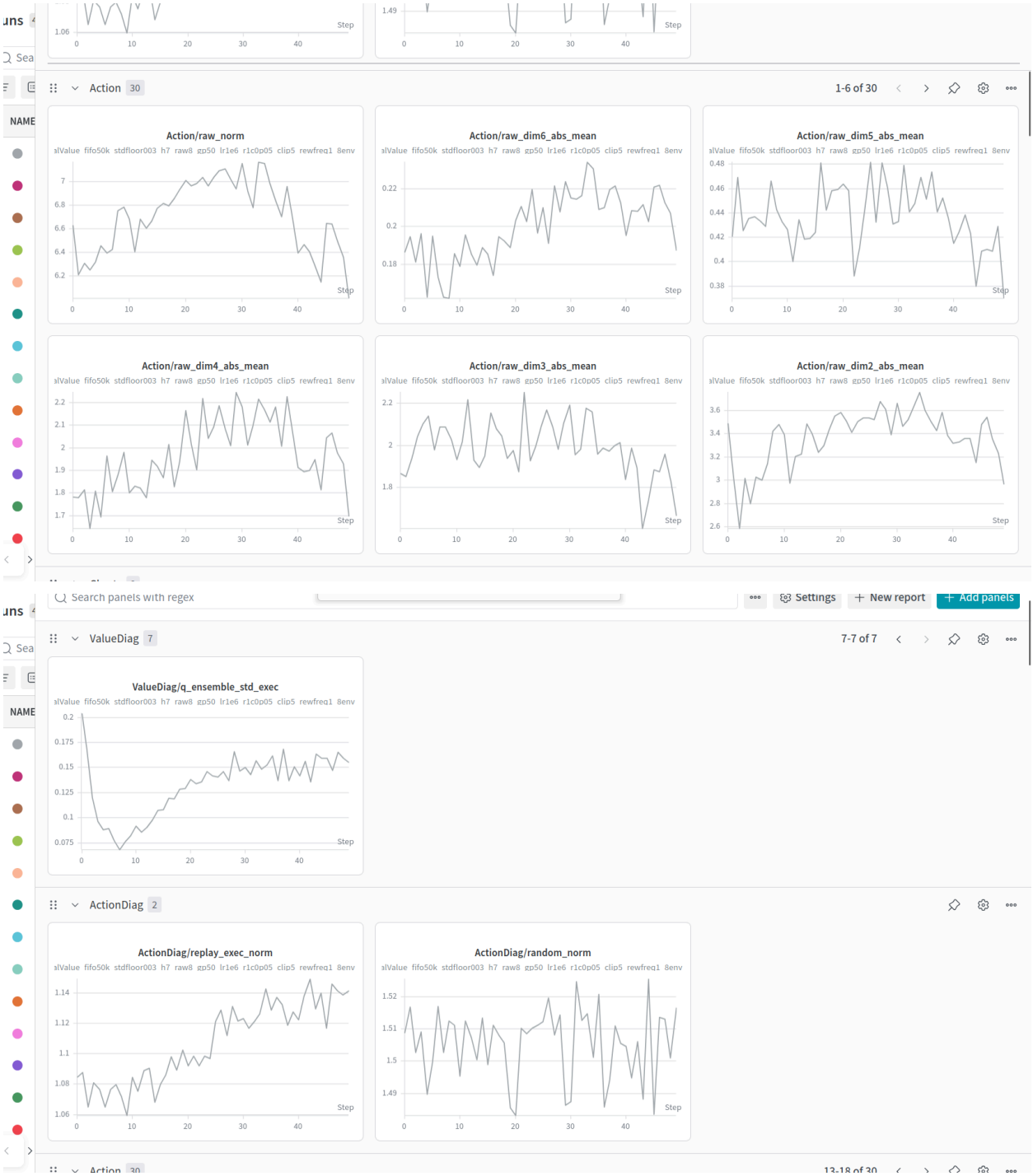

Env 11
Episode_Reward 8
1-6 of 8
Episode_Reward/success_reward
Episode_Reward/progress_context
Episode_Reward/joint_vel
Episode_Reward/ee_asset_distance
Episode_Reward/dense_success_reward
Episode_Reward/action_rate
Step

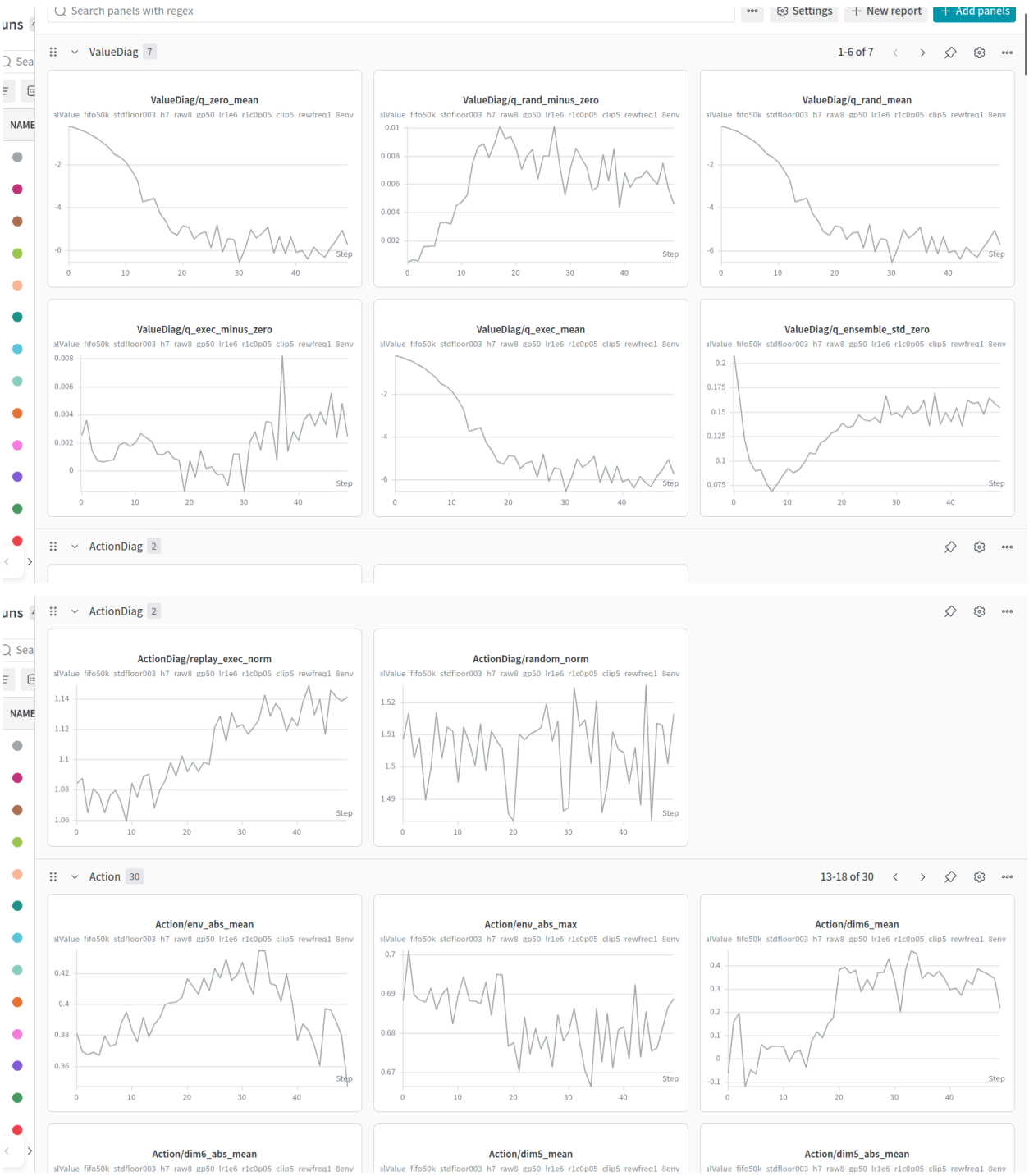

Data 2
Dyn 5
1-5 of 5
Dyn/recon_loss
Dyn/mean_loss
Dyn/learn_time
Dyn/action_sensitivity_rand_vs_zero
Dyn/action_sensitivity_exec_vs_zero
Env 11
Step


Action 30
25-30 of 30
Action/dim1_mean
Action/dim1_abs_mean
Action/dim0_mean
Action/dim0_abs_mean
Action/abs_mean
Action/abs_max
Action 30
19-24 of 30
Action/dim4_mean
Action/dim4_abs_mean
Action/dim3_mean
Action/dim3_abs_mean
Action/dim2_mean
Action/dim2_abs_mean
Step


Action 30
13-18 of 30
Action/env_abs_mean
Action/env_abs_max
Action/dim6_mean
Action/dim6_abs_mean
Action/dim5_mean
Action/dim5_abs_mean
Step


Action 30
7-12 of 30
Action/raw_dim1_abs_mean
Action/raw_dim0_abs_mean
Action/raw_abs_mean
Action/raw_abs_max
Action/norm
Action/env_norm
Step


Action 30
1-6 of 30
Action/raw_norm
Action/raw_dim6_abs_mean
Action/raw_dim5_abs_mean
Action/raw_dim4_abs_mean
Action/raw_dim3_abs_mean
Action/raw_dim2_abs_mean
Search panels with regex
Settings
New report
Add panels
ValueDiag 7
7-7 of 7
ValueDiag/q_ensemble_std_exec
ActionDiag 2
ActionDiag/replay_exec_norm
ActionDiag/random_norm
Step



深入的根据算法训练逻辑分析这次的现象，证明了什么，

6:04 AM

核心结论

这次 S18 证明得更清楚了：terminal value 确实是之前压坏 MPPI 的一部分原因，但不是根因。把 MPPI 的 terminal value 关掉以后，`MPPI/Average Value` 直接变成 0，说明开关生效；`MPPI/Max Reward` 明显能找到

正 reward 样本，说明 reward rollout 不是完全不可用。但最终仍然 success=0、progress_context=0、Average Reward 继续变负，说明真正断的是：

reward 能评价 demo/replay 差异
但 learned dynamics + MPPI 仍找不到能让真实环境进步的动作序列

也就是：value 不是唯一坏点；模型里的“动作 -> 未来状态 -> 更高 reward”的路径仍然没有建立起来。

这次各类 log 说明什么

Value/td_target_mean 和 Value/mean_q_value 仍然一路变负，Value/mean_loss 到 17 左右。这里要注意：S18 只是 MPPI 不用 terminal value，value learner 本身还在训练。value 仍然学到“当前 replay 分布未来很差”。这说明 critic 悲观不是单纯 planner terminal value 造成的，而是来自 value target 里的 reward rollout 分布本身。

但这次 MPPI 里 Average Value=0、Max Value=0、Min Value=0，证明 planner 评分已经不再被 value 直接影响。也就是说，如果 S18 仍然不动，问题就不能再甩给 “terminal value 把 planner 压死”。

Reward/mean_demo_reward 上升到约 2.5，Reward/mean_gen_reward 下降到约 -2.5，disc_acc 到 0.77 左右，disc_proportion_expert_pred 回到 0.5 附近。这是比较健康的 discriminator 形态：它不是像之前那样极端饱和到全判一边，而是能区分 demo 与 learner。

但是 Reward/diag_newest_replay_score 后半段非常抖，并且经常是负的；diag_demo_score 稳定上升。这个组合说明 reward 学到的“专家区域”越来越明确，但 learner replay 没有稳定靠近它。也就是说 reward 给出了目标，但 rollout 没走向目标。

MPPI/Average Reward 从接近 0 掉到约 -3，MPPI/Max Reward 却能到正的 2~3。这非常关键：MPPI 的采样分布里存在少数高 reward 候选，但整体候选大多很差。按 MPPI 逻辑，如果高 reward 候选代表真实可执行进步，它应该逐渐把 action distribution 拉过去。但 Episode/progress_context=0、success_reward=0，说明这些高 reward 候选大概率是 learned model/reward 的“局部假象”，不是环境里真正有效的动作。

MPPI/Mean Policy Influence 仍然只有约 0.002，非常低。这个比 S17 稍微好一点，但本质还是接近 0。说明 learned policy proposals 仍然没有被 MPPI elite 选中。也就是 policy 学不到能被 planner 认可的动作，planner 主要还是靠随机/CEM 噪声探索。

Dyn/mean_loss 下降，看起来 dynamics 在拟合；但 Dyn/action_sensitivity_rand_vs_zero 和 exec_vs_zero 持续下降。这是最像根因的信号。模型越训越“不相信动作会强烈改变 latent”。如果 dynamics 对 action 不敏感，那么 MPPI 展开不同动作序列时，未来 latent 差别不够任务相关，reward 排序自然就不可靠。

Action/raw_norm、env_norm 前中期变大，后期又回落；动作不是太小。视频如果仍然没进展，不是因为 action limit 太小，而是因为 planner 选出来的动作没有语义方向。动作幅度存在，但没有 task-relevant effect。

这次证明了什么

1. value terminal 是放大器，不是根因。
   S17 里 terminal value 把 return 压得更惨；S18 关掉以后 MPPI 不再直接吃 value，但任务仍不进展。
2. reward discriminator 已经不是最烂的部分。
   demo/gen 分离是正常的，gp/r1 这组也比之前健康。继续只调 gp/r1 收益很有限。
3. 核心瓶颈转移到 dynamics/action credit。
   现在最像的问题是 learned dynamics 在 latent 空间没有保留足够的“动作可控性”，导致 MPPI 看到的高 reward rollout 不对应真实环境进步。
4. policy 没接上 planner。
   `Mean Policy Influence≈0` 说明算法里的 policy improvement 环节几乎没起作用。policy 不是在模仿 expert，也不是在帮 MPPI，只是在被一个弱/错的 model-return 信号训练。

下一步方向

我建议下一步不要再动 reward_update_every、gp、r1 这类了。应该做一个更直接的验证：把 horizon 降到 3 或 5，并增加 dynamics/action-sensitive 诊断或约束 planner 更短视地追 immediate reward。

最推荐下一跑：

```
S19:
terminal_value_scale=0
h=3 或 h=5
loss_horizon=3 或 5
reward_center_for_planning=True
reward_clip_for_planning=5
gp=50
r1=0.05
raw_action_limit 不变
```

判断标准很简单：

如果 h 变短后 `MPPI/Average Reward` 变好、`progress_context` 有动静，说明 long rollout dynamics compounding error 是主因。

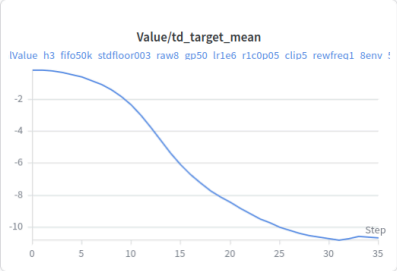
如果 h 变短后仍然 `progress=0`、`policy influence≈0`，那问题更底层：当前 latent dynamics/reward 表示对动作控制不够敏感，MPPI 在这个 model 里根本找不到真实可执行路径。

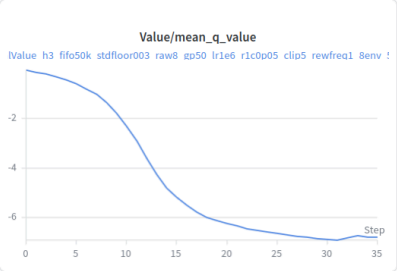
6:05 AM
ok 帮我跑下一个实验吧
6:10 AM

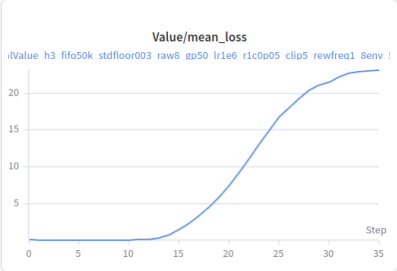
我直接开 S19：沿用 S18 的 no-terminal-value，但把 MPPI/value 的 horizon 从 7 降到 3，专门验证是不是长 rollout 的 dynamics compounding error 在把 planner 带偏。

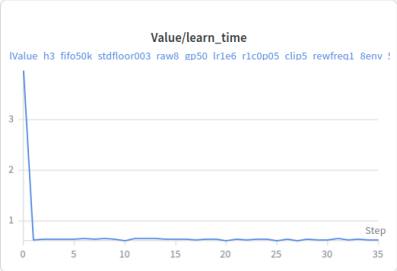
Value 4

1-4 of 4







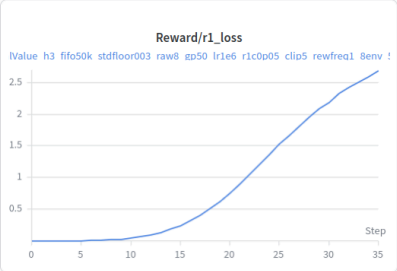


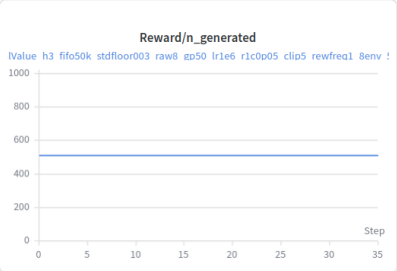
System 33

+ Add section

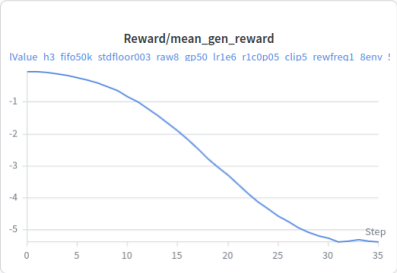
Reward 22

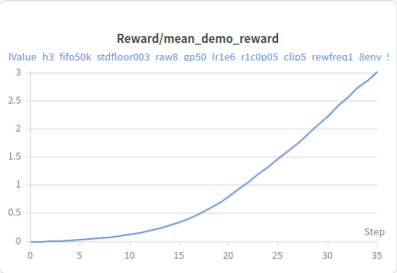
1-6 of 22

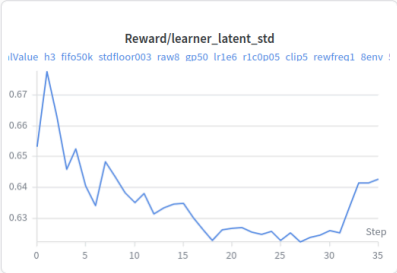










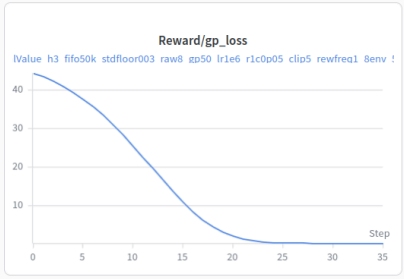


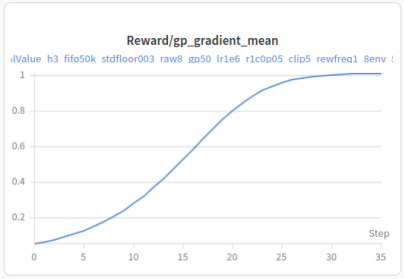
Storage 6

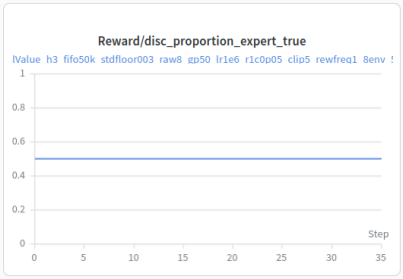
Training 2

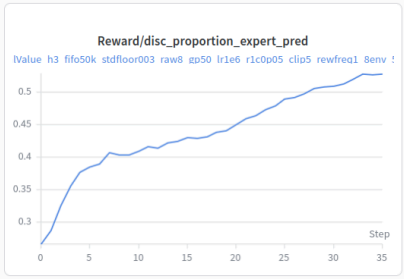
Reward 22 7-12 of 22









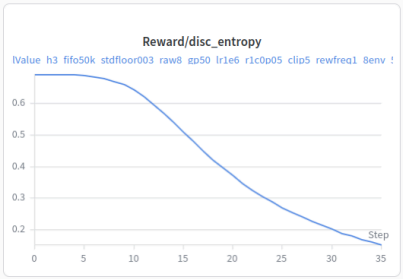




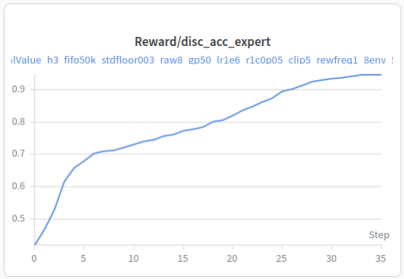
Storage 6

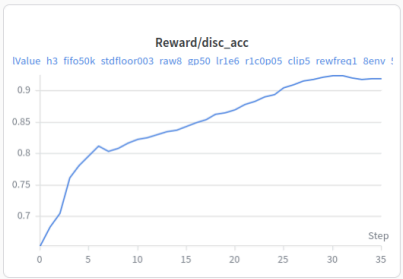
Training 2

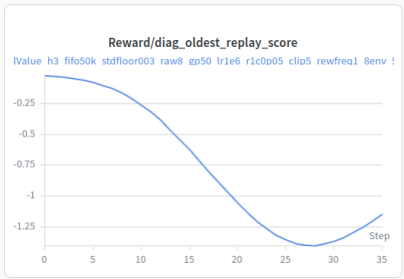
Reward 22 13-18 of 22

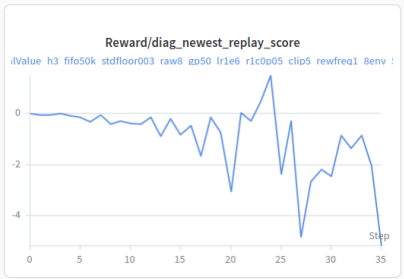








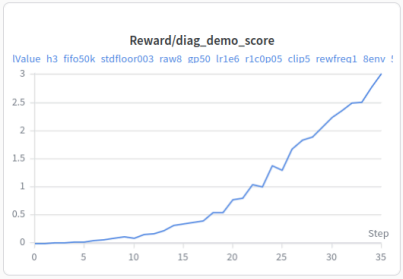


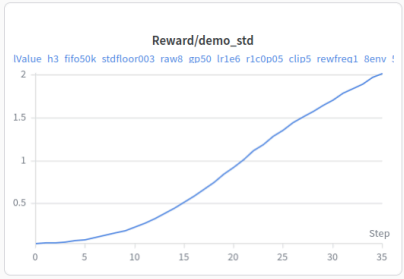


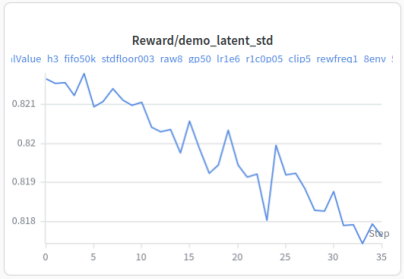
Storage 6

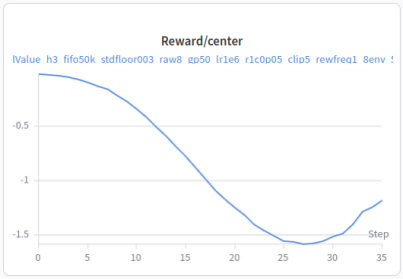
Training 2

Reward 22 19-22 of 22





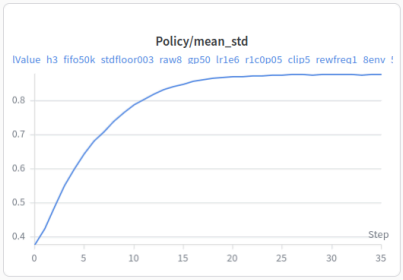


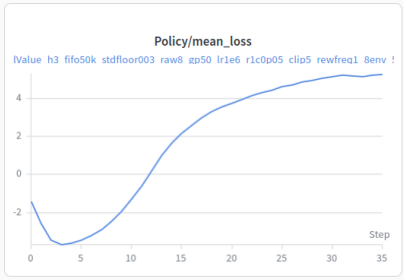


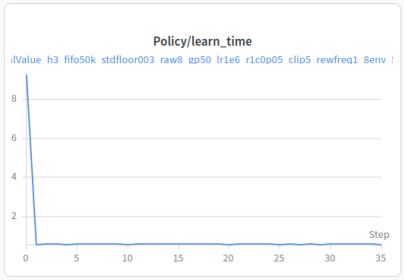
Storage 6

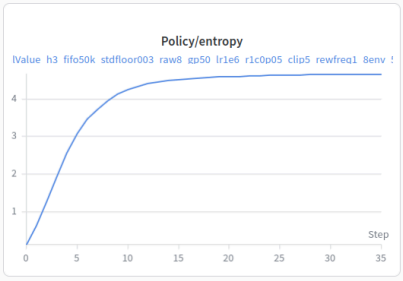
Training 2

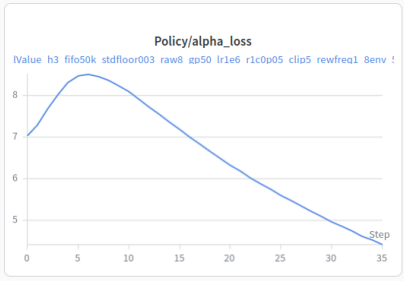
Policy 6 1-6 of 6

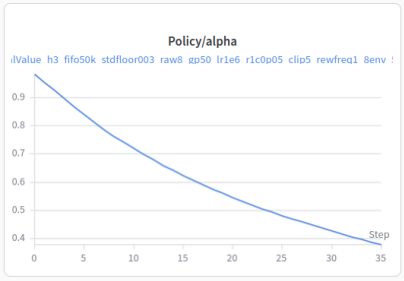








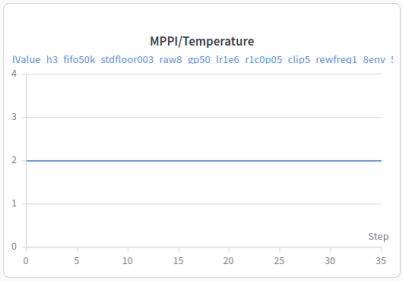


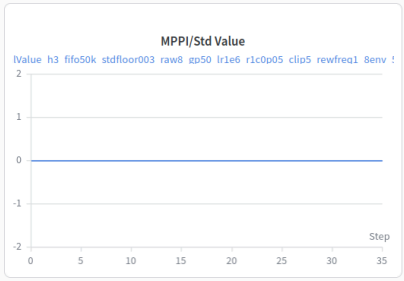


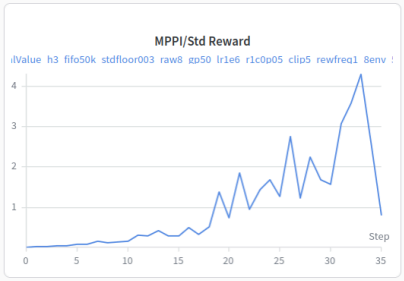
Reward 22 19-22 of 22

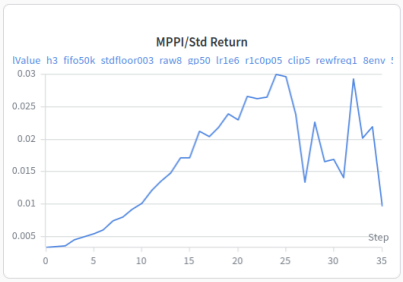
Metrics 5

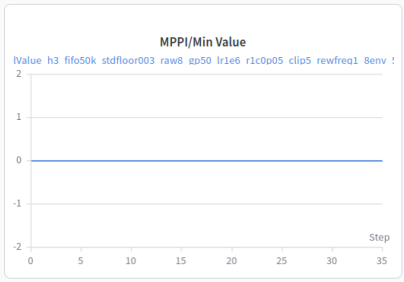
MPPI 20 1-6 of 20

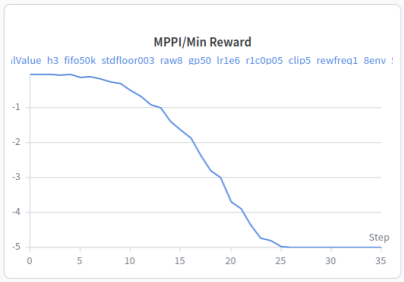






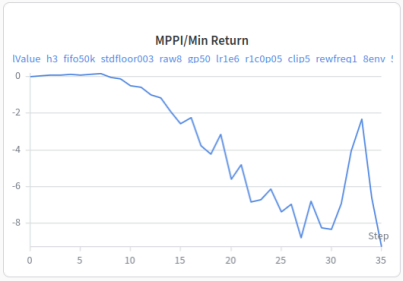


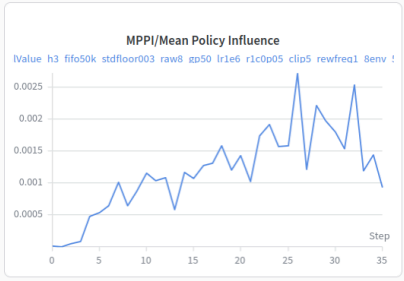


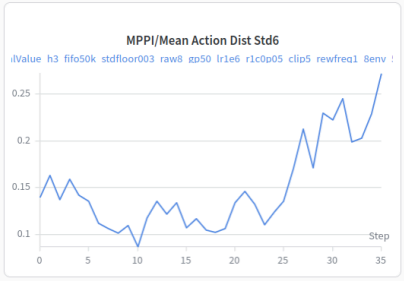


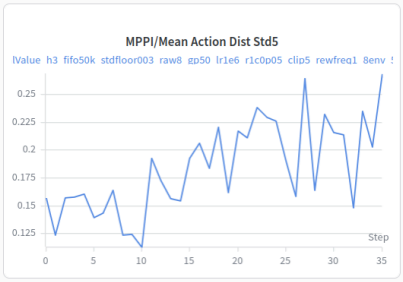
Metrics 5

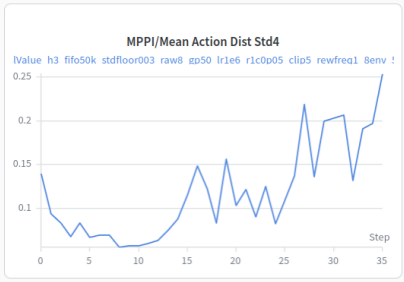
MPPI 20 7-12 of 20

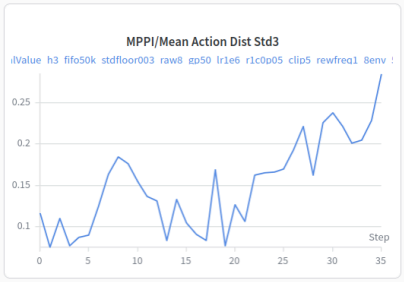






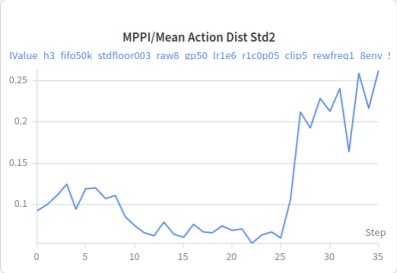


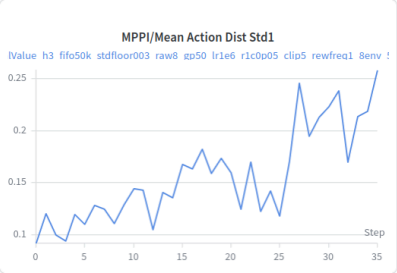


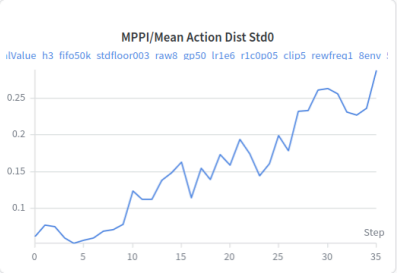


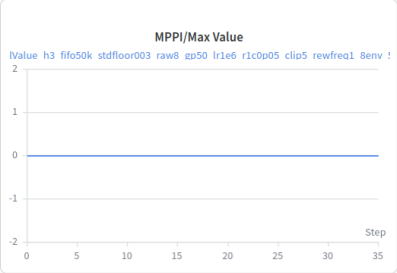
Metrics 5

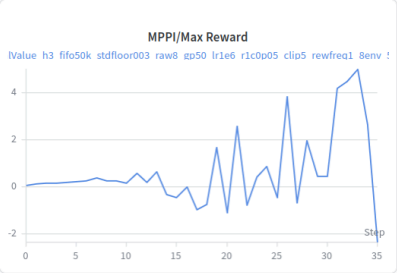
MPPI 20 13-18 of 20

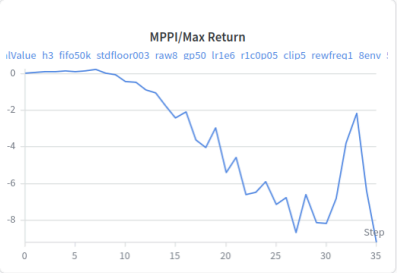


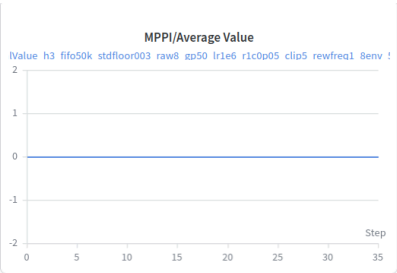


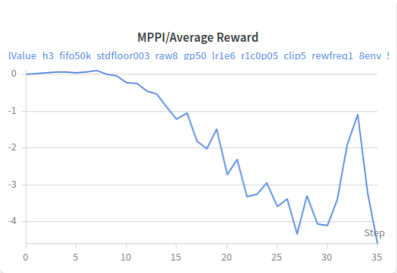






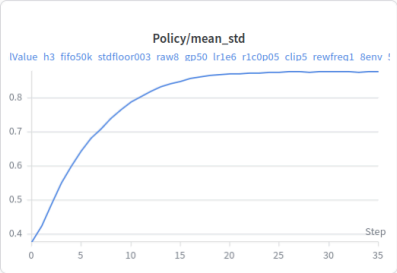


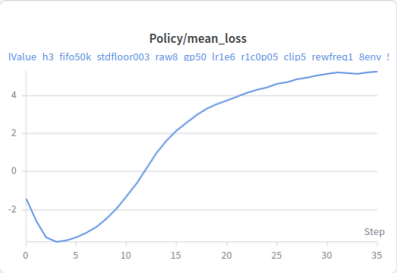


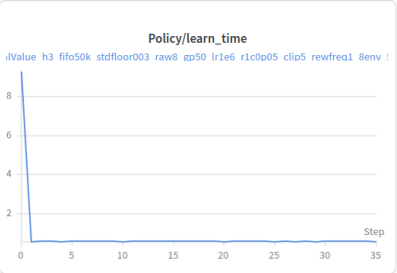


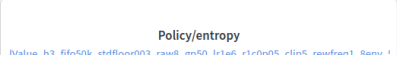
Perf 3

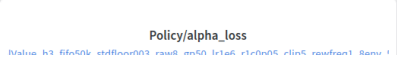
Policy 6 1-6 of 6









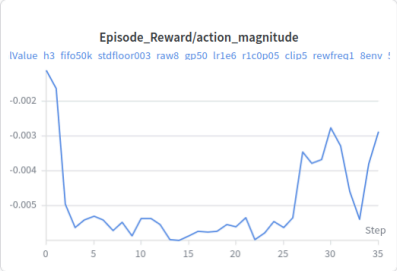


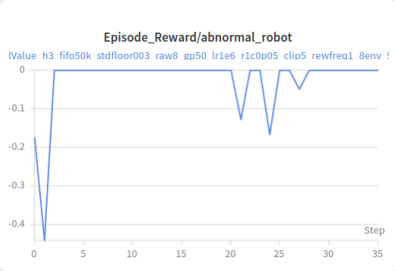


Env 11

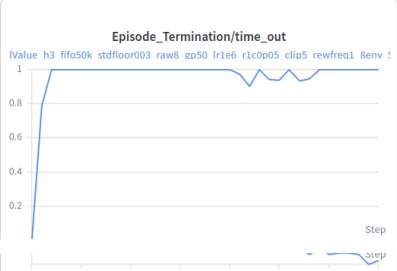
Episode_Reward 8

7-8 of 8





Episode_Termination 2

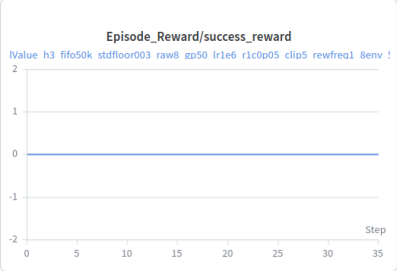


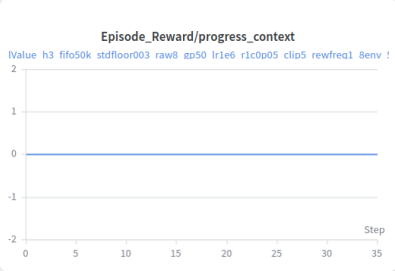


Env 11

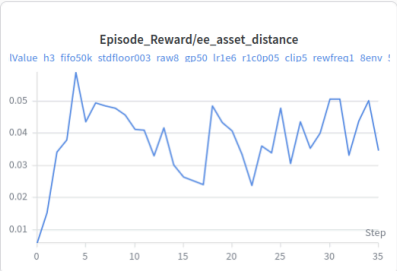
Episode_Reward 8

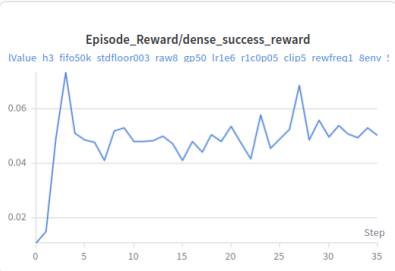
1-6 of 8

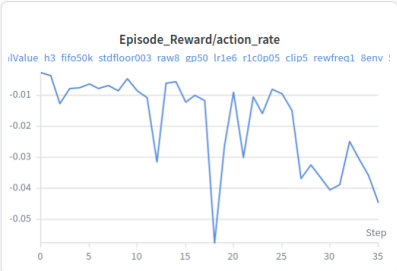








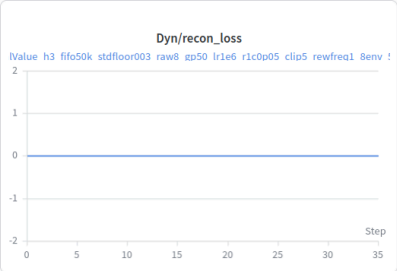


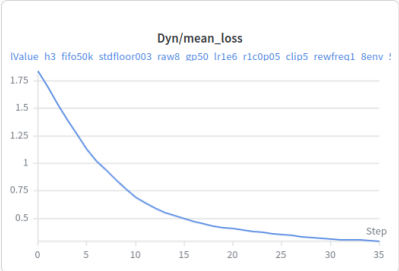


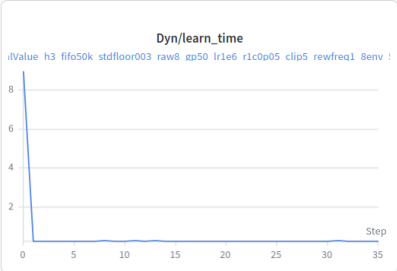
Data 2

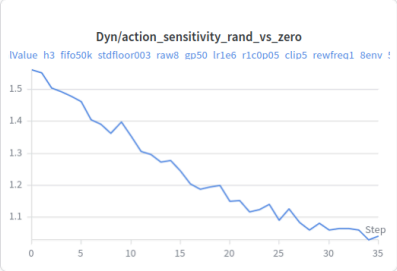
Dyn 5

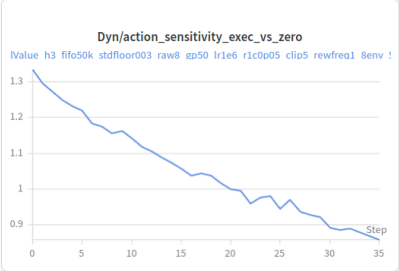
1-5 of 5



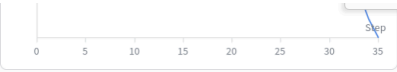


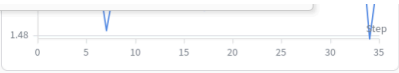






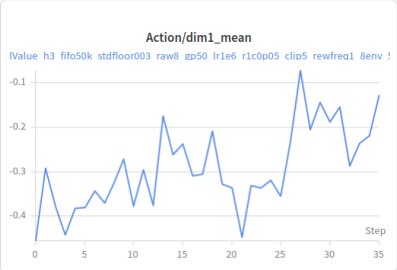
Env 11

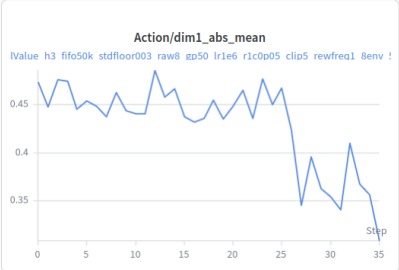




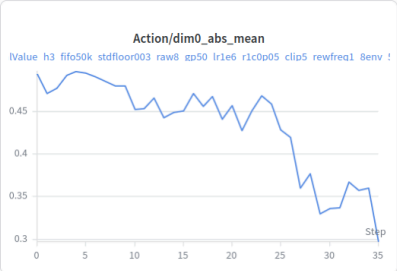
Action 30

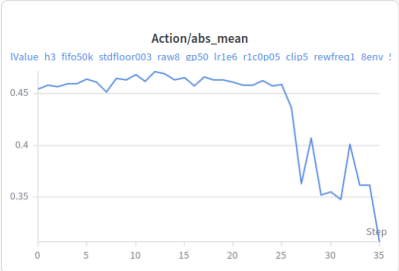
25-30 of 30



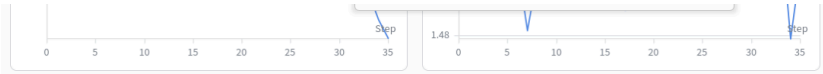




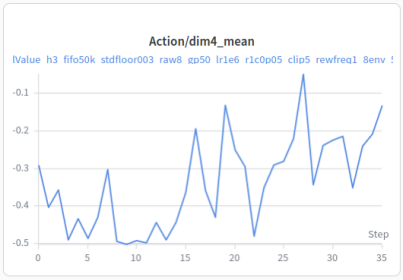


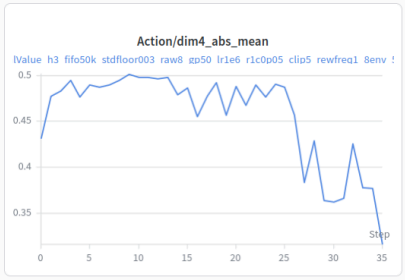


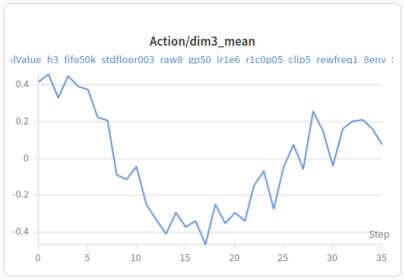


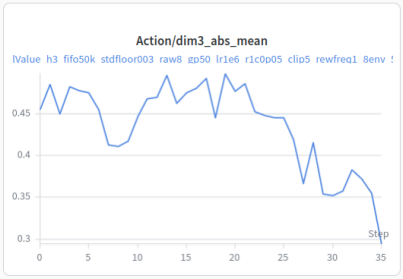


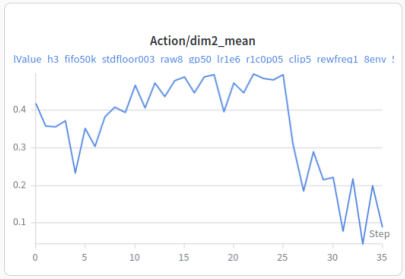
Action 30 19-24 of 30







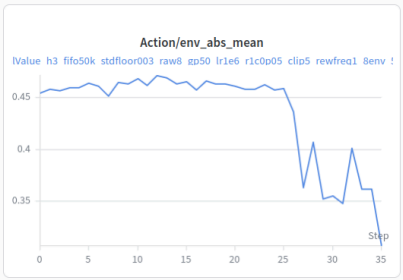


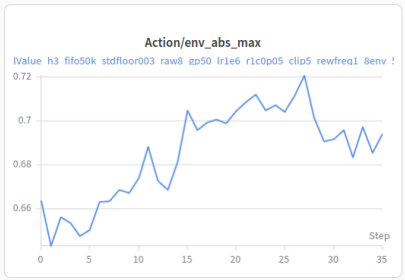


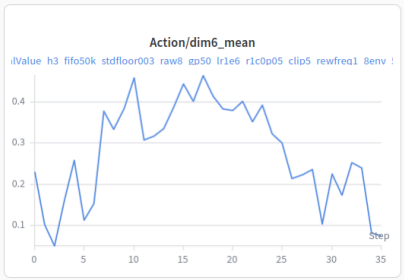


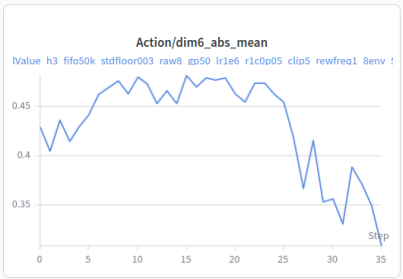


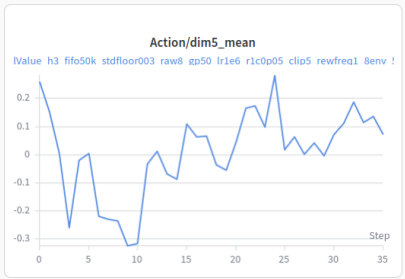
Action 30 13-18 of 30

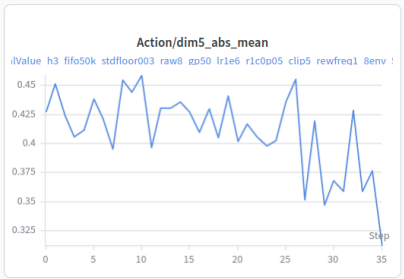


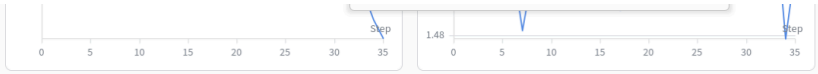




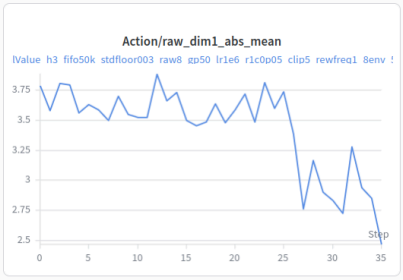


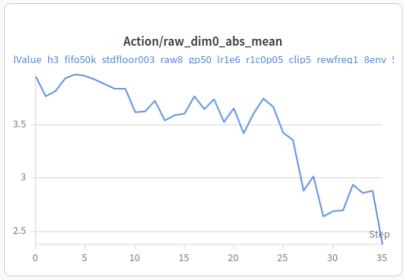


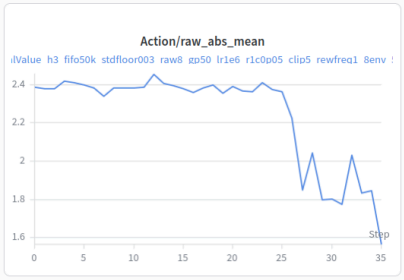


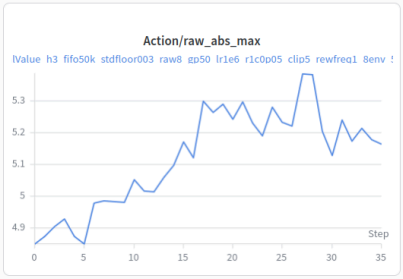


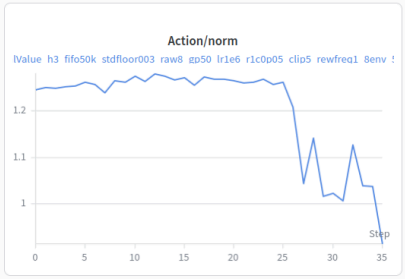
Action 30 7-12 of 30

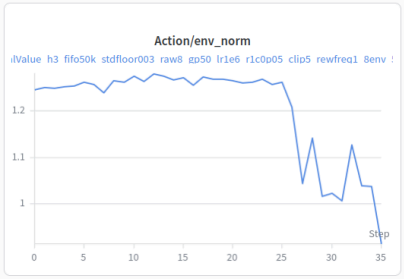


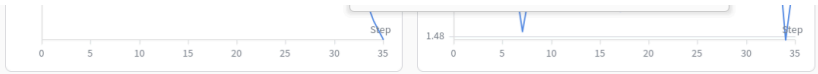




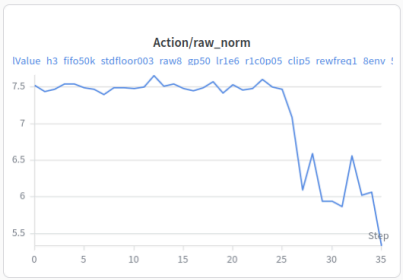


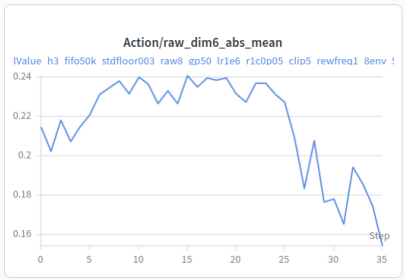


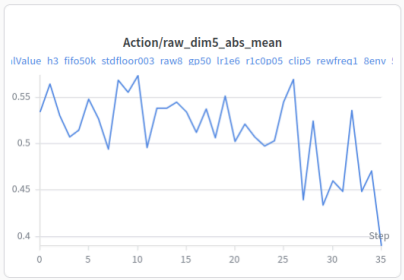


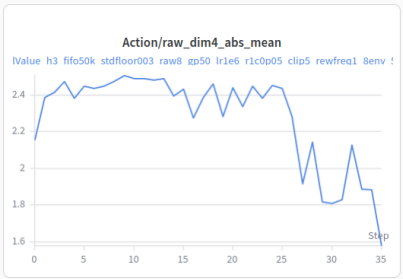


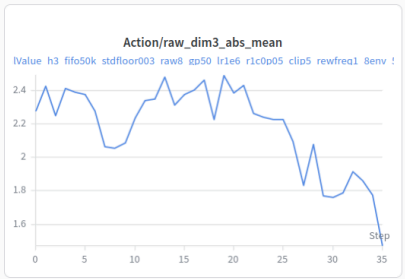
Action 30 1-6 of 30

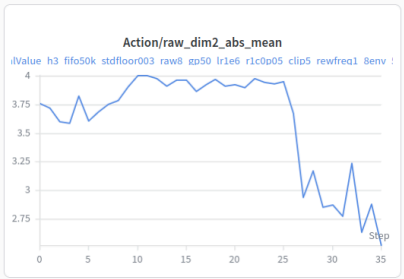


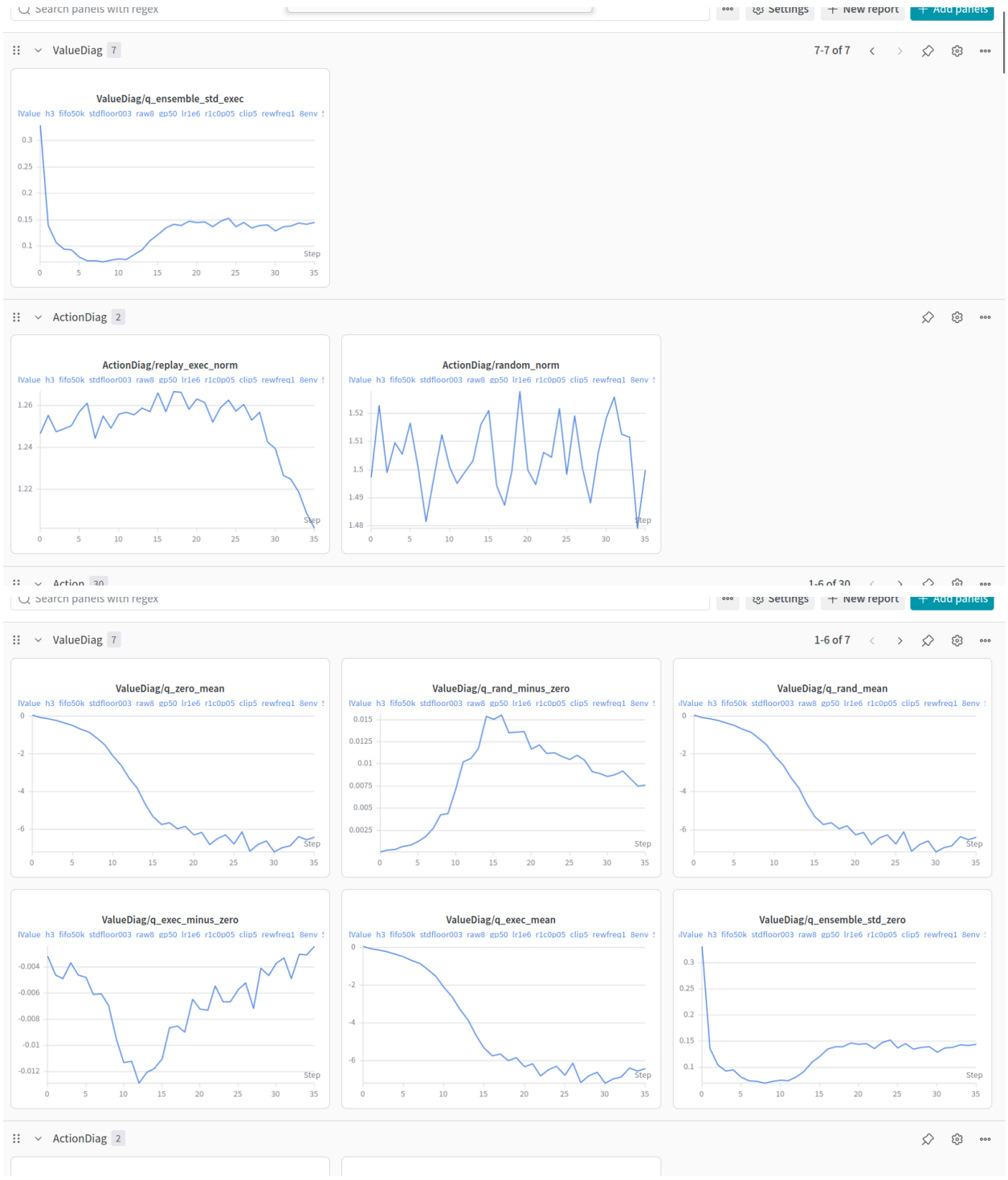












深入的根据算法训练逻辑分析这次的现象，证明了什么，现在这个实验结果把整个训练算法的逻辑链条推到了哪一步？

6:38 AM

这次实验很有信息量。它把问题链条进一步往后推了：不是 terminal value 导致的，也不是 horizon=7 的长程 rollout 才导致的；即使 h=3、terminal value=0，reward 已经能分开 demo/replay，planner 仍然没有找到能带来真实任务进展的动作。

核心结论：现在失败点在 “reward 学到了判别，但这个 reward 在 planner 可达的动作/状态分布上没有形成可用控制梯度”。也就是说算法链条走到：

`demo/replay -> reward discriminator` 这一步基本能工作

`dynamics supervised prediction` 表面也在收敛

但 `planner`用`dynamics`滚动动作 `->` 找到高`reward`轨迹 `->` 执行动作 `->` 环境真实进展 这一步断了。

这次证明了什么

1. value 不是主因
   这次 `MPPI/Average Value=0`、`Max/Min Value=0`，说明 terminal value 已经完全拿掉了。但 MPPI 仍然 `Average Reward` 下降、`Min Return` 变差、success/progress 为 0。所以之前 value 爆炸/误导是放大器，不是根因。
2. horizon 不是主因
   h 从 7 降到 3，短程模型误差应该明显减轻；如果主要问题是长 horizon dynamics compounding error，这次应该明显稳定。但结果仍然没有任务进展，说明不是“滚太远导致模型漂了”这么简单。
3. reward 能分开 demo 和 replay
   `mean_demo_reward` 上升到约 3，`mean_gen_reward` 下降到 -5；`disc_acc` 到 0.9 左右，`disc_acc_expert` 也高。说明 reward/discriminator 不瞎，它确实认为 demo 比 replay 好。
4. 但 reward 不可控
   最关键现象是：`MPPI/Max Reward` 有时能冲正，但 `Average Reward` 仍然整体下降，`Mean Policy Influence` 仍然约 0.001~0.002，非常小。
   这说明 MPPI 采样里偶尔有高 reward 假轨迹，但这些轨迹没有稳定改变执行策略。planner 不是在“逐步朝 demo 走”，而是在采样噪声里偶尔碰到 reward 高点。
5. 真实环境指标完全没跟上
   `success_reward=0`，`progress_context=0`，`ee_asset_distance` 没稳定下降，绝大多数 `time_out=1`。这直接说明：reward 曲线变好不是任务真的变好，而是 reward 在模型/latent 空间里产生了可利用但不可执行的方向。
6. 动作幅度不是太小
   这次 raw action 一开始很大，`raw_norm`、`env_norm` 都不小，后期还下降。这和之前“动作特别小”不同。现在不是“动不了”，而是“动了也不是有效动作”。所以问题从 “action scale 太小” 被排除了很多。

本质问题

现在最像的是：dynamics latent 训练目标和 reward/planning 所需的控制目标不一致。

代码逻辑上，dynamics 是学：

`z_t + action -> z_{t+1}`

reward 是判别：

`z / transition` 是否像 demo

MPPI 是在 learned dynamics 里试动作，选 predicted reward 高的动作。

问题是，如果 latent dynamics 对 action 的可控性不准，或者 encoder latent 没有把“任务进展方向”编码成平滑可规划的几何，MPPI 就会出现现在这种情况：

- reward 在 demo/replay batch 上能判别；
- dynamics loss 能下降；
- 但对 planner 来说，高 reward 区域不是一条可执行路径，而是孤立/虚假的 latent 区域；
- MPPI 找不到能稳定推进真实状态的动作；
- policy influence 接近 0，说明 policy sampler 学不到 planner 的有效选择。

这次把训练链条推到哪一步
已经推到：
reward/value 超参不是第一矛盾；下一步应该检查 planner 的动作分布和 dynamics 的 action-conditioning 是否真的能预测“执行动作后的任务相关变化”。

更具体地说，下一步不该继续调 gp/r1/h/value，而应该做一个诊断实验：

1. 固定当前 S19 配置。
2. 加/看 `z_next(action) - z_next(zero)` 的任务相关维度变化，尤其和 demo transition direction 的 cosine similarity。
3. 用真实环境 rollout 检查：MPPI 选中的 top-k action 在真实 env 里是否让 `ee_asset_distance` 或 dense_success_reward 变好。
4. 如果 top-k predicted reward 高，但真实 env 没进展，那就是 model/planner mismatch。
5. 如果 top-k predicted reward 本身也不稳定，那是 reward landscape 对 planner 不可用。

我现在的判断：下一步最该试的是“让 planner 更贴近 replay/expert action manifold”，而不是继续增强 reward。比如降低采样噪声/动作上限，或者给 MPPI 加 action prior/BC prior 权重，让它别在 dynamics 不可靠区域里找 reward 高点。现在 reward 已经够强，再强只会更容易产生不可执行的假高分。

好吧你帮助我完成下一步实验，但是我rollout渲染有问题，只能根据跑完的checkpoint回看，还有办法搞清楚吗
9:41 AM

有办法搞清楚，而且现在最该做的不是依赖 rollout 实时渲染，而是把 checkpoint 当成“可复盘的实验样本”。

我建议下一步这样做：

1. 继续跑下一步实验，但每个 checkpoint 固定保存可回放信息
   训练时除了正常存 checkpoint，还要存：
   `policy checkpoint`、`reward checkpoint`、`value checkpoint`、normalizer stats、当前 replay 的一小批 sample indices/ages。

这样即使 rollout 渲染坏了，我们也能离线问：这个 policy 到底会输出什么动作、reward/value 到底在给什么信号。

2. 加一个 offline checkpoint evaluator
   对每个 checkpoint 做三类检查，不需要实时渲染：
   - 对 demo states、replay states、zero action、random action、policy action 打分。
   - 记录 `Q(policy)-Q(zero)`、`R(policy)-R(zero)`、`Q(random)-Q(zero)`。
   - 记录 policy 输出动作的 raw/env norm、每维均值、clip 比例。
3. 这能直接回答现在最核心的问题：MPPI/BC 是不是在跟一个错误的 reward/value 梯度走。
4. 回放 checkpoint 生成视频
   如果环境只能从 checkpoint 回看，那就写/用一个 eval 脚本：加载指定 checkpoint，固定 seed，跑 5-10 条 episode，保存视频和 action log。这样不需要训练中渲染，跑完再看就行。
5. 下一步实验目标
   现在链条已经推到：reward/value/MPPI 三者里，`value` 很容易变成负向地形，policy/MPPI 在追一个不可靠的 planning score。所以下一个实验要隔离 value：
   - `h=3`
   - `value_return_type=zero/none` 或者 MPPI planning 暂时只用 reward，不用 terminal value
   - `gp=50`
   - `r1=0.05`
   - `reward_center_for_planning=True`
   - 其他保持：FIFO、`std_floor=0.03`、`reward_update_every=1`

如果这个实验里 `MPPI/Average Reward` 仍然下降，说明 reward 本身在 replay/policy 区域给了坏方向；如果 reward 不降但任务仍然不动，说明 policy/action wrapper 或 env action scaling 是主问题；如果 reward 稳了、action 也合理但 value 一接回来就崩，value bootstrap 就是根因。

所以：可以搞清楚，而且比实时渲染更可靠。下一步我会优先做 checkpoint offline evaluator + 跑 “disable/neutralize value influence” 这个实验。

Forked from conversation
好吧跑实验吧